C&H Technology

Version 1.5

# Smart Kit

## User's Manual

**May 1, 2000**

# Table of Contents

# Part I

## Smart Kit
## ( SK-1000,SK-820,SK-420 )

# 1. Smart Kit

## Introduction

Smart Kit is the In-circuit Emulator which is designed for a Samsung Micro-controller Development System (MDS) to make the process of implementation and check out of SAM4 and SAM8 micro-controller family applications as fast and easy as possible.
Smart Kit has three kinds of models, SK-420 for SAM4 family, SK-820 for SAM8 family, and SK-1000 for SAM4/SAM8 family combined model.

## About Smart Kit

Smart Kit is designed for total solution tool of SAM4 and SAM8 family that consists of Main body (SK-1000, SK-820, SK-420), TARGET board and OTP programming adapter socket. Also, you can change a TARGET board depending on the selected micro-controller in your project.

## Features

- **All MCU families for SAM4(SK-1000, SK-420) or SAM8(SK-1000, SK-820) are available**
  By changing Target board, Main body can be used for all SAM4 or SAM8 families of the Samsung MCU that is corresponding to core CPU type.

- **RS-232C interface**
  This is a general-used communication prototype.

- **CPU oscillation clock frequency are changeable**
  You can change CPU oscillation frequency of the target board from 400KHz to 100MHz by software selection in the setup menu of Debugger (Smart Studio).

- **Execution speed:**
  - SAM4: 30 KHz~10MHz
  - SAM8: 30 KHz~25MHz

- **Operating voltage:**
  - 15VDC Power Adapter (AC input: 110/220 V ) for Smart Kit
  - To User system including the Target board, 300mA with programmable operating voltage 3.0~5.0V is available

## Debugging Function

- Download/Upload and code edit function
- Register monitoring or editing function
- Single step execution
  - Execution by single step
  - Continuous single step

- Address(ROM) breakpoint run
  - Up to 255 countable breakpoints can be set for an instruction
  - Unlimited breakpoints can be set in program size
    (SAM4: 32Kbyte, SAM8: 64Kbyte)
  - Continuous breakpoint run

- Real-time trace run
  - Address trigger can be set in full range space
  - 16K step trigger memory depth (Trigger depth is adjustable)
  - Trace capture can be started at any address

- Timer function
  - Execution time measurement (Interval time between trigger and stop flags)
  - Fetch cycle measurement
  - Instruction execution counter

- RAM(Register data) breakpoint run
  - Read, Write, Read/Write condition selectable
  - Continuous breakpoint run

- Real-time RAM(Register data) display
  - 100ms refresh for CRT display

- Run History
  - Run history can be saved in any execution mode
    (Run, Single step, and Breakpoint run)

- Skip Program execution
  - Skip to address
  - Skip to cursor
  - Step over subroutine call

- OTP/MTP Programming function
  - Serial and Parallel type OTP, or MTP (Flash type ROM)

- Watch register window
  - Edit or View a watch window for customer selected registers

## System Configuration (Smart Kit)

Smart Kit is designed for total solution tool that consists of MAIN body, TARGET board and OTP /MTP programming socket adapter. The Target board and the adapter socket are optional depending on the target CPU.

**Figure 1.1 Smart Kit System Configuration**

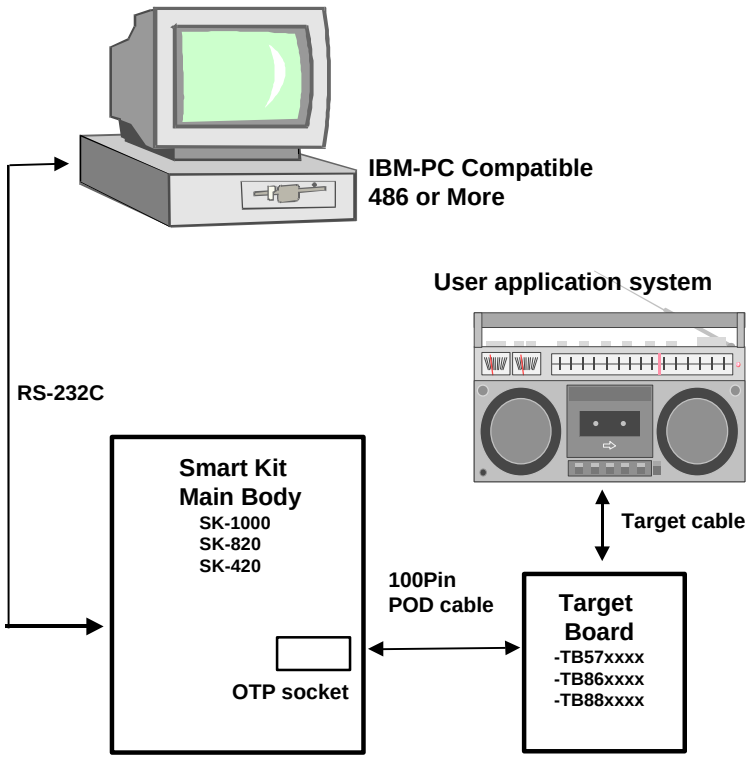
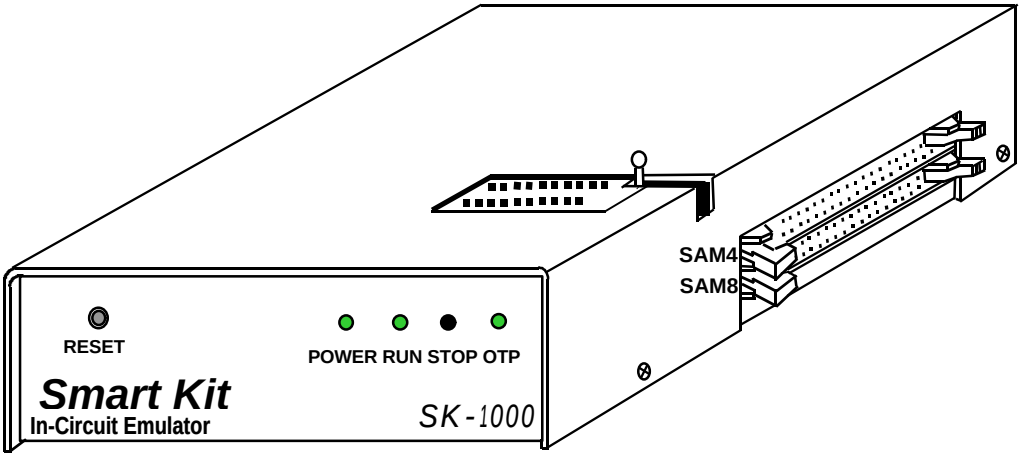
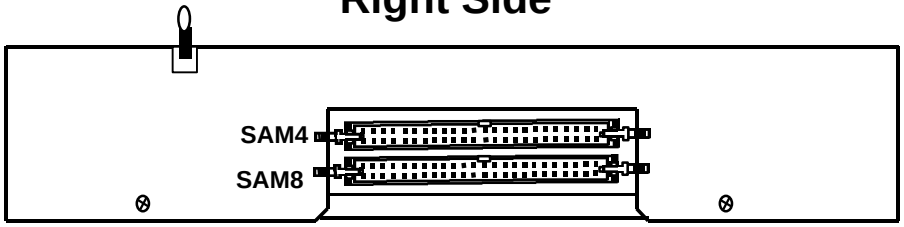


**Figure 1.2 Smart Kit Main Body( SK-1000 )**

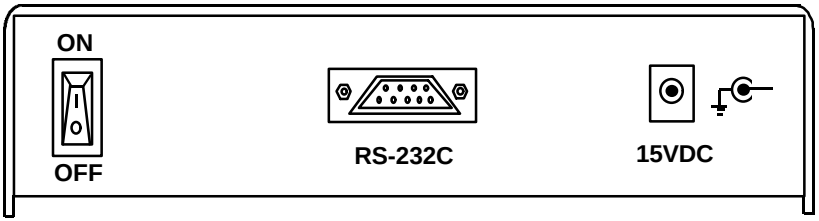
## Top view




## Right Side

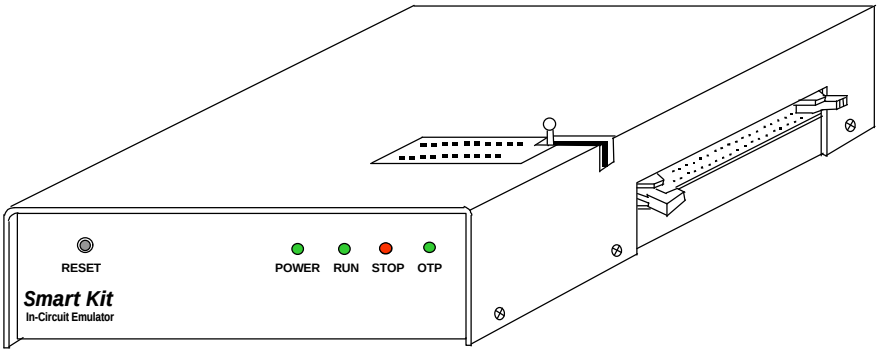



## Rear Side

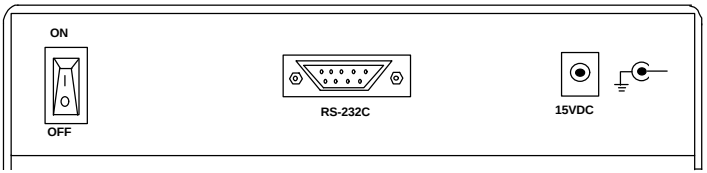


## Smart Kit Main Body( SK-820, SK-420 )

### Top view




Rear Side



## 100-Pin POD Connector Pin assignment (SAM4)

| Pin No. | Name | I/O | Description | Pin No. | Name | I/O | Description |
|---|---|---|---|---|---|---|---|
| 1 | IB7 | O | Instruction Data 7 | 26 | N.C | - | |
| 2 | IB6 | O | Instruction Data 6 | 27 | GND | - | |
| 3 | IB5 | O | Instruction Data 5 | 28 | GND | - | |
| 4 | IB4 | O | Instruction Data 4 | 29 | GND | - | |
| 5 | IB3 | O | Instruction Data 3 | 30 | GND | - | |
| 6 | IB2 | O | Instruction Data 2 | 31 | GND | - | |
| 7 | IB1 | O | Instruction Data 1 | 32 | GND | - | |
| 8 | IB0 | O | Instruction Data 0 | 33 | GND | - | |
| 9 | PC0 | I | ROM Address 0 | 34 | GND | - | |
| 10 | PC1 | I | ROM Address 1 | 35 | GND | - | |
| 11 | PC2 | I | ROM Address 2 | 36 | GND | - | |
| 12 | PC3 | I | ROM Address 3 | 37 | GND | - | |
| 13 | PC4 | I | ROM Address 4 | 38 | GND | - | |
| 14 | PC5 | I | ROM Address 5 | 39 | GND | - | |
| 15 | PC6 | I | ROM Address 6 | 40 | GND | - | |
| 16 | PC7 | I | ROM Address 7 | 41 | GND | - | |
| 17 | DB7 | I | Internal Data Bus 7 | 42 | GND | - | |
| 18 | DB6 | I | Internal Data Bus 6 | 43 | GND | - | |
| 19 | DB5 | I | Internal Data Bus 5 | 44 | GND | - | |
| 20 | DB4 | I | Internal Data Bus 4 | 45 | GND | - | |
| 21 | DB3 | I | Internal Data Bus 3 | 46 | N.C | - | |
| 22 | DB2 | I | Internal Data Bus 2 | 47 | N.C | - | |
| 23 | DB1 | I | Internal Data Bus 1 | 48 | GND | - | |
| 24 | DB0 | I | Internal Data Bus 0 | 49 | GND | - | |
| 25 | N.C | - | | 50 | GND | - | |

## 100-Pin POD Connector Pin assignment (SAM4)

| Pin No. | Name | I/O | Description | Pin No. | Name | I/O | Description |
|---|---|---|---|---|---|---|---|
| 51 | GND | - | | 76 | EVA_RD | I | I/O read |
| 52 | GND | - | | 77 | S3 | I | Instruction Fetch 3 |
| 53 | GND | - | | 78 | SKIP | I | Instruction SKIP |
| 54 | GND | - | | 79 | T_RST | O | Target RESET |
| 55 | N.C | - | | 80 | N.C | - | |
| 56 | N.C | - | | 81 | DIS_INT | O | Disable Interrupt |
| 57 | GND | - | | 82 | DISWDT | O | Disable WDT |
| 58 | GND | - | | 83 | PC8 | I | ROM Address 8 |
| 59 | N.C | - | | 84 | PC9 | I | ROM Address 9 |
| 60 | N.C | - | | 85 | PC10 | I | ROM Address 10 |
| 61 | VDD | O | +3.0~5.0V | 86 | PC11 | I | ROM Address 11 |
| 62 | VDD | O | +3.0~5.0V | 87 | PC12 | I | ROM Address 12 |
| 63 | VDD | O | +3.0~5.0V | 88 | PC13 | I | ROM Address 13 |
| 64 | GND | - | | 89 | PC14 | I | ROM Address 14 |
| 65 | GND | - | | 90 | S2 | I | Instruction Fetch 2 |
| 66 | GND | - | | 91 | EOI | I | End of Instruction |
| 67 | GND | - | | 92 | N.C | - | |
| 68 | GND | - | | 93 | EVA_WR | I | I/O Write |
| 69 | GND | - | | 94 | SAMB0 | I | Bank Selection 0 |
| 70 | GND | - | | 95 | SAMB1 | I | Bank Selection 1 |
| 71 | GND | - | | 96 | SAMB2 | I | Bank Selection 2 |
| 72 | GND | - | | 97 | SAMB3 | I | Bank Selection 3 |
| 73 | N.C | - | | 98 | N.C | - | |
| 74 | GND | - | | 99 | CLOCK | O | Main Clock for T/B |
| 75 | GND | - | | 100 | N.C | - | |

# 100-Pin POD Connector Pin assignment (SAM8)

| Pin No. | Name | I/O | Description | Pin No. | Name | I/O | Description |
|---|---|---|---|---|---|---|---|
| 1 | PC0 | I | ROM Address 0 | 26 | N.C | - | |
| 2 | RIND | I | Register Indirect Access | 27 | PC1 | I | ROM Address 1 |
| 3 | BANK | I | Bank Select | 28 | PC2 | I | ROM Address 2 |
| 4 | PA0 | I | Page Address 0 | 29 | PC3 | I | ROM Address 3 |
| 5 | PA1 | I | Page Address 1 | 30 | PC4 | I | ROM Address 4 |
| 6 | PA2 | I | Page Address 2 | 31 | PC5 | I | ROM Address 5 |
| 7 | PA3 | I | Page Address 3 | 32 | PC6 | I | ROM Address 6 |
| 8 | RA0 | I | Register file Address 0 | 33 | PC7 | I | ROM Address 7 |
| 9 | RA1 | I | Register file Address 1 | 34 | PC8 | I | ROM Address 8 |
| 10 | RA2 | I | Register file Address 2 | 35 | PC9 | I | ROM Address 9 |
| 11 | RA3 | I | Register file Address 3 | 36 | PC10 | I | ROM Address 10 |
| 12 | RA4 | I | Register file Address 4 | 37 | PC11 | I | ROM Address 11 |
| 13 | RA5 | I | Register file Address 5 | 38 | PC12 | I | ROM Address 12 |
| 14 | RA6 | I | Register file Address 6 | 39 | PC13 | I | ROM Address 13 |
| 15 | RA7 | I | Register file Address 7 | 40 | PC14 | I | ROM Address 14 |
| 16 | DB0 | I | Register file Data 0 | 41 | PC15 | I | ROM Address 15 |
| 17 | DB1 | I | Register file Data 1 | 42 | GND | - | |
| 18 | DB2 | I | Register file Data 2 | 43 | N.C | - | |
| 19 | DB3 | I | Register file Data 3 | 44 | T_RST | O | Target RESET |
| 20 | DB4 | I | Register file Data 4 | 45 | N.C | - | |
| 21 | DB5 | I | Register file Data 5 | 46 | VDD | O | +3.0~5.0V |
| 22 | DB6 | I | Register file Data 6 | 47 | VDD | O | +3.0~5.0V |
| 23 | DB7 | I | Register file Data 7 | 48 | N.C | - | External Event Trig |
| 24 | DIS_INT | O | Disable Interrupt | 49 | N.C | - | |
| 25 | N.C | - | | 50 | REGWR | I | Register file Write |

## 100-Pin POD Connector Pin assignment (SAM8)

| Pin No. | Name | I/O | Description | Pin No. | Name | I/O | Description |
|---|---|---|---|---|---|---|---|
| 51 | N.C | - | | 76 | N.C | - | |
| 52 | N.C | - | | 77 | N.C | - | |
| 53 | N.C | - | | 78 | N.C | - | |
| 54 | N.C | - | | 79 | N.C | - | |
| 55 | N.C | - | | 80 | N.C | - | |
| 56 | N.C | - | | 81 | N.C | - | |
| 57 | GND | - | | 82 | N.C | - | |
| 58 | GND | - | | 83 | N.C | - | |
| 59 | N.C | - | | 84 | N.C | - | |
| 60 | N.C | - | | 85 | PUR | O | 10Kohm pull-up |
| 61 | N.C | - | | 86 | N.C | - | |
| 62 | N.C | - | | 87 | N.C | - | |
| 63 | N.C | - | | 88 | N.C | - | |
| 64 | GND | - | | 89 | N.C | - | |
| 65 | GND | - | | 90 | CLOCK | O | Main Clock for T/B |
| 66 | GND | - | | 91 | AS | I | Address Strobe |
| 67 | GND | - | | 92 | DS | I | Data Strobe |
| 68 | REGRD | I | Register file Read | 93 | IB7 | O | Instruction Data 7 |
| 69 | EOI | I | End of Instruction | 94 | IB6 | O | Instruction Data 6 |
| 70 | N.C | - | | 95 | IB5 | O | Instruction Data 5 |
| 71 | N.C | - | | 96 | IB4 | O | Instruction Data 4 |
| 72 | N.C | - | | 97 | IB3 | O | Instruction Data 3 |
| 73 | VDD | O | +3.0~5.0V | 98 | IB2 | O | Instruction Data 2 |
| 74 | DM | I | Data Memory | 99 | IB1 | O | Instruction Data 1 |
| 75 | R/W | I | Read/Writ | 100 | IB0 | O | Instruction Data 0 |

## 40-Pin Socket Pin assignment (OTP/MTP)

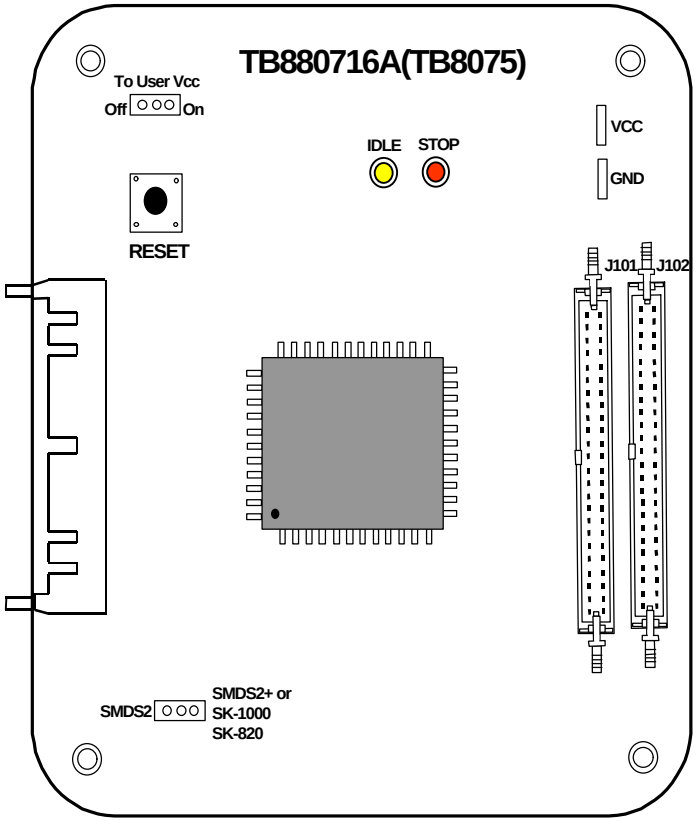
| Pin No. | Name | I/O | Description | Pin No. | Name | I/O | Description |
|---|---|---|---|---|---|---|---|
| 1 | SCLK | O | Cock (Serial) | 40 | RESET | O | Chip RESET (Serial) |
| 2 | SDAT | I/O | Data (Serial) | 39 | N.C | - | |
| 3 | VPP | O | VPP (Serial) | 38 | N.C | - | |
| 4 | VDD | O | VDD (Serial) | 37 | MEM/REG | O | M/R Select (Parallel) |
| 5 | A15 | O | Address 15 (Parallel) | 36 | N.C | - | |
| 6 | A14 | O | Address 14 (Parallel) | 35 | N.C | - | |
| 7 | VPP | O | VPP (Parallel) | 34 | VDD | O | VDD (Parallel) |
| 8 | A12 | O | Address 12 (Parallel) | 33 | PGMB | O | PGM (Parallel) |
| 9 | A7 | O | Address 7 (Parallel) | 32 | A13 | O | Address 13 (Parallel) |
| 10 | A6 | O | Address 6 (Parallel) | 31 | A8 | O | Address 8 (Parallel) |
| 11 | A5 | O | Address 5 (Parallel) | 30 | A9 | O | Address 9 (Parallel) |
| 12 | A4 | O | Address 4 (Parallel) | 29 | A11 | O | Address 11 (Parallel) |
| 13 | A3 | O | Address 3 (Parallel) | 28 | OEB | O | Out Enable (Parallel) |
| 14 | A2 | O | Address 2 (Parallel) | 27 | A10 | O | Address 10 (Parallel) |
| 15 | A1 | O | Address 1 (Parallel) | 26 | GND | - | Ground (VSS) |
| 16 | A0 | O | Address 0 (Parallel) | 25 | D7 | I/O | Data 7 (Parallel) |
| 17 | D0 | I/O | Data 0 (Parallel) | 24 | D6 | I/O | Data 6 (Parallel) |
| 18 | D1 | I/O | Data 1 (Parallel) | 23 | D5 | I/O | Data 5 (Parallel) |
| 19 | D2 | I/O | Data 2 (Parallel) | 22 | D4 | I/O | Data 4 (Parallel) |
| 20 | GND | - | Ground (VSS) | 21 | D3 | I/O | Data 3 (Parallel) |

## RS-232C Interface Signal (Smart Kit side)

| Pin No. | Signal Name | I/O | Description |
|---|---|---|---|
| 1 | N.C | - | |
| 2 | TXD | O | Transmitted Data |
| 3 | RXD | I | Received Data |
| 4 | N.C | - | |
| 5 | GND | - | Signal Ground |
| 6 | N.C | - | |
| 7 | N.C | - | |
| 8 | N.C | - | |
| 9 | N.C | - | |

## Figure 1.3 Target Board for Smart Kit

**[ Target Board ]**



# Part II

## Smart Studio
### (skStudio.exe - V1.30)

# 2. Smart Studio

## Introduction

Smart studio is the Host Interface for debugging using Smart Kit (In-circuit emulator). It is powerful, fast, efficient and user-friendly host interface program, and you can easily debug any application software for SAM4 and SAM8 micro controller families.

## What's in Smart Studio

Smart Studio includes the following features:

- **Creating and editing source file**
  Smart Studio offers you an editor to create and edit your application program source files. Smart editor is supported by default, but you can customize the your desired editor by your own selection.

- **Assembling a program**
  Smart Studio supports SAMA and SASM (Re-locatable assembler). SASM57, SASM86 and SASM88 are the re-locatable assembler for SAM4, and SAM8 to assemble your application source file to object code.

- **Debugging a program**
  You can run your application program using reset, normal run, single step, breakpoint, trace run, timer run, and run history very quickly. Also, you can debug your code using uploading and register inspection.

- **Programming an OTP and MTP(Flash type Program memory)**
  Smart Studio offers OTP/MTP programmer to program SAMSUNG OTP or MTP.

- **ROM Code Generator**
  You can generate a ROM code for masking release using ROM Code generator.

# *Chapter 1* Installing Smart Studio

Smart Studio comes with an automatic installation program called Setup.exe on the disk #1. Because of file compression techniques, you must use this program; you can not just copy the Smart Studio files onto your hard disk. Setup.exe automatically copies and decompresses the Smart Studio. For your reference, the Setup.lst file on the installation disk includes a list of the distribution files.

This chapter explains how to setup Smart Studio on your hard disk. Once you have installed Smart Studio, you are ready to read other chapters to start Smart Studio properly.

## Using Setup

Setup.exe creates directories when needed and transfers files from your distribution disks to your hard disk. Its actions are easy to understand. The following text explains you all you need to know.

## System Requirements

Smart Studio requires the following:

- ◆ IBM PC 486/pentium machine, or 100% compatible computer.
- ◆ Color Monitor with a VGA card.
- ◆ Win95 operating system or later.
- ◆ Over 32MB Main Memory
- ◆ 3.5" Floppy Disk Driver.
- ◆ A hard disk drive (At least 10MB of disk space is required).
- ◆ RS232C Serial interface; COM1, COM2, COM3, or COM4 port.

## About the Installation Program

The Setup.exe program copies the Smart Studio from the distribution floppy disks to the directory you specify.

## Starting the Installation program

Begin with the Setup Disk #1.

To setup Smart Studio:

1. Insert the setup disk#1 into the drive A: or B:
   Run Setup.exe file using start button or explorer.

2. Please continue according to setup guide message.

3. Please make Smart Studio icon on the Window95 screen for convenience.

# *Chapter 2* Smart Studio Basics

Smart Studio has everything you need to write, edit, compile and debug your application programs. It provides:

- Multiple, movable, resizable windows
- Mouse support and dialog boxes
- On-line Help
- Built-in assembler (SAMA, SASM)
- Simple editor and viewer
- Win95 calculator
- OTP/MTP programmer
- ROM code generator

This chapter explains the following topics:

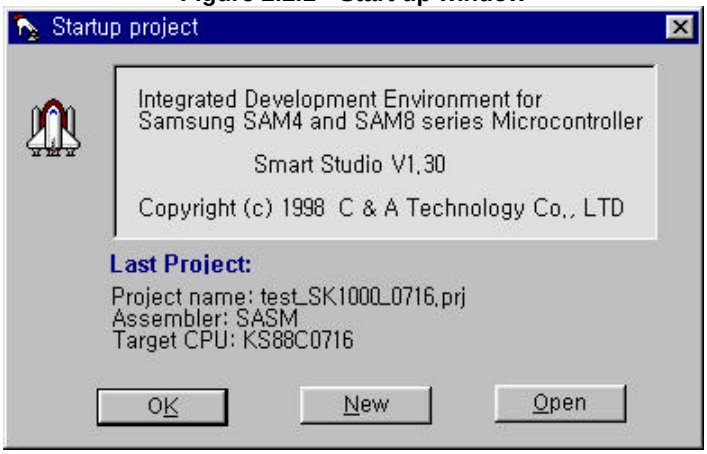
- Starting and exiting from the Smart Studio
- Using Smart Studio components

## Starting and Exiting from the Smart Studio

### Starting the Smart Studio

To start the Smart Studio, click skStudio icon on the Win95/98 screen or double click skStudio.exe file using window explorer.

**Figure 2.2.1 Start up window**



If your project is already exist, above window will be appeared on the main window.

**[ OK button ]**
If you want to open your last project, click this button.

**[ New button ]**
If you want to set up a new project, click this button. It displays the new project dialog box. In this dialog box, you can create a new project you want to debug. For further information, please refer to Project|New menu operation description in the chapter3.

**[ Open button ]**
If you want to open your existing project, click this button. it displays the open project dialog box. In this dialog box, you can select a project you want to debug or open project. For further information, please refer to Project|Open menu operation description in the chapter3.

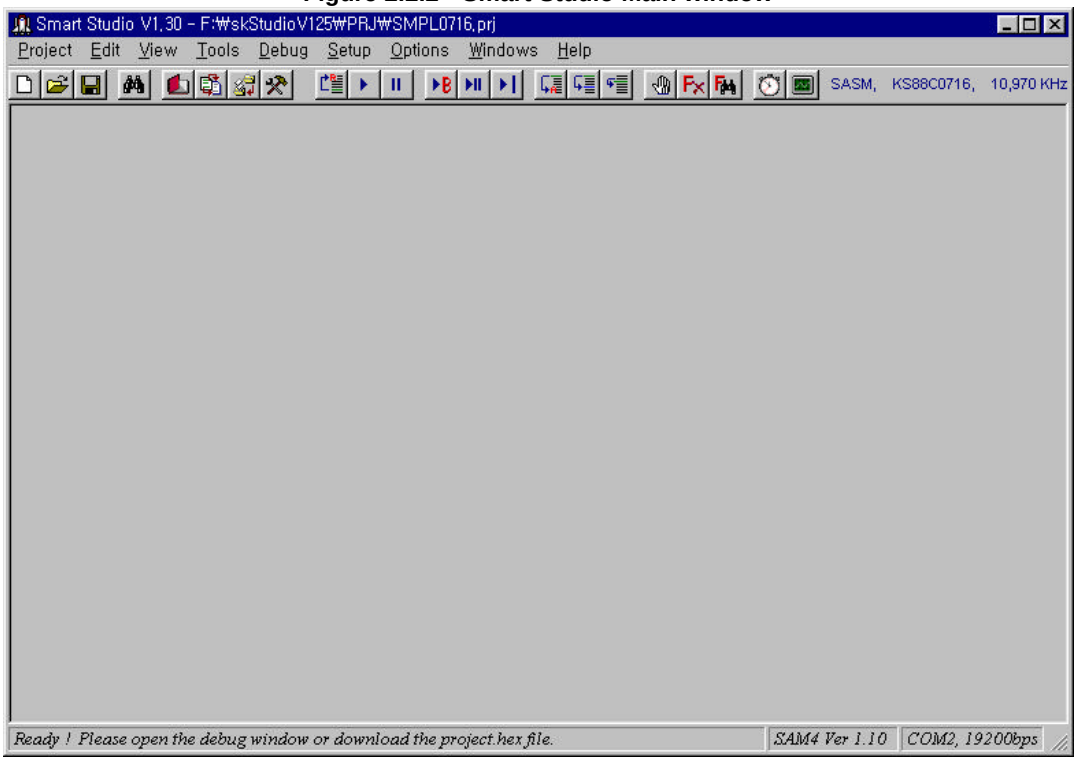
**Exiting from Smart Studio**

Choose Project|Exit to exit the Smart Studio completely; you have to click skStudio icon again to reenter it. You'll be prompted to save your project before exiting.

## Smart Studio Components

There are four visible components in the Smart Studio: the menu bar at the top line, the tool bar at the second line, the working window area in the middle, and the status line at the bottom of the screen. Many menu items also offer dialog boxes. Although there are several different ways to make selections in the Smart Studio, they access the same commands and dialog boxes. Before we list menu items in the Smart Studio, we'll explain those more general components.

**Figure 2.2.2 Smart Studio Main window**




## The menu bar and menu

The menu bar is your primary access to all the menu commands. If a menu command is followed by an ellipsis (...), choosing the command displays a dialog box. If the command is followed by an arrow (→), the command leads to another menu. If the command has neither an ellipsis nor an arrow, the action is done occurs as soon as you choose the command.

**Here is how you choose the menu command using the keyboard:**

- Press Alt key. This makes the menu bar active; the next thing you type will correspond to the item that is associated with the key on the menu bar. Each key for a menu is highlighted in blue in the menu bar.

- Use the arrow keys to select the menu you want to display. Then press ENTER key. As a shortcut for this step, you can just press the underlined letter of the menu title. For example, when the menu bar is active, press ALT and the underlined letter (such as Alt+E) to display the menu you want.

- Use the arrow keys again to select a command from the menu that you have opened. Then press ENTER key. At this point, Smart Studio either executes the command, displays a dialog box, or displays another menu (Sub menu).

**--- There are two ways to choose commands with a mouse ---**

- Click the desired menu title to display the menu and click the desired command.

- Or, drag straight from the menu title down to the menu command. Release the mouse button on the command you want. ( If you change your mind, just drag off the menu; no command will be chosen.)

**Shortcuts**
Smart Studio provides a number of quick ways to choose menu commands. The click-drag method for mouse users is an example. From the keyboard, you can use a number of keyboard shortcuts (or hot keys) to access the menu bar, choose commands, or work within dialog boxes. You need to hold down Alt key while pressing the underlined letter when moving from an input box to a group of buttons or boxes.

## Smart Studio Windows

Most of what you see and do in the Smart Studio happens in a window. A window is a screen area that you can open, close, move, resize, zoom, tile and overlap.

You can have many windows open in the Smart Studio, but only one window can be active at any time. Any command you choose or text you type generally applies only to the active window. You can spot the active window easily: It's the one with blue colored title bar. The active window has a close box and/or scroll bars. If your windows are overlapping, the active window is always on the top of all the others (The foremost one).

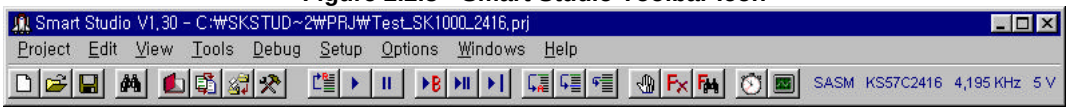
There are several types of windows, but most of them have these items in common:

- a title bar
- a close box
- scroll bars( some case, not available )
- a zoom box( some case, not available )

## The tool bar( Icon )

Smart Studio supports the toolbar function, so you can operate main feature by clicking appropriate icon in the toolbar. Here is what the toolbar looks like:

**Figure 2.2.3 Smart Studio Toolbar Icon**




1. Icon for Project|New menu function
2. Icon for Project|Open menu function
3. Icon for Project|Save menu function
4. Icon for Find text in the debug window
5. Icon for Edit source file menu function
6. Icon for Tool|Assemle|Making menu function
7. Icon for Debug|Downloading menu function
8. Icon for View|Debug window menu function
9. Icon for Debug|Reset target CPU menu function
10. Icon for Debug|Run menu function
11. Icon for Debug|Stop menu function
12. Icon for Debug|ROM break run menu function
13. Icon for Debug|Singlestep run menu function
14. Icon for Debug|Goto cursor menu function
15. Icon for Debug|Skip to address menu function
16. Icon for Debug|Skip to cursor menu function
17. Icon for Debug|Stepover (subroutine call) menu function
18. Icon for Debug|Edit flags menu function
19. Icon for Debug|Clear all flags menu function
20. Icon for Debug|Search flag menu function
21. Icon for Debug|Timer run menu function
22. Icon for Debug|Trace run menu function
23. Assembler of the current project
24. Target MCU of the current project
25. Clock frequency value for target board
26. VDD voltage value for target board

## The status line

The status line appears at the bottom of the screen to:

- Tell you what the program is doing. For example, it displays Running... when Smart Kit is running.
- Display a running error or warning message.
- Offer one-line hints on any selected menu command and dialog box items.

The status line changes as you switch windows or activities. One of the most common status lines is the one you see when you're actually debugging your application program in the debug screen window.

# *Chapter 3* Menu References

These chapters provide a reference to each menu option in the Smart Studio. It is arranged in order of menus as menus appear on the screen, For information on starting and exiting from the Smart Studio and general information on how the Smart Studio works, see Chapter 2.

Example

**Alt + Z**

( This menus Alt+Z is a hot key.)

All menus are accessed by pressing Alt+Z, where Z is the first letter of the menu or underlined letter. For example, the "Debug" menu is pulled down by Alt+D.

## Project

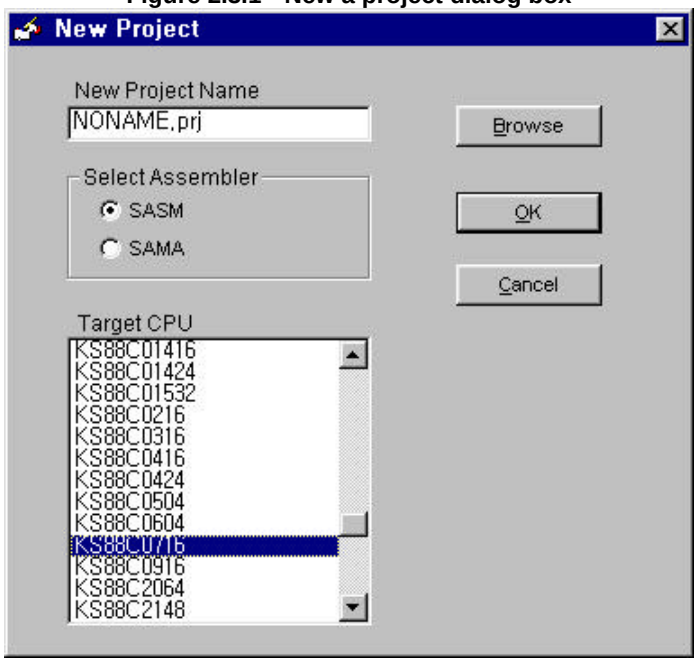
**Alt** + **P**

The Project menu provides choices for creating project, opening project, closing project, saving project and exiting from the Smart Studio.

### Project|New

The Project|New menu displays the new project dialog box. In this dialog box, you can create a new project you want to debug. This menu is supported by shortcut operation using the icon for ‘New project’ in the toolbar.
The New Project dialog box in the Smart Studio looks like this:

**Figure 2.3.1 New a project dialog box**



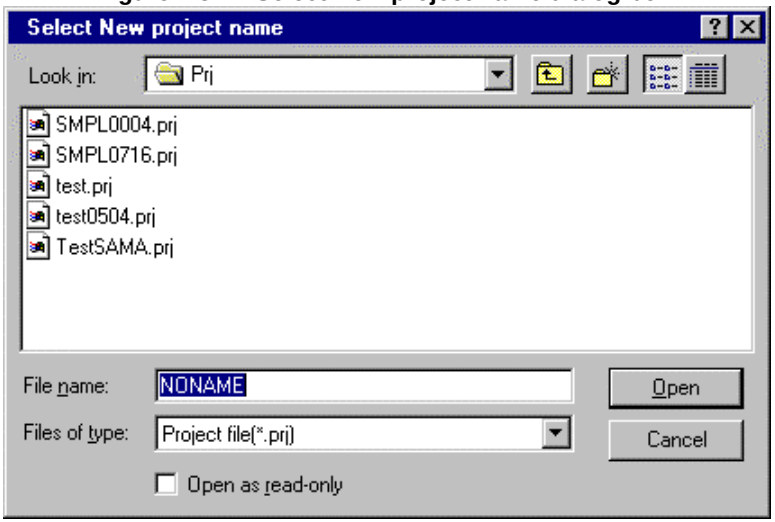

In the New Project dialog box, we can edit project, target and assembler. If this project name exists, the message asking whether to overwrite will be shown. Press the 'Yes' button for overwriting. Pressing the 'No' button will bring out new dialog box which later will be used for creating a new project.

**[ New Project Name text input box ]**
Select a project file. You need to build this project first. Project is a working space environment in which you select target and assembler to start a project; If you want to select or edit a project name you can type file name directly or 'Browse' button to open the File Select dialog box in order to change directory.

If the selected project exists already, this project overwrites or displays the new project dialog box. If not, a extension is setup with this project name. Default file extension is '.prj'. If you do not select a project, default project name is 'NONAME.prj'.

**[ Browse Button ]**
The Browse button opens the Select New project name dialog box. You can select a different directory and an existing project file name, or make a new folder and project file name as new project. Here is what the box looks like:

**Figure 2.3.2 Select New project name dialog box.**



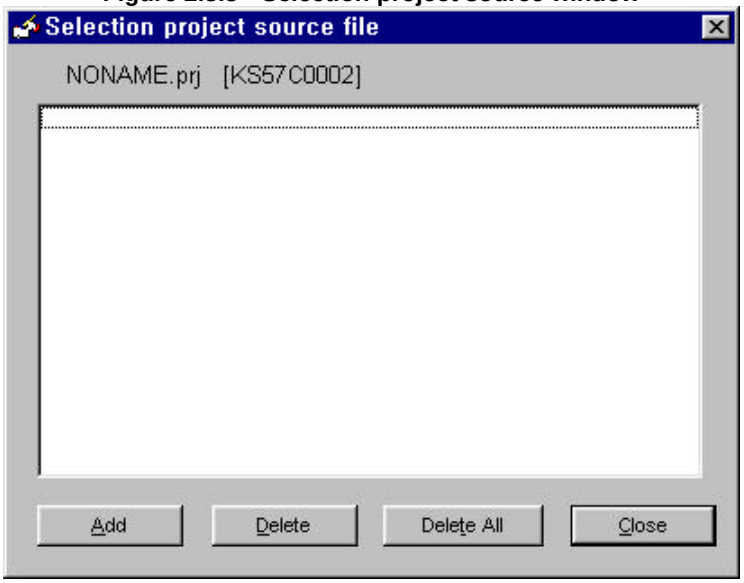

The dialog box contains an input box for file name, a file list, button labeled Open, Cancel. Now you can do any of these actions like file opening method in Win95/98:

- View the contents of different directories(folder) by selecting a directory icon in the dialog box.
- Type in a full file name and choose Open. Open button loads the file into new project.
- Or choose a name from the list by double-clicking it if you want an existing file as new project.
- Or type in full file name after making a new folder using new folder icon in the dialog box.

Once you've typed in or selected the file you want, choose the Open button. (Choose Cancel button if you change your mind).

You can also just press Enter once the file is selected, or you can double-click the file name in the file list. If you create a project, Selection project source window will be opened automatically. In this window, you can add and delete the source files that have been made before by using editor. Therefore you should make your source file(s) before building new project environment, or select source file(s) using the Selection Source menu after making source files for your new project
Here is what the Selection project source window looks like:

**Figure 2.3.3 Selection project source window**




Above window will be explained in the Selection Source menu.

**[ Select Assembler radio button – Figure2.3.1 ]**
The Smart Studio supports two assemblers, SAMA and SASM (SASM57.EXE, SASM86.EXE, SASM88.EXE). If you want to select a assembler, you can click the radio button you want to using the left mouse button.

**[ Target CPU combo box ]**
Select a target controller. Target is a program associated with the target board that is attached to the Smart Kit. You can select a target CPU using scrollbar.

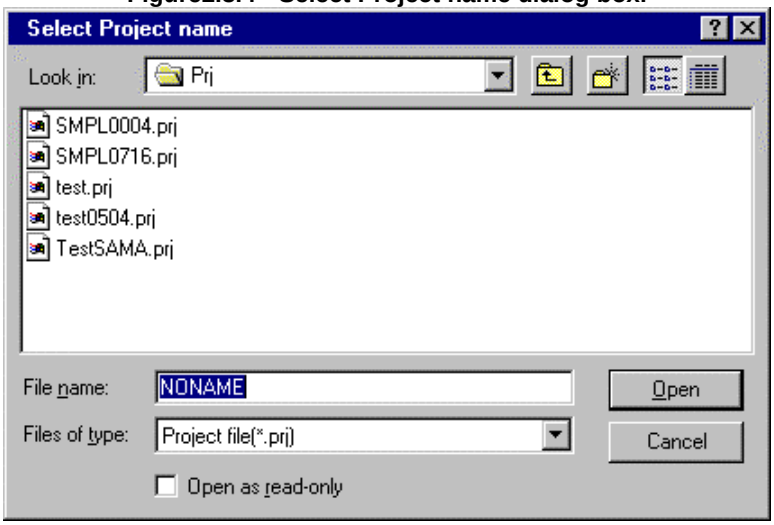
**[ OK button ]**
The OK button opens current project ***without saving***. If you want to save the current project configuration, you had better to click the save icon in the toolbar before opening new project or existing project again.

**[ Cancel button ]**
The Cancel button discards project setup.

## Project|Open

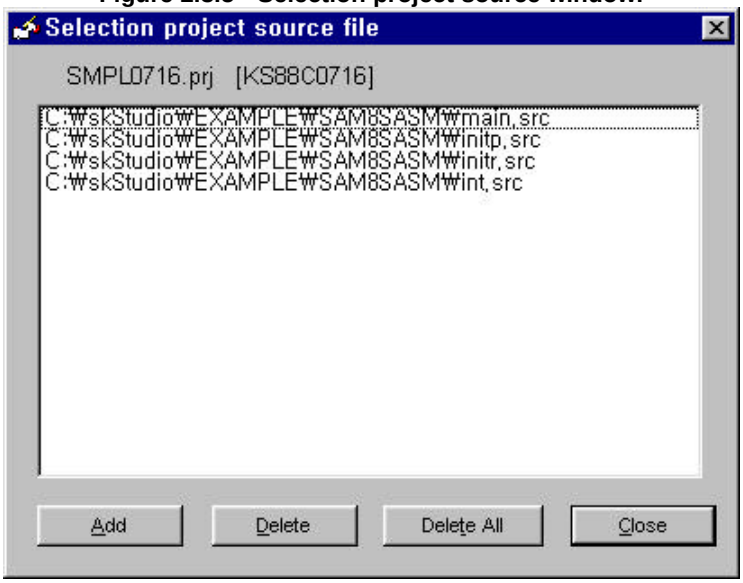
The Project|Open menu displays the Select Project name dialog box. In this dialog box, you can select a project you want to open project for debugging. This menu is supported by shortcut operation using the icon for 'Open existing project' in the toolbar. The Select Project name dialog box in the Smart Studio looks like this:

**Figure2.3.4 Select Project name dialog box.**




In the Select Project name dialog box, we can select or edit project. If the selected project is exists, you can open this project. If you want to select a project, you can type file name directly to the file name input box or double-click any file name in the box to select it. And then, Selection project source window will be opened automatically to confirm, add, delete source files as Figure 2.3.5.

**Figure 2.3.5 Selection project source window.**




Above window will be explained in the Selection Source menu.

## Project|Save

Project|Save menu saves current project information such as project name, assembler, target CPU, screen format, watch window and so on to the disk. This menu is supported by shortcut operation using the icon for ‘Save project’ in the toolbar.

## Project|Save As

Project|Save As… menu brings up the save project as... dialog box in which you can save project as other name.

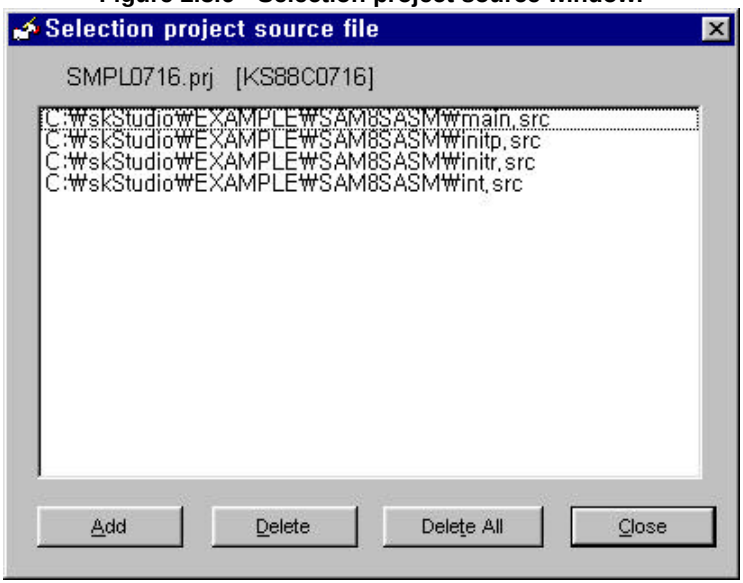
## Project|Close

Project|Close menu saves and closes current open project.

## Project|Selection Source

Project|Selection Source menu adds, or deletes source files in your current project. You can select only one source file when you use SAMA assembler. If you use re-locatable assembler (SASM) in your project, you need to select one or several your source files for your project. This menu should be used after making source file(s) using editor.

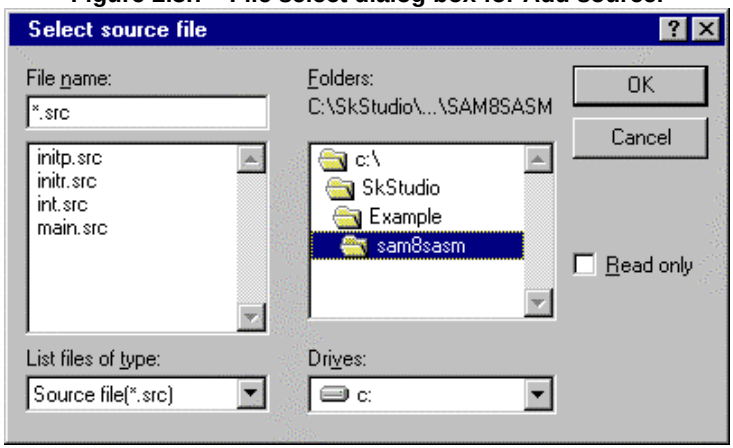
**Figure 2.3.6 Selection project source window.**




**[ Add button ]**
Add button adds source files to the current project using the file-select dialog box.

***Important***
You should first add the source file which includes the CPU reset vector (ORG inst ), and then add the other sources in case that you use re-locatable assembler, SASM, in your project. If not, assembling error will be occurred when you assemble or link operation.

**Figure 2.3.7 File select dialog box for Add source.**




The dialog box contains a File name input box, a File list box, Drives combo box, Folder list box, button labeled OK and Cancel. Now you can do any of these actions like file opening method in Win95/98:

- Choose directories (folder) which contain source files by using a folder icon in the dialog box.
- Choose a name from the list by double-clicking it if you want add a file as project source.
- Or choose all listed source files by using the Shift key + left mouse button.
- Or choose the source files which you want to select by using Ctrl key + left mouse button.

Once selected the file you want, choose the OK button. (Choose Cancel button if you change your mind). You can also just press Enter once the file is selected, or you can double-click the file name in the file list.

**[ Delete button – Figure 2.3.6 ]**
Delete button deletes the selected source file from project window.

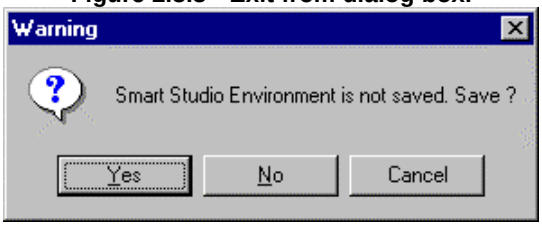
**[ Delete All button ]**
Delete All button deletes all source files in project window.

**[ Close button ]**
Close button closes Selection project source file window.

## Project|Exit

The Project|Exit displays the following dialog box to terminate Smart Studio.
Here is what the dialog box looks like:

**Figure 2.3.8 Exit from dialog box.**




Quit Smart Studio and save any change to the current project, or discard any changes.

**[ Yes button ]**
If you choose yes, save changes made (checked items from Options|Environment menu) to the current project and quit Smart Studio. The Yes button is chosen by defaults.

**[ No button ]**
Choose this button; if you want to discard the changes made and quit Smart Studio.

**[ Cancel button ]**
Choose this button; if you want to discard Project|Exit menu operation.

# Edit

Alt + E

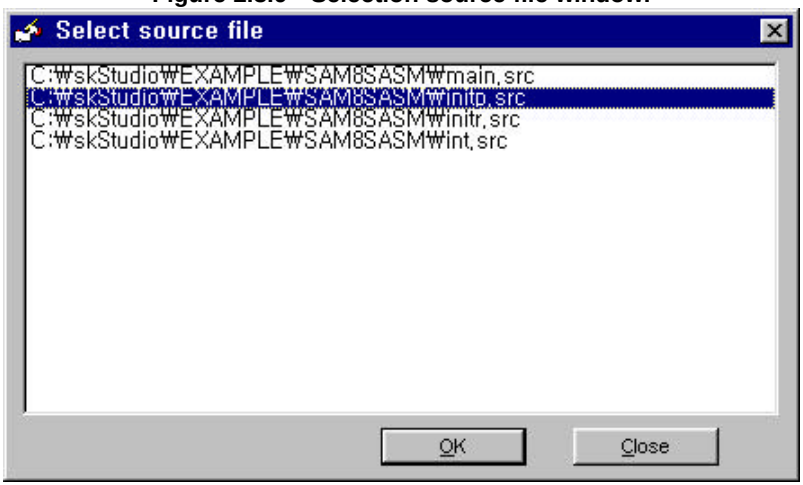
The Edit menu allows editing the source file or other files.

## Edit|Edit Source File

If SAMA assembler is selected in the project, the editor which is selected by you in Option|Editor menu will open source file directly. But if SASM re-locatable assembler is selected in the project, the Selection source file window will be opened automatically.

If 'Selection source file' window is opened, edit the source by selecting a source file in the window. This menu executes editor which is selected in Option|Editor menu. The 'Smart editor' is supported as default editor, but you can install your most desired editor from Options|Editor. See also Options|Editor. This menu is supported by shortcut operation using the icon for ‘Edit source file’ in the toolbar.

**Figure 2.3.9 Selection source file window.**




If you double-click a source file in the Selection source file window, you can edit the source file without OK button click. Also, you can edit a source file by pressing OK button after selecting the desired file by mouse.

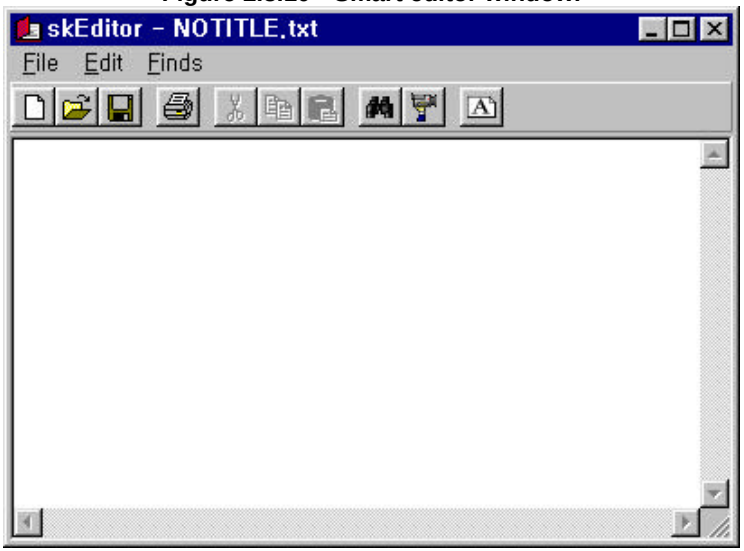
## Edit|Edit Other Files

The Edit|Edit Other Files menu brings up the ‘Select a File to Edit’ dialog box in which you can select a file to edit. The 'Smart editor' is supported as default editor, but you can install your most desired editor from the Options|Editor. See also Options|Editor

## Edit|Smart Editor

The Edit|Smart editor menu brings up the ‘Smart editor’ which is a default editor supported by Smart Studio. Smart editor has simple functions such as open New file, open Existing file, Save file, Exit from Smart editor, Print, Cut, Copy, Paste, Find, Find Next, and Font selection menu. Also, all functions are supported by toolbar icon operation. Smart editor has basic functions for a simple usage, so you’d better to use a professional editor which is familiar for you by using Option|Editor menu.
Here's what that Smart editor window looks like:

**Figure 2.3.10 Smart editor window.**



# View

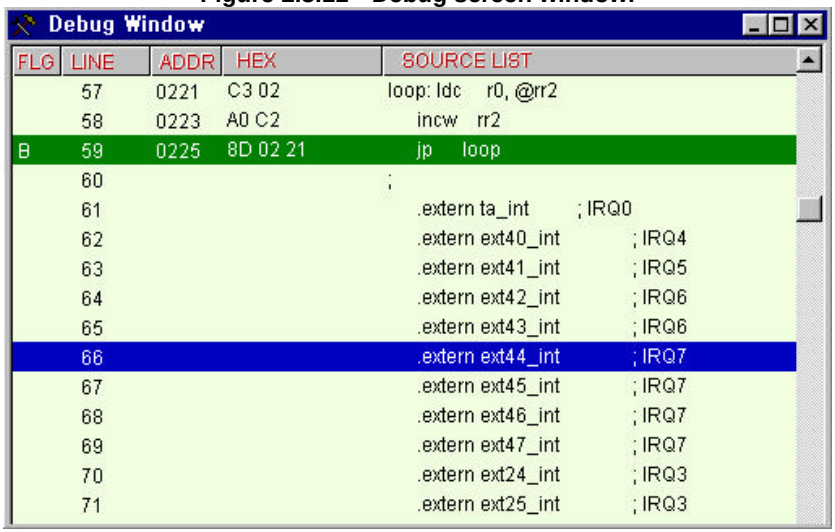
Alt + V

The View menu allows to open Debug window, Working registers window, General registers window, System registers window, Watch window, Project configuration, Program memory, Run history, Listing file and Error file windows.

## View|Debug window

The View|Debug window menu opens the Debug screen window including Working, System, and General register window, which lets you debug your application program after downloading hex file(assembled source file). The Debug screen window allows you to debug application program with single step, breakpoint operations and run/stop operation. All of the processor registers can be displayed or modified in the register view windows. This menu is supported by shortcut operation using the icon for 'Open debug window' in the toolbar.
Here's what that debug screen window looks like:

**Figure 2.3.11 Debug screen window.**



**< Cursor Bar >**
The green bar indicates the running-position and the blue bar indicates the current scrolling-position(cursor bar).

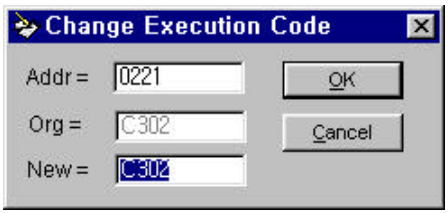
**< FLG column >**
This columns allows toggle on or off a breakpoint flag(B), a trigger point flag(T) and a stop point flag(S) at the address pointed by the cursor bar(Blue-color bar). You can set or clear flag bit(B,T,S) at the cursor bar by using keyboard, Popup menu operation, and left-mouse clicking.

**[ Breakpoint 'B' flag ]**
- Toggle on or off a breakpoint flag(B) at the cursor bar address by pressing 'B' key
- Toggle on or off a breakpoint flag(B) at any address by clicking right mouse button on the FLG column to invoke the popup menu operation.
- Also, a breakpoint flag (B) at the cursor bar can be set or cleared by clicking the left mouse button.

**[ Trigger point 'T' flag ]**
- Toggle on or off a trigger point flag(T) at the cursor bar address by pressing 'T' key.
- Toggle on or off a trigger point flag(T) at any address by clicking right mouse button on the FLG column to invoke the popup menu operation. This flag is exclusive with the stop flag(S).

**[ Stop point 'S' flag ]**
- Toggle on or off a stop point flag(S) at the cursor bar address by pressing 'S' key.
- Toggle on or off a stop point flag(S) at any address by clicking right mouse button on the FLG to invoke the popup menu operation. This flag is exclusive with the trigger flag(T).

**< Modify code data value >**
In order to modify the code data value in the HEX column, double click the code data cell in the window you want to modify. Following change code data dialog box will be opened. This window lets you edit a code data.




The window has two basic modes, called Address and New. Type a new data that you want to modify.

**[ OK button, Enter key ]**
Modify action will be performed. The existing data which is pointed by Address will be changed as the entered value. The changed value is temporary data in the emulation memory. If you download a new hex file to the emulator again, the changed data will be lost.

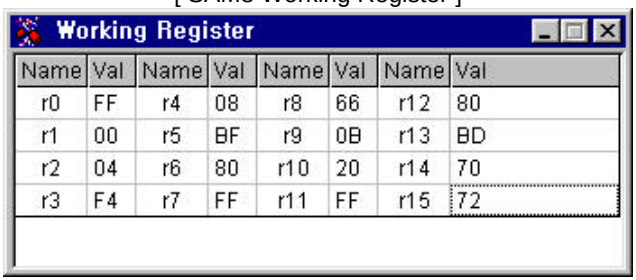
**[ Cancel button, ESC key ]**
This button will discard modify action and close the dialog box window.

## View|Working Registers

The View|Working Registers menu displays the working register view window, which lets you view working register group and modify register value data for debugging your application program. The red-colored data means a changed value after special operation such as single step, breakpoint, go to cursor operation.
Here 's what that working register window looks like:

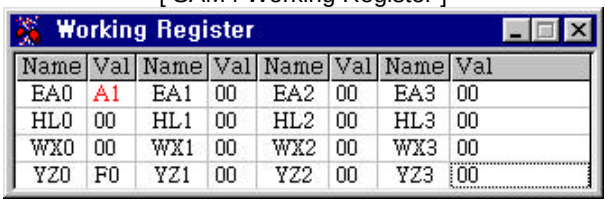
**Figure 2.3.12 Working Register window.**

[ SAM8 Working Register ]



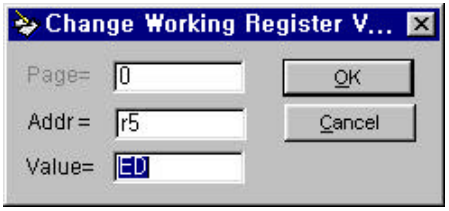

[ SAM8 Flags ]

[ SAM4 Working Register ]




[ SAM4 Flags ]


**< Modify Register value >**
In order to modify the working register value, double click the value data cell in the window you want to modify. Following change working register dialog box will be opened. This dialog box lets you edit register value. Flag bit can not be modified.




The dialog box has two basic modes, called Address and Value. Type your desired value data which is pointed by Address.

**[ OK button, Enter key ]**
Modify action will be performed. The existing data that is pointed by address will be changed as the entered value.

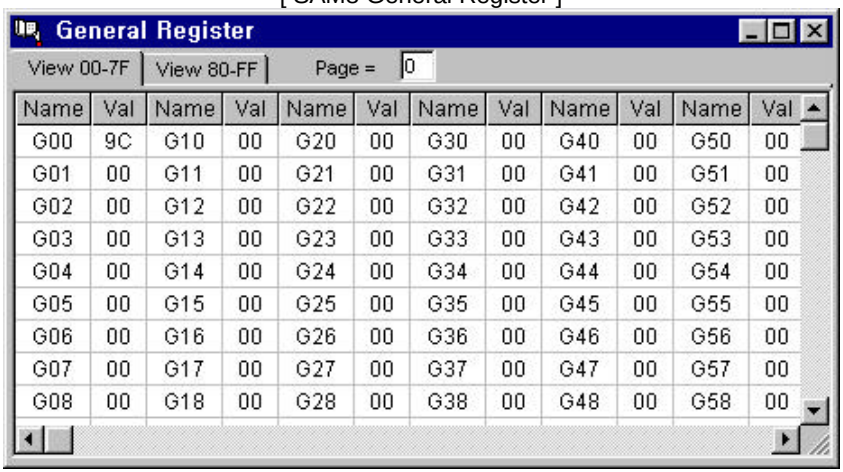
**[ Cancel button , ESC key ]**
This button will discard modify action and close the dialog box window.

## View|General Registers

The View|General Registers menu displays the General register view window, which lets you view general register group and modify register value data for debugging your application program. Here 's what that general register window looks like:

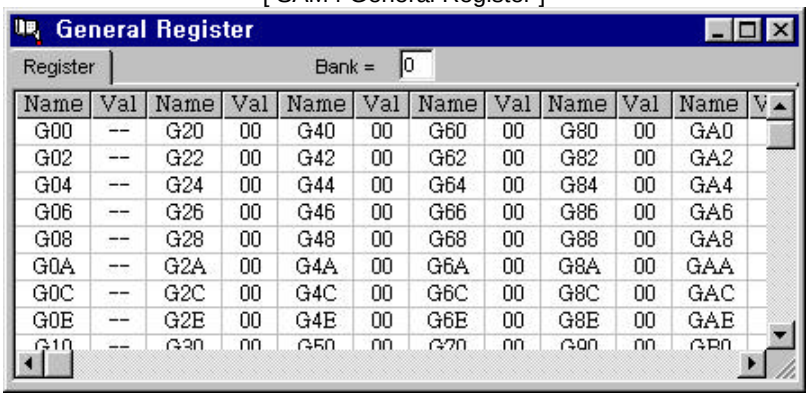
**Figure 2.3.13 General Register window.**

[ SAM8 General Register ]




| Name | Val | Name | Val | Name | Val | Name | Val | Name | Val | Name | Val |
|---|---|---|---|---|---|---|---|---|---|---|---|
| G00 | 9C | G10 | 00 | G20 | 00 | G30 | 00 | G40 | 00 | G50 | 00 |
| G01 | 00 | G11 | 00 | G21 | 00 | G31 | 00 | G41 | 00 | G51 | 00 |
| G02 | 00 | G12 | 00 | G22 | 00 | G32 | 00 | G42 | 00 | G52 | 00 |
| G03 | 00 | G13 | 00 | G23 | 00 | G33 | 00 | G43 | 00 | G53 | 00 |
| G04 | 00 | G14 | 00 | G24 | 00 | G34 | 00 | G44 | 00 | G54 | 00 |
| G05 | 00 | G15 | 00 | G25 | 00 | G35 | 00 | G45 | 00 | G55 | 00 |
| G06 | 00 | G16 | 00 | G26 | 00 | G36 | 00 | G46 | 00 | G56 | 00 |
| G07 | 00 | G17 | 00 | G27 | 00 | G37 | 00 | G47 | 00 | G57 | 00 |
| G08 | 00 | G18 | 00 | G28 | 00 | G38 | 00 | G48 | 00 | G58 | 00 |

[ SAM4 General Register ]




| Name | Val | Name | Val | Name | Val | Name | Val | Name | Val | Name | V |
|---|---|---|---|---|---|---|---|---|---|---|---|
| G00 | -- | G20 | 00 | G40 | 00 | G60 | 00 | G80 | 00 | GA0 | |
| G02 | -- | G22 | 00 | G42 | 00 | G62 | 00 | G82 | 00 | GA2 | |
| G04 | -- | G24 | 00 | G44 | 00 | G64 | 00 | G84 | 00 | GA4 | |
| G06 | -- | G26 | 00 | G46 | 00 | G66 | 00 | G86 | 00 | GA6 | |
| G08 | -- | G28 | 00 | G48 | 00 | G68 | 00 | G88 | 00 | GA8 | |
| G0A | -- | G2A | 00 | G4A | 00 | G6A | 00 | G8A | 00 | GAA | |
| G0C | -- | G2C | 00 | G4C | 00 | G6C | 00 | G8C | 00 | GAC | |
| G0E | -- | G2E | 00 | G4E | 00 | G6E | 00 | G8E | 00 | GAE | |

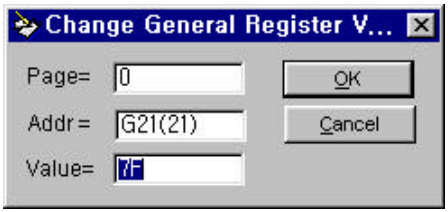
The General Register View window you see is one of the several general register (SAM4: bank0 bank14, SAM8: Page0-Page15) groups available for the current target processor. Micro-controller (depending on the device) has hundreds or perhaps thousands of general registers (Data Memory). Given the large number of registers, it is not possible to display all of the processor registers on a

single window. Bank or Page input box allows you to select general registers group from all of those available to be displayed as the current instruction while debugging.

**[ Page or Bank text input box ]**
Allows direct entry of a new page value of SAM8 or a new bank value of SAM4. A new value can be effective after pressing the Enter key.

**[ LCD RAM check box ]**
This check box will be displayed only when the target CPU with LCD drive is selected. If this box is checked (selected), the display format of the general register will be changed into 4-bitwise or 5-bitwise format to show the RAM data of LCD display for easy understanding.

**< Modify Register value >**
In order to modify the general register value, double click the value data cell in the window you want to modify. Following change general register dialog box will be opened. This window lets you edit register value.




The window has three basic modes, called Page(SAM8) or Bank(SAM4), Address and Value. Type your desired value data that is pointed by Page and Address.

**[ OK button, Enter key ]**
Modify action will be performed. The existing data which is pointed by Page(or Bank) and Address will be changed as the entered value.

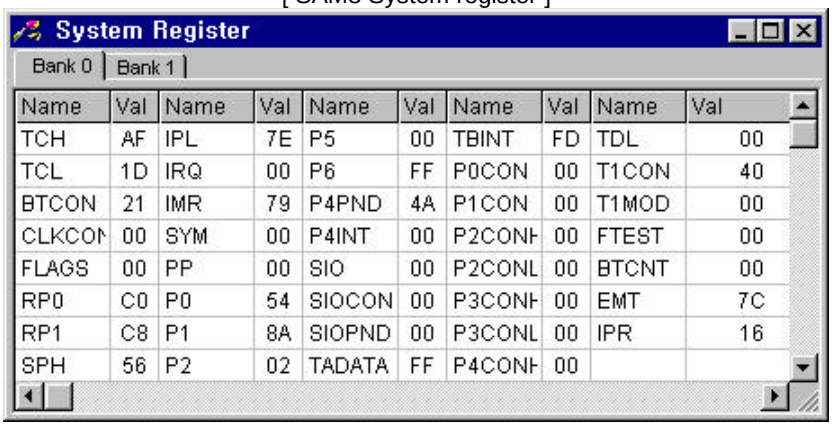
**[ Cancel button, ESC key ]**
This button will discard modify action and close the dialog box window.

## View|System Registers

The View|System Registers menu displays the System register View window, which lets you view system register group and modify register value data for debugging your application program. Here 's what that system register window looks like:

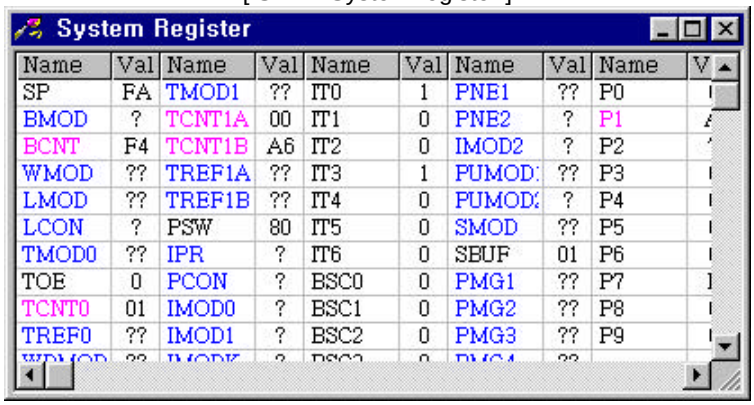
**Figure 2.3.14 System Register view window.**

[ SAM8 System register ]




| Name | Val | Name | Val | Name | Val | Name | Val | Name | Val |
|---|---|---|---|---|---|---|---|---|---|
| TCH | AF | IPL | 7E | P5 | 00 | TBINT | FD | TDL | 00 |
| TCL | 1D | IRQ | 00 | P6 | FF | P0CON | 00 | T1CON | 40 |
| BTCON | 21 | IMR | 79 | P4PND | 4A | P1CON | 00 | T1MOD | 00 |
| CLKCON | 00 | SYM | 00 | P4INT | 00 | P2CONH | 00 | FTEST | 00 |
| FLAGS | 00 | PP | 00 | SIO | 00 | P2CONL | 00 | BTCNT | 00 |
| RP0 | C0 | P0 | 54 | SIOCON | 00 | P3CONH | 00 | EMT | 7C |
| RP1 | C8 | P1 | 8A | SIOPND | 00 | P3CONL | 00 | IPR | 16 |
| SPH | 56 | P2 | 02 | TADATA | FF | P4CONH | 00 | | |

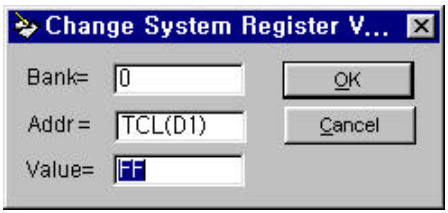
[ SAM4 System register ]




| Name | Val | Name | Val | Name | Val | Name | Val | Name | V |
|---|---|---|---|---|---|---|---|---|---|
| SP | FA | TMOD1 | ?? | IT0 | 1 | PNE1 | ?? | P0 | |
| BMOD | ? | TCNT1A | 00 | IT1 | 0 | PNE2 | ? | P1 | |
| BCNT | F4 | TCNT1B | A6 | IT2 | 0 | IMOD2 | ? | P2 | |
| WMOD | ?? | TREF1A | ?? | IT3 | 1 | PUMOD1 | ?? | P3 | |
| LMOD | ?? | TREF1B | ?? | IT4 | 0 | PUMOD2 | ? | P4 | |
| LCON | ? | PSW | 80 | IT5 | 0 | SMOD | ?? | P5 | |
| TMOD0 | ?? | IPR | ? | IT6 | 0 | SBUF | 01 | P6 | |
| TOE | 0 | PCON | ? | BSC0 | 0 | PMG1 | ?? | P7 | |
| TCNT0 | 01 | IMOD0 | ? | BSC1 | 0 | PMG2 | ?? | P8 | |
| TREF0 | ?? | IMOD1 | ? | BSC2 | 0 | PMG3 | ?? | P9 | |

The System Register View window you see is special register group such as peripheral hardware block (SAM4: bank15, SAM8: SET0,1) of the current target processor. Red-color registers mean the read only type, blue-color registers mean the write only type and the others (black color) are the read/write types.

**< Modify Register value >**
In order to modify the system register value, double click the value data cell in the window you want to modify. Following change system register dialog box will be opened. This window lets you edit register value.




The window has three basic modes, called Bank(SAM8, SAM4), Address and Value. Type your desired value data that is pointed by Bank and Address.

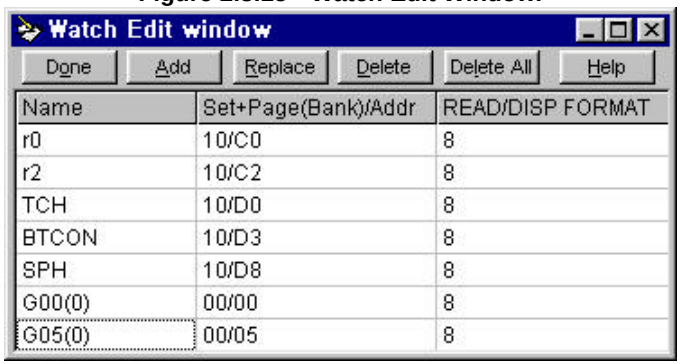
**[ OK button, Enter key ]**
Modify action will be performed. The existing data that is pointed by Bank and Address will be changed as the entered value.

**[ Cancel button, ESC key ]**
This button will discard modify action and close the dialog box window.

## View|Watch Window

The View|Watch Window menu displays the Watch edit window, which lets you select registers to inspect during debugging your application program. In the Watch edit window, you can edit the watch register items using Add, Replace, Delete, and Delete All button. After editing watch window, please click the 'Done' button to display the watch view window.
Here's what that window looks like:

**Figure 2.3.15 Watch Edit Window.**




**[ Add button ]**
To add the register item you want to watch in the Watch edit window, click the desired register item (name) on the System, General, or Working register window. And then click the 'Add' button in the Watch edit window.

**[ Replace button ]**
To replace a register item in the Watch window, click the desired register item (name) on the System, General, or Working register window. After selecting an existing item and click the 'Replace' button in the Watch edit window to replace it.

**[ Delete button ]**
To delete the register item selected from the Watch edit window, click the 'Delete' button.

**[ Delete All button ]**
To delete all register items from the Watch edit window, click the 'Delete All' button.

**[ Done button ]**
If you choose Done, the Watch edit window disappears and the Watch view mode will be displayed.

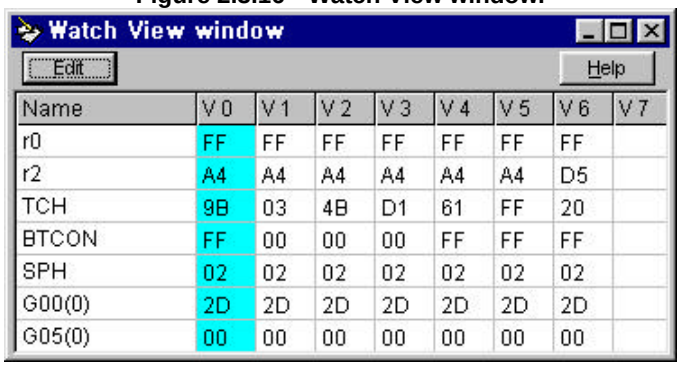
**[ Help button ]**
This button will open help message for the Watch window like below Example.

**< Example >**
The name or display format of an item in the Watch edit window can be changed before clicking the 'Done' button. For example, to change name Bit0 of P2(Port2) 'POWER' and display its value:

-. Click P2(Port2) item on the System register window.
-. Click 'Add' button in the Watch edit window to add it to the Watch edit window.
-. Double click the P2 item in the watch edit window.
-. Type a new name 'POWER' in the Change variable name window.
-. Double click the responding item in the 'READ/DISP FORMAT'.
-. Add new format '/BIT0' at the end of the old format to modify the display format.
-. Then the P2 item is changed to 'POWER F/(80 to FF) 4/BIT0'.
-. Click the 'Done' button.

Watch View window displays the value of registers in the Watch window in accordance with debugging operation. The values with blue color (V0) are the current data of the registers in the watch view window.

**Figure 2.3.16 Watch View window.**

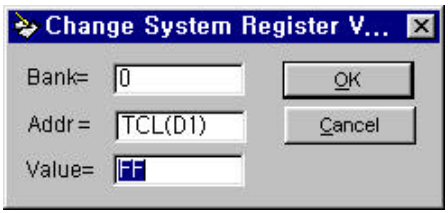



| Name | V0 | V1 | V2 | V3 | V4 | V5 | V6 | V7 |
|---|---|---|---|---|---|---|---|---|
| r0 | FF | FF | FF | FF | FF | FF | FF | |
| r2 | A4 | A4 | A4 | A4 | A4 | A4 | D5 | |
| TCH | 9B | 03 | 4B | D1 | 61 | FF | 20 | |
| BTCON | FF | 00 | 00 | 00 | FF | FF | FF | |
| SPH | 02 | 02 | 02 | 02 | 02 | 02 | 02 | |
| G00(0) | 2D | 2D | 2D | 2D | 2D | 2D | 2D | |
| G05(0) | 00 | 00 | 00 | 00 | 00 | 00 | 00 | |

**[ Edit button ]**
If you click the Edit button, the Watch View window disappears and the Watch edit window will be opened again. You can edit the Watch window again.

**[ Help button ]**
This button will open help message for the Watch window like Example.

**< Modify Register value >**
In order to modify the register value, double click the value data cell in the watch window you want to modify. Following change register dialog box will be opened. This window lets you edit a register value like the change method of any other register windows.




The window has three basic modes, called Bank(SAM8) or Bank(SAM4), Address and Value. Type your desired value data that is pointed by Bank and Address.

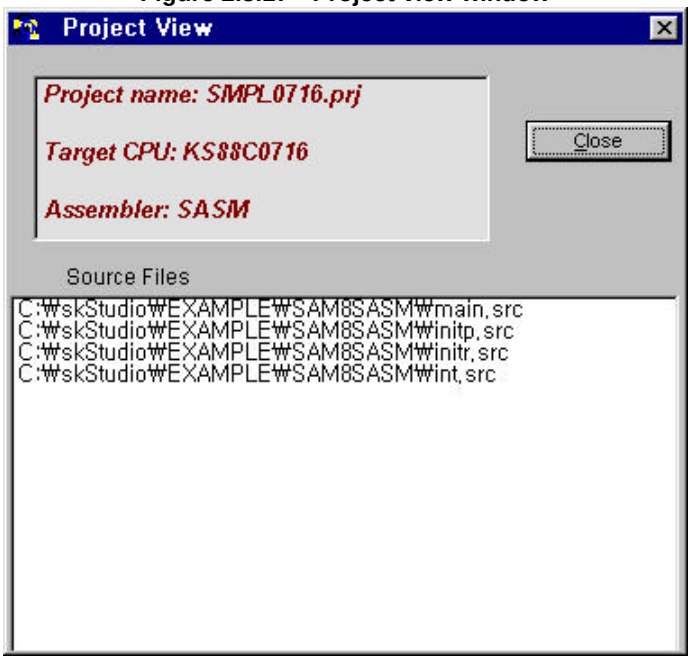
**[ OK button, Enter key ]**
Modify action will be performed. The existing data that is pointed by Bank and Address will be changed as the entered value.

**[ Cancel button, ESC key ]**
This button will discard modify action and close the dialog box window.

## View|Project configuration

View|Project configuration displays project window with project name, target CPU, assembler and source files information. This menu is useful to see the project environment.
Here's what that window looks like:

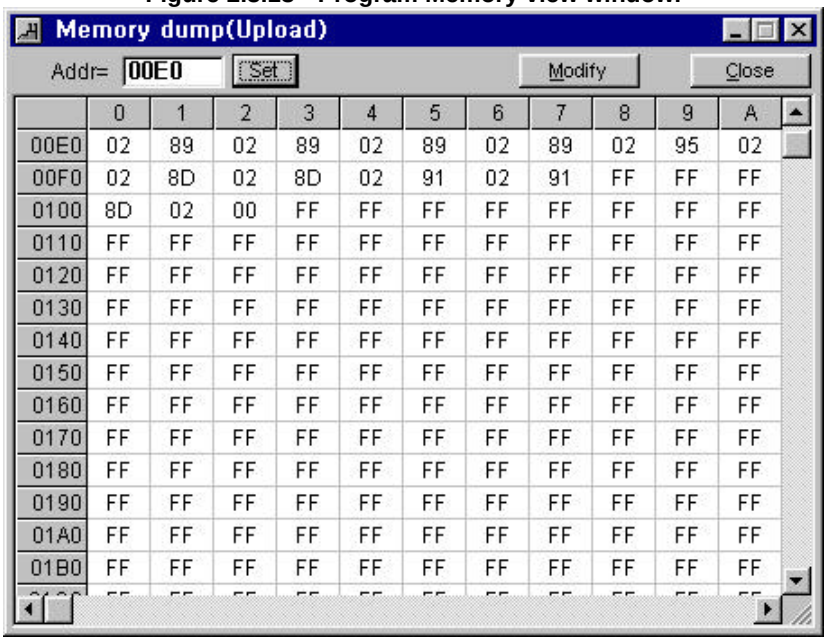
**Figure 2.3.17 Project view window**



## View|Program(Emulation) memory

The View|Program memory menu displays the dump results window of the emulation memory which patches emulation memory and allows patch editing (hot patch) of the target emulation memory. Code that has already been loaded into the emulation memory can be patch edited with this function. It is useful while debugging in order to avoid having to reassemble and load when testing a small change. The edit function in the debugger provides a safer method of editing memory, by limiting hex editing to the bytes of a single instruction at one time.
Here's what Program Memory view window looks like:

**Figure 2.3.18 Program Memory view window.**




| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 00E0 | 02 | 89 | 02 | 89 | 02 | 89 | 02 | 89 | 02 | 95 | 02 |
| 00F0 | 02 | 8D | 02 | 8D | 02 | 91 | 02 | 91 | FF | FF | FF |
| 0100 | 8D | 02 | 00 | FF | FF | FF | FF | FF | FF | FF | FF |
| 0110 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0120 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0130 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0140 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0150 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0160 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0170 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0180 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 0190 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 01A0 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |
| 01B0 | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF | FF |

**[ Addr text box ]**
Allows direct entry of a new window starting address.

**[ Set button, or Enter key ]**
Allows refresh new window contents which starts from address in the ‘Addr’ text box.

**[ Modify button ]**
This button lets you edit the code data in the emulation memory. Modify dialog box will be opened. Also, you can modify the code data directly by double-clicking it in the window.

**[ Close button ]**
This button will close the program memory view window.

## View|Run History

The View|Run History allows an examination of the captured history of program, whenever single step, breakpoint or unconditional run is executed. This menu brings up the run history window. Here's what Run History view window looks like:

**Figure 2.3.19 Run History view window.**


**< Column Title >**
Offset “0000” is the last step which was executed. Address and List means program counter values and source list.

**[ Refresh button ]**
Refresh Run history window (re-reading trace results).

**[ Find button ]**
This button displays the Find a Text dialog box, which lets you type in the text you want to find for and set options that customize the find.

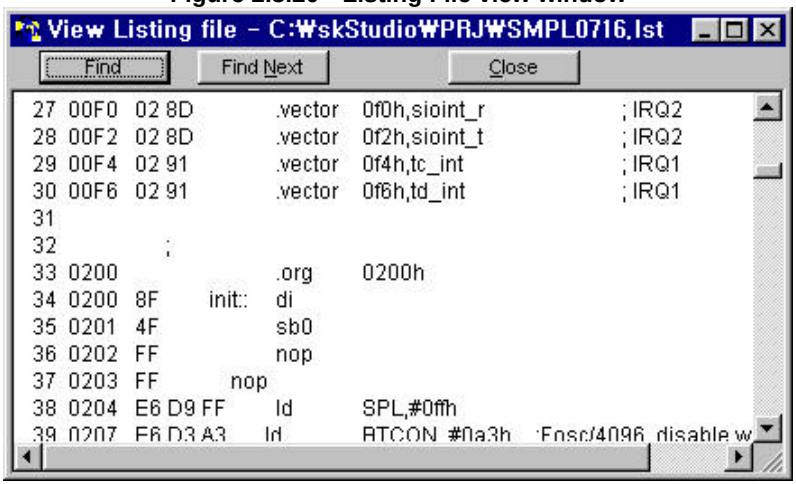
**[ Find Next button ]**
This button repeats the last Find menu. All settings you made in the last Find a Text dialog box used remain in effect when you choose Find Next.

**[ Close button ]**
This button will close the Run History view window.

## View|Listing File

The View|Listing File menu lets you show contents of the source.lst file for the current project. The View|Listing File menu executes viewer which is supported by default viewer; skViewer.
Here's what that window looks like:

**Figure 2.3.20 Listing File view window**




**[ Find button ]**
This button displays the Find a Text dialog box, which lets you type in the text you want to find for and set options that customize the find.

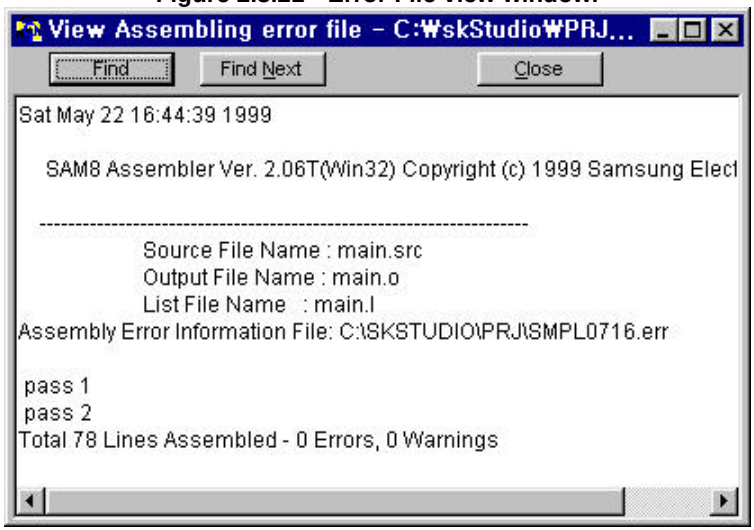
**[ Find Next button ]**
This button repeats the last Find menu. All settings you made in the last Find a Text dialog box used remain in effect when you choose Find Next.

**[ Close button ]**
This button will close the Listing file view window.

## View|Error File

The View|Error File menu lets you show contents of the project.err file generated only by SASM assembler if an error was detected during assembling. This menu is helpful to edit the source file when an error was detected during assembling.
Here's what that window looks like:

**Figure 2.3.21 Error File view window.**




**[ Find button ]**
This button displays the Find a Text dialog box, which lets you type in the text you want to find for and set options that customize the find.

**[ Find Next button ]**
This button repeats the last Find menu. All settings you made in the last Find a Text dialog box used remain in effect when you choose Find Next.

**[ Close button ]**
This button will close the Error file view window.

# Tool

Alt + T

Use the tool menu to assemble the source file, operate the OTP programmer, ROM code generation, convert SAMA source to SASM format, and open the Calculator window in the current project.

## Tool|Assemble

Use the Tool|Assemble menu to assemble the sources file and link the object files in the current project. You should include xxx.def file for SAMA or xxx.reg file for SASM assembler at the head line of the your source file because these file define the target CPU information such as ROM size, register name with attribute. Please refer to the Assembler Control Operations in the Part III.

## Tool|Assemble|Making

Use this menu to assemble and link all the source files in the project window with the predefined assembler (SAMA.EXE, SASM.EXE). The Making is assembling checks time record between xxx.o(object file) file and xxx.src(source file) file. But SAMA.EXE is not re-locatable assembler, so dose not link and check time record between xxx.o file and xxx.src file.

The assembler is predefined in the Project|New or Project|Open a Project, the sources are predefined in the Project|Selection project source menu. This menu is supported by shortcut operation using the icon for ‘Assemble source file’ in the toolbar.

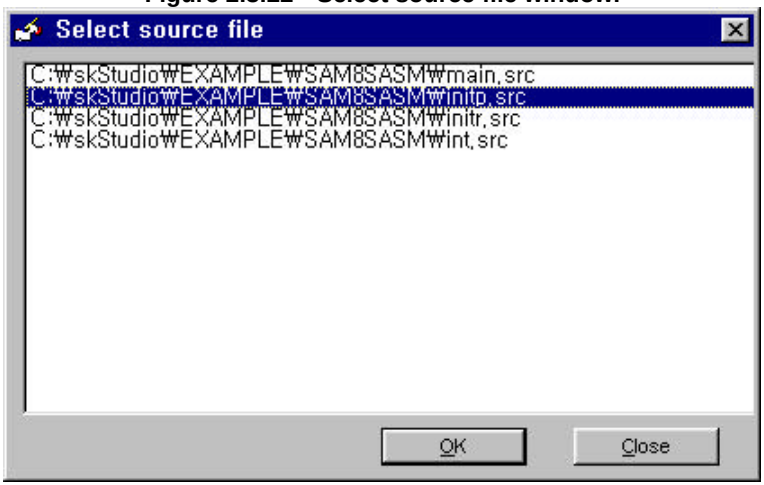
## Tool|Assemble|Rebuild All

This menu is for SASM (Re-locatable assembler). Use this menu to assemble and link all the source files in the project window with the predefined assembler (SASM57.EXE, SASM86.EXE, ASM88.EXE) The Rebuild All dose not check time record between xxx.o file and xxx.src file. The assembler is predefined in the Project|New or Project|Open a Project, the sources are redefined in the Project|Selection source menu.

## Tool|Assemble|Assembling

This menu is for SASM. Use this menu to assemble the selected source file in the project window with the predefined assembler (SASM57.EXE, SASM86.EXE, SASM88.EXE). The assembler is predefined in the Project|New or Project|Open a Project, the sources are predefined in the Project|Selection source menu. 'Select source file' window will be shown if you choose this menu.

**Figure 2.3.22 Select source file window.**




## Tool|Assemble|Linking

This menu is for SASM re-locatable assembler. Use this menu to link all the object files. This menu executes Linker such as SLINK57.EXE, SLINK86.EXE, SLINK88.EXE in accordance with the predefined target CPU in the project menu.

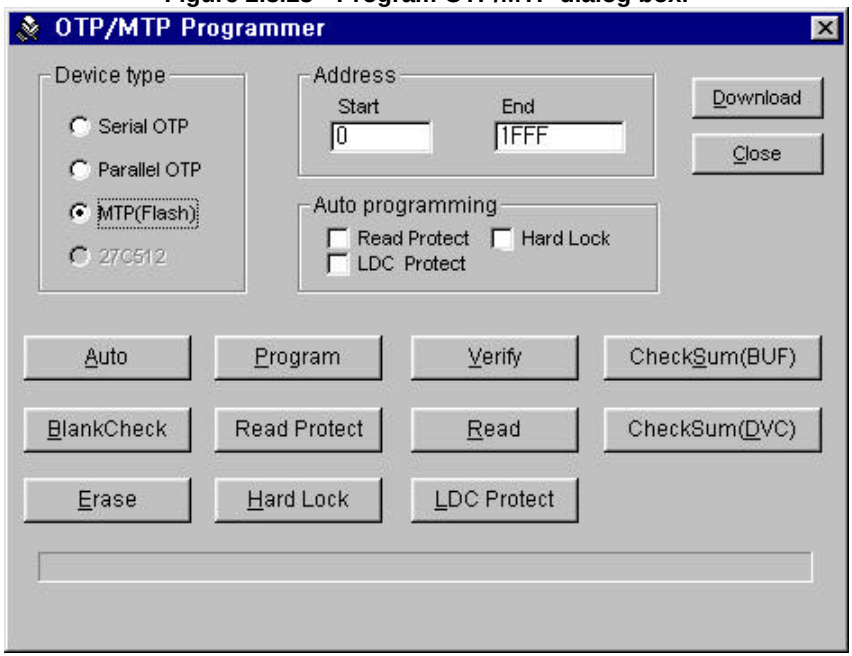
## Tool|Assemble|Assembling All

This menu is for SASM re-locatable assembler. Use this menu to assemble the all source files with the predefined assembler. (SASM57.EXE, SASM86.EXE, SASM88.EXE). The assembler is predefined by the target CPU in the Project|New or Project|Open a Project, the source are predefined in the Project|Selection source menu.

## Tool|OTP Programmer

This menu supports you can program OTP/MTP(Flash type ROM) device. If this menus is selected by the mouse, Program OTP/MTP dialog box will be displayed. In this dialog box you can select the device type, Auto programming option, function, and address range you want to program.
Here's what that dialog box looks like:

**Figure 2.3.23 Program OTP/MTP dialog box.**




**[ Device type radio button ]**
Select the OTP type, serial or parallel (KS88P0916/P0416/P0424/P01424), MTP.

**[ Start Address input box ]**
Set the starting address for OTP/MTP operations. Default value is 0000[Hexadecimal]

**[ End Address input box ]**
Set the end address in accordance with ROM size of all OTP/MTP operations.

**[ Auto Read Protect check box ]**
Select to enable or disable read protection after programming automatically. If this box is checked, the read protection operation is performed automatically after programming.

**[ Auto Hard Lock check box ]**
This option is available only when MTP device is selected. Select to enable or disable Hard Lock after programming automatically. If this box is checked, the Hard Lock operation is performed after programming automatically. For more information about Hard Lock function, please refer to User's Manual of Samsung MTP devices.

**[ Auto LDC Protect check box ]**
This option is available only when MTP device is selected. Select to enable or disable LDC Protect after programming automatically. If this box is checked, the LDC Protect operation is performed after programming automatically. For more information about LDC Protect function, please refer to User's Manual of Samsung MTP devices that have a function of external program memory access.

**< Program Function Command Button >**

**[ Auto button ]**
This command makes following function to be operated sequentially Blank check, Program and Verify using the start and end addresses specified by using Start, End address input boxes.

**[ Program button ]**
Programs the OTP/MTP in the socket from the emulation memory contents, using the start and end address specified by using Start, End address input boxes. The start address in emulation memory is always the same as the start designation address in the OTP/MTP.

**[ Verify button ]**
Compare contents between OTP/MTP and Emulation memory, using the start and end address as specified by using Start, End Address input boxes.

**[ CheckSum BUF button ]**
Calculate and display the checksum of the emulation memory, using the start and end address as specified by using Start, End address input boxes.

**[ Blank Check button ]**
Checks contents of OTP/MTP is the blank data (FF) or not, in the range of the start and end address which is specified by Start, End address input boxes.

**[ Read Protect button ]**
Enable OTP/MTP read protection.

**[ Read button ]**
Copy the contents of the OTP/MTP in the socket into emulation memory, in the range of the start and end addresses specified by Start, End address input boxes.

**[ CheckSum DVC button ]**
Calculate and display the checksum of the OTP/MTP in the socket, in the range of the start and end address specified by Start, End Address input boxes.

**[ Erase button ]**
This button is available only when MTP device is selected. Erase contents of MTP device in the socket. Erased data will be ‘FF’ hexadecimal.

**[ Hard Lock button ]**
This button is available only when MTP device is selected. Enable MTP device into the Hard Lock.

**[ LDC Protect button ]**
This button is available only when MTP device is selected. Enable MTP device into the LDC Protection.

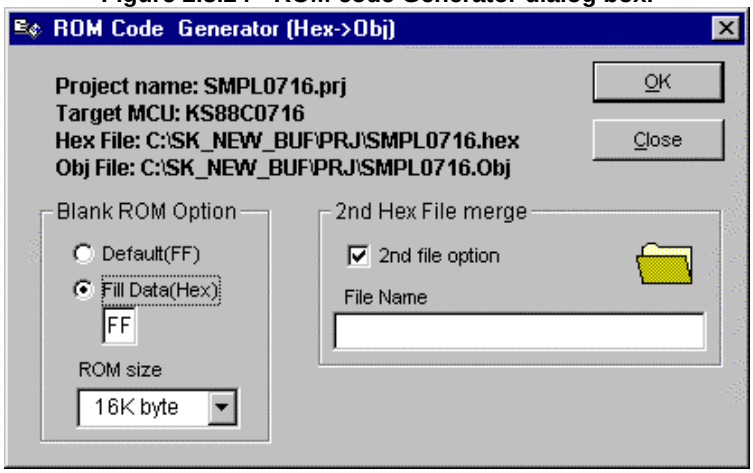
**[ Download button ]**
The download command button invokes a file opening dialog box to select a file (.hex or .obj) which is downloaded into the emulation memory. End address will be changed automatically as last address of the downloaded file.

**[ Close button ]**
This button will close the OTP/MTP programmer window.

## Tool|ROM Code Generator

This menu supports you can make xxxx.obj file from xxxx.hex file. When you release ROM code for masking to Samsung after debugging is finished, you should provide a xxxx.obj file and checksum of it using this menu. This menu selects a target MCU in accordance with predefined project automatically. Here's what that dialog box looks like:

**Figure 2.3.24 ROM code Generator dialog box.**




**[ Blank ROM option radio buttons ]**

**< Default(FF) radio button >**

If this option is selected, default data (FF) will be filled out in the blank ROM area. This 'FF' default data are recommended.

**< Fill Data radio button >**

If this option is selected, ROM code generator will fill blanks into xxx.obj(object) file with the hexadecimal data specified in the input text box until the last address of ROM size.

**[ ROM size combo box ]**

The ROM size means last address of xxx.obj file. If you want to change your desired address which is different from the listed ROM sizes, you can type any value of end address directly.

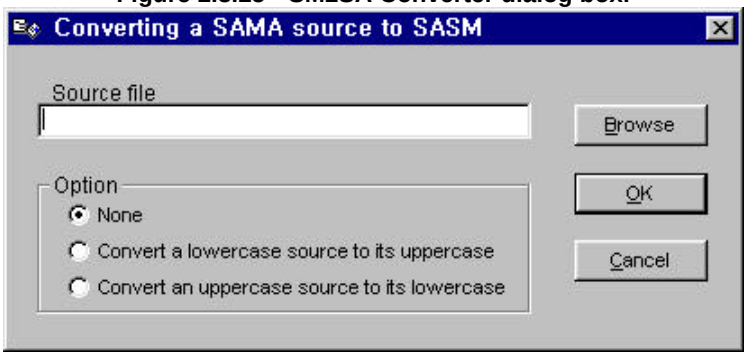
**[ 2nd Hex file merge check box ]**
If you want to merge another hexadecimal file such as a font data ROM to the program code file, this option can be used to make one obj(object) file. This option is useful to program OTP/MTP with the program code and the font data together using OTP/MTP programmer.

**[ File opening icon ]**
You can select the 2nd file to be merged using the file open icon.

## Tool|SM2SA Converter

This menu provides the converter that changes SAMA source file(s) into SASM source file(s). When you choose this menu, the Converter ( SA2AS.EXE ) is invoked under the Smart Studio.

**Figure 2.3.25 SM2SA Converter dialog box.**




**[ Source file text box, Browse button ]**
You can select the SAMA source file using this text box or browse button.

**[ Option radio buttons ]**

**< None >**
If you select this option, the SAMA source file will be converted as SASM without the character conversion.

**< Convert a lowercase source to it's uppercase >**
If you select this option, the SAMA source file will be converted as SASM with uppercase characters.

**< Convert an uppercase source to it's lowercase >**
If you select this option, the SAMA source file will be converted as SASM with lowercase characters.

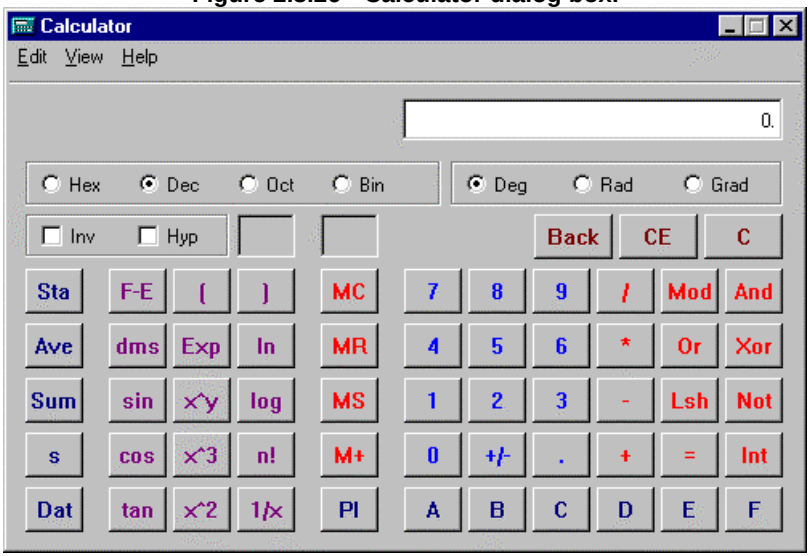
**[ OK button ]**
Performs conversion operation.

**[ Cancel button ]**
This button will discard conversion action and close dialog box window.

## Tool|Calculator

When you choose Calculator, Calculator provided from win95/98 is opened on the desktop. The calculator is a simple four-function and scientific calculator. To operate the calculator you can either use the key unit or press the button on the calculator with the mouse.

**Figure 2.3.26 Calculator dialog box.**



# Debug

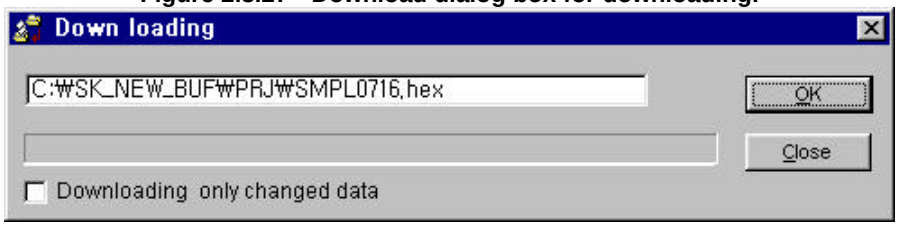
Alt + D

The Debug menu provides sub menus for debugging your program including download, run/stop, single-step run, breakpoint, trace run, timer run and etc.

## Debug|Downloading

The Debug|Downloading menu displays the Download dialog box which lets you choose the hex file to be download to emulation memory and operation option. This menu is supported by shortcut operation using the icon for 'Download project.hex file' in the toolbar and Ctrl+L key.
Here is what the dialog box looks like:

**Figure 2.3.27 Download dialog box for downloading.**



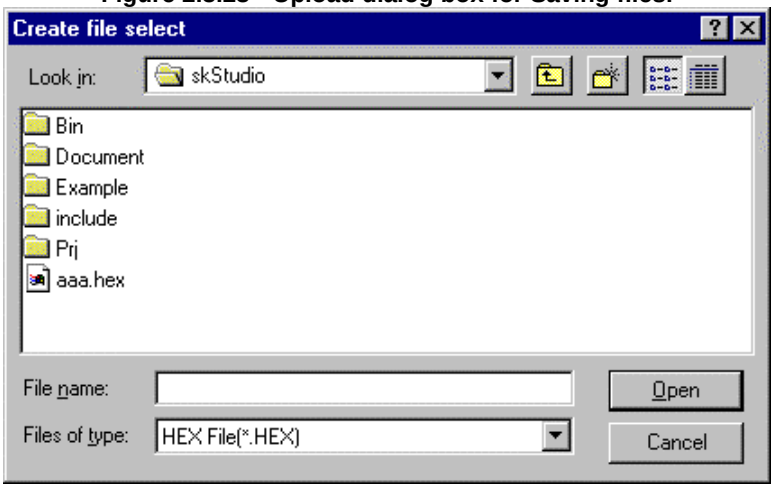

**[ File name text input box ]**
Default hex file to be downloaded is automatically selected by predefined project environment. If you want to change hex file to be downloaded, you can type your desired file name with directory in the file name text input box.

**[ Download only changed data check box ]**
In order to save downloading time, you can select this option. If this option is selected, only the changed data from the previous downloaded file will be downloaded. After downloading a hex file, the debug screen including Working, General and the System register view window will be opened automatically.

## Debug|Uploading

The Debug|Uploading menu displays the Upload dialog box which lets you choose the hex file to be saved with INTEL-HEX format from the emulation memory, and then you can type the start and end address of the emulation memory to be saved.
Here is what the dialog box looks like:

**Figure 2.3.28 Upload dialog box for Saving files.**




The dialog box contains an input box for file name, a file list, button labeled Save, Cancel. Now you can do any of these actions like file opening method in Win95/98:

- View the contents of different directories (folder) by selecting a directory icon in the dialog box.
- Type in a full file name and choose Save. Save button decides the file into the save file.
- Or choose a name from the list by double-clicking it if you want an existing file as save file.
- Or type in full file name after making a new folder using new folder icon in the dialog box.

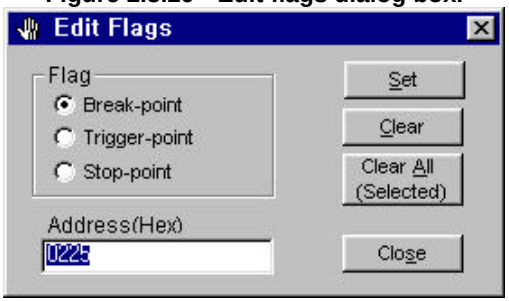
Once you've typed in or selected the file you want, choose the Save button. (Choose Cancel button if you change your mind). You can also just press Enter once the file is selected, or you can double-click the file name in the file list.
If you select the save file, the Address input dialog box will be opened automatically. In this dialog box, you can specify the start and end address of the file into which emulation memory is to be saved.

The hex files which is created by upload feature can not be used for debugger because it was only INTEL-HEX format file without any control data used in debugging. So the hex file which is created by assembler can be used for debugger. (It is impossible to run the hex files that are created by upload in the debugger).

## Debug|Edit Flags

The Debug|Edit flag menu open the Edit Flags dialog box that lets you set or clear flags in the debug screen window. This menu is supported by shortcut operation using the icon for 'Edit flags' in the toolbar. See also Debug|ROM break run, Debug|Timer run, and Debug|Trace run.
Here is what the dialog box looks like:

**Figure 2.3.29 Edit flags dialog box.**




The Edit Flags... dialog box allows the setting of various flags into debug screen window for the target controller's emulation memory.

**[ Flag Radio Buttons ]**
Choose among the Flag radio buttons to determine which flag is to be set or clear. The Break-point radio button is chosen by default.

**< Break-point >**
Choose the break-point radio button if you want to set or clear breakpoint at the current address.

**< Trigger-point >**
Choose the trigger-point radio button if you want to set or clear trace/timer trigger point at the current address. This flag is exclusive with the stop point.

**< Stop-point >**
Choose the stop-point radio button if you want to set or clear the timer-stop point at the current address. This flag is exclusive with the trigger point.

**[ Address text input box ]**
Type the specific address where you want to set or clear flag.

**[ Set button ]**
Press Set button if you want to set the current chosen flag at the address.

**[ Clear button ]**
Press Clear button if you want to clear the current chosen flag from the address.

**[ Clear All(selected) button ]**
Clear all of the selected flags specified by breakpoint, trigger-point, or stop-point radio button.

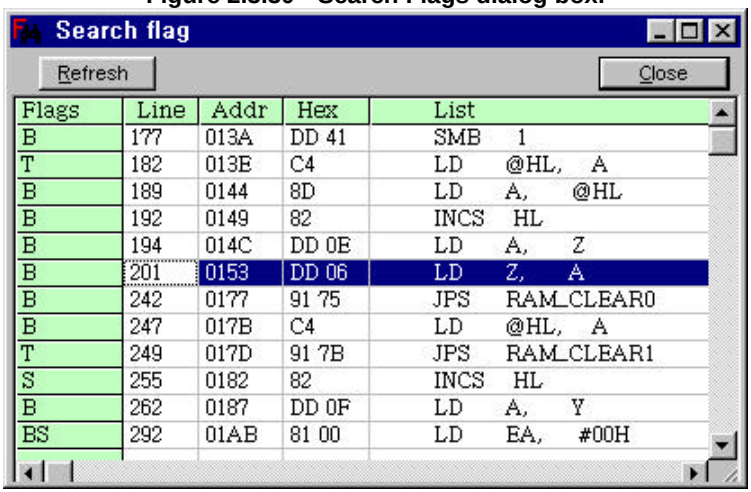
## Debug|Clear All Flags

This menu will clear all flags including break, trigger, and stop point flags in the debug screen. This menu is supported by shortcut operation using the icon for 'Clear all flags' in the toolbar.

## Debug|Search Flags

Display all flags that were already set. Search for the selected flag in the debug screen by double-clicking the listed flag column in the Search Flags dialog box. This menu is supported by shortcut operation using the Shift+F4 key and the icon for 'Search Flags' in the toolbar.
Here is what the dialog box looks like:

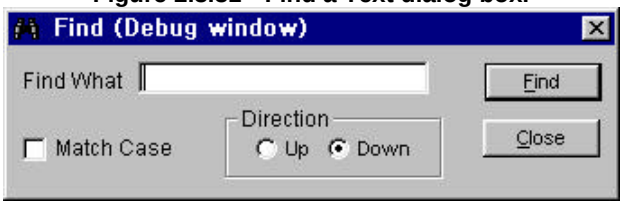
**Figure 2.3.30 Search Flags dialog box.**



## Debug|Search Text

Search for the selected text in the Debug screen. The Debug|Search menu displays the Find a Text dialog box, which lets you type in the text you want to search for and set options that customize the search. This menu is supported by shortcut operation using the Ctrl+F key and the icon for 'Find a text in the debug window' in the toolbar. Here is what the dialog box looks like:

**Figure 2.3.31 Find a Text dialog box.**




In the Find a Text dialog box we can select a text to search for.

**[ Match Case check box ]**
Check the Match Case check box if you do want to search with differentiating uppercase from lowercase.

**[ Direction radio button ]**
Choose among the Direction buttons to determine where to search begins. The down button is chosen by default.

## Debug|Reset Target MCU

The Debug|Reset Target MCU menu will reset the target CPU in the development system to the initial point. If you have any problem such as lock-up during debugging operation, please reset the target CPU to escape from the lock-up. For shortcut operation, F9 key and the icon for 'Reset target CPU' in the toolbar are supported.

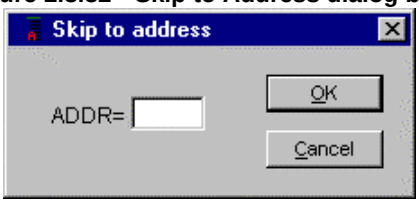
## Debug|Run/Stop

The Debug|Run/Stop will start unconditional execution in the debug mode of the Smart Kit emulator system. Breakpoints are ignored. This menu toggles Run or Stop operation. The register view windows on the screen are updated with their new contents after stop operation is executed. For shortcut operation, F4 key, the icons for 'Run(Go)' and 'Stop(Break)' in the toolbar are supported.

## Debug|Skip to Address

The Debug|Skip to Address menu will open Skip to address dialog box, which let you allow entry of an address in the text input box. If press Enter key (or click OK button) after typing your desired address in the Address input box, The program will go to the set address without any instruction execution. For shortcut operation, F5 key and the icon for ‘Skip to Address’ in the toolbar are supported. Here is what the dialog box looks like:

**Figure 2.3.32 Skip to Address dialog box.**




## Debug|Skip to Cursor

The Debug|Skip to Cursor menu will allow the program to skip to the current cursor position without any instruction execution. For shortcut operation, F6 key and the icon for ‘Skip to cursor’ in the toolbar are supported.

## Debug|Go to Cursor

The Debug|Go to Cursor menu will allow the program to run up to the current cursor position. The program is executed like breakpoint run. For shortcut operation, F7 key and the icon for ‘Go to cursor’ in the toolbar are supported.

## Debug|Single step run

Perform single step instruction execution. The register view windows on the screen are updated with their new contents. For shortcut operation, F8 key and the icon for ‘Single step run’ in the toolbar are supported. At very low target processor clock speed (Example: 32KHz sub clock operation), a single instruction step can take a few seconds to be completed. The 'WAITING' message will be displayed while waiting for the single-step to be completed. While this operation can be interrupted, the target processor may be left in an unstable state if the single step is not allowed to complete.

## Debug|Step Over

The Debug|Step Over menu will allow the program to run up to the next instruction step without entering single step operation in the subroutine. The program is executed like breakpoint run. For shortcut operation, Shift+F8 key and the icon for ‘Step over subroutine call’ in the toolbar are supported.

## Debug|ROM Break run

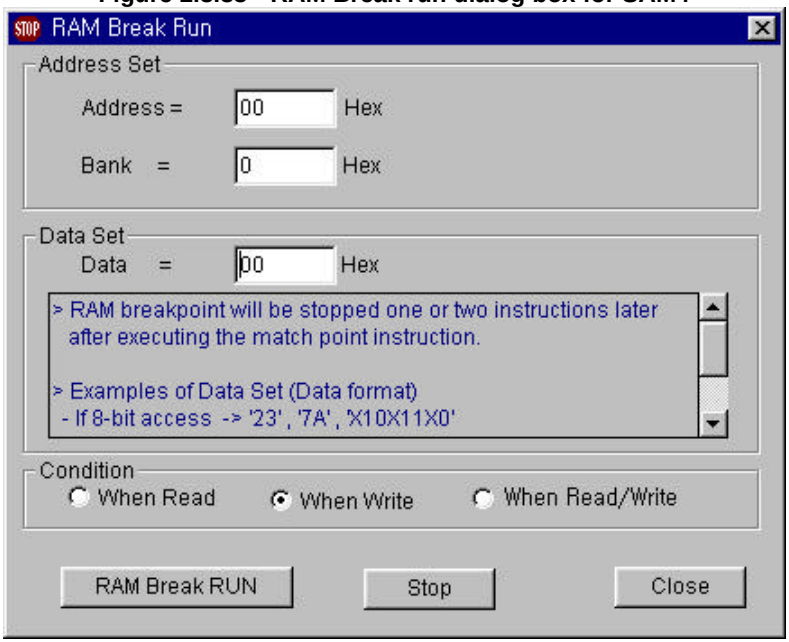
The Debug|ROM Break run menu commences execution and waits until a breakpoint (Address break stop) is reached. Set a breakpoint (Flag ‘B’) somewhere in your code that will get executed at least once. (Currently if you forget to set a break point that never becomes hit, then breakpoint run will wait forever or until you click ‘Stop’ icon in the toolbar or press F4 key.)

The register view windows on the screen are updated with their new contents when a breakpoint is reached. Breakpoint flag can be set or clear by clicking the left mouse button in the FLG column of the debug window or press letter ‘B’ at the current cursor bar. For shortcut operation, F11 key and the icon for ‘Go to breakpoint’ in the toolbar are supported.

## Debug|RAM Break run

The Debug|RAM Break run menu displays the 'RAM Break Run' dialog box that let you setup RAM break conditions for SAM4, or SAM8 device. Also this menu commences execution and waits until a break condition is reached.

**Figure 2.3.33 RAM Break run dialog box for SAM4**




**[ Address Input Box ]**
Types a value between 0h and FFh directly to specify an address that you want to examine. Hexadecimal is default radix.

**[ Bank Input Box ]**
Types a value between 0h and Fh directly to specify a bank that you want to examine. Hexadecimal is default radix.

**[ Data Input Box ]**
The data is a specified hexadecimal or a binary condition that you want to match pattern.
If you want to examine only the specified bit for data matching, you can use don't care symbol (X) for other bits. See Ex.2) of examples.

**< Examples >**

```
Ex.1)       smb0                ; select Bank0
            ld      EA,#45H     ; EA<-45h
            ld      30H,EA      ; load 45H to the data memory with 30H
```

In this case, you can set a condition as follows.

| Bank | Address | Data |
|---|---|---|
| 0 | 30 | 45 |

Ex.2) When the stop condition set is 'Data=45H, bit0=1', the only bit0 is used.

| Bank | Address | Data |
|---|---|---|
| 0 | 30 | XXXXXXX1 |

**[ Condition radio buttons ]**
Decides whether to stop on read, write or read/write of data to the RAM pointed by address.

**< Read mode >**
This is a comparison mode when input condition is read operation.

```
            ld      02H,00H         ; 00H data is read operation
```

**< Write mode >**
This is a comparison mode when input condition is write operation.

```
            ld      02H,00H         ; 02H data is write operation
```

**< Read/Write mode >**
This is a comparison mode when the input condition is the read or write operation.

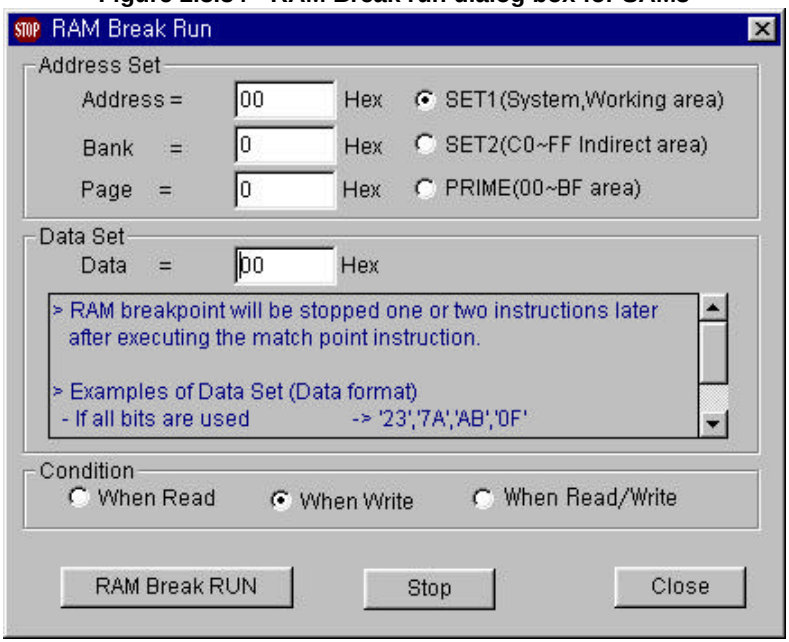
**[ RAM Break RUN button ]**
The RAM Break run will start until the break condition is encountered.

**[ Stop button ]**
The RAM Break running will be stopped.

**[ Close button ]**
This button will close the RAM Break run dialog box.

**< SAM8 >**

**Figure 2.3.34 RAM Break run dialog box for SAM8**




**[ Address Input Box ]**
Types a value between 0h and FFh directly to specify an address that you want to examine. Hexadecimal is default radix.

**[ Bank Input Box ]**
Types a value between 0h and 1h directly to specify a bank of the SET1 area that you want to examine. Hexadecimal is default radix. There is not any bank register in the KS86C series (S3C9xxx series) of SAM8.

**[ Page Input Box ]**
Types a value between 0h and Fh directly to specify a page that you want to examine. Hexadecimal is default radix.

**[ SET Radio buttons ]**
The SET radio buttons can be used to choose SET1, SET2, PRIME area of RAM breakpoint conditions. There is not any SET registers in the KS86C series (S3C9xxx series) of SAM8.

**[ Data Input Box ]**
The data is a specified hexadecimal or a binary condition that you want to match pattern.
If you want to examine only the specified bit for data matching, you can use don't care symbol (X) for other bits. See Ex.4) of examples.

**[ Conditions radio buttons ]**
Decides whether to stop on read, write or read/write of data to the RAM pointed by address.

**< Read mode >**
This is a comparison mode when input condition is read operation.

```
ld      02H,00H          ;00H data is read operation
```

**< Write mode >**
This is a comparison mode when input condition is write operation.

```
ld      02H,00H          ; 02H data is write operation
```

**< Read/Write mode >**
This is a comparison mode when the input condition is the read or write operation.

**< Examples >**

```
Ex.1)  ld      IMR,#33H        ; SET1, Bank0, Address of IMR=DDH, Data=33H
```

In this case, you can set a condition as follows:

| SET | Bank | Page | Address | Data |
|---|---|---|---|---|
| 1 | 0 | x | DD | 33 |

```
Ex.2) When the register 00H=D0H, register D0H=55H, Page=1.
       ld      02H,@00H        ; register 00H=D0H, D0H=55H
                               ; after execution, register 02H=55H
```

The condition set that data 55 is ***read into address D0*** is as follows.

| SET | Bank | Page | Address | Data |
|---|---|---|---|---|
| 2 | x | 1 | D0 | 55 |

Ex.3) From the example2 above, the condition set which ***data 55 is written into register 02H*** is as follows:

| SET | Bank | Page | Address | Data |
|---|---|---|---|---|
| x | x | 1 | 02 | 55 |

Ex.4) From the example3 above, the condition set when data bit7,5,3 and 1 are set to 0 is as follows:

| SET | Bank | Page | Address | Data |
|---|---|---|---|---|
| x | x | 1 | 02 | 0X0X0X0X |

In this case, the stop operation is triggered when data value is one of 0, 1, 4, 5, 10H, 11H, 14H, 15H, 40H, 41H, 44H, 45H, 50H, 51H, 54H or 55H.

**[ RAM Break RUN button ]**
The RAM Break run will start until the break condition is encountered.

**[ Stop button ]**
The RAM Break running will be stopped.

**[ Close button ]**
This button will close the RAM Break run dialog box.

**< Note 1 >**
The stopped address cased by the RAM breakpoint condition is supposed to be the next instruction. However, the stopped address becomes an instruction one or two later from the instruction that the target CPU reaches the RAM breakpoint condition, because the RAM breakpoint condition is matched to the actual target chip status after the instruction is executed. A user can see results of the Run History function in order to understand the RAM breakpoint results easily.

**< Note 2 >**
Some of the System registers such as SP(Stack Pointer ), T0CNT(Timer/Counter), SYM(System mode register), PWM0(PWM data register) can not be met the break status even though their data is reached to the breakpoint value if their data was changed by CPU (Ex: SP changed by PUSH,POP instructions) or the internal signals( Ex: Timer and PWM data changed by internal clock). However, they can be stopped if they are changed by the data path instructions such as LD SP,#33h or LD T0CNT,#45h.

## Debug|Continue

The Continue menu pulls down the sub-menu box that contains Single step run and ROM break run menu items. This menu commences the execution of repeating step you select.

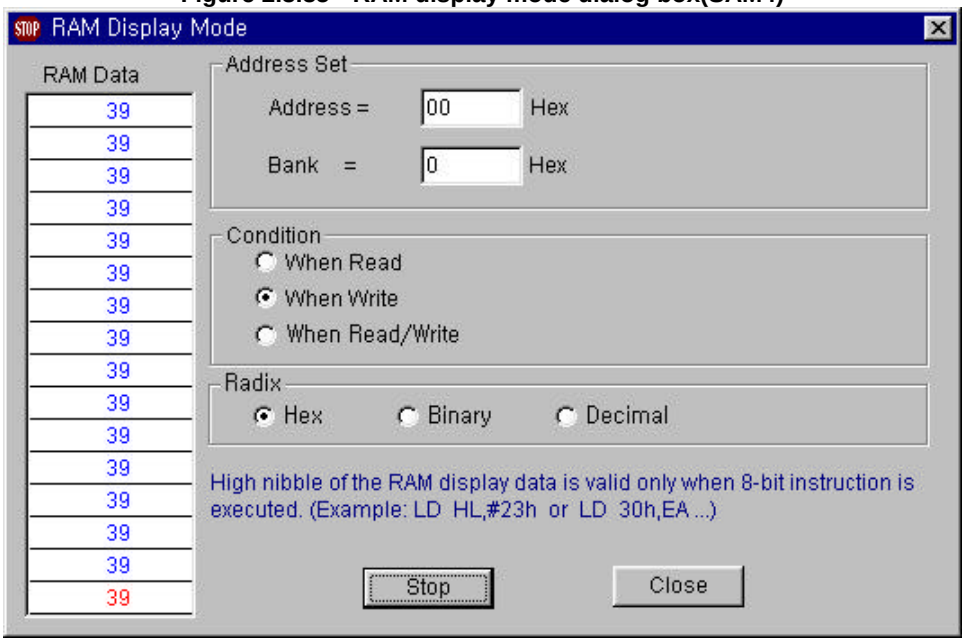
**[ Single step run sub-menu ]**
Perform single step instruction execution continuously until F4 key is pressed or click Stop icon in the toolbar.

**[ ROM Break run sub-menu ]**
The Debug|Continue|ROM Break run menu commences the execution of repeating the Debug|ROM Break run menu (Go to breakpoint), and waits for the user to press F4 key or click Stop icon to terminate execution. The debug and register view windows on the screen are updated after each breakpoint run command is performed.

## Debug|RAM Display mode

When you choose this menu, real-time RAM display mode dialog box is opened on the screen. The Debug|RAM display mode… menu displays the RAM display mode dialog box, which lets you select conditions and address to inspect data during debugging your application programs. In the RAM display mode dialog box, you can specify address, data format (radix), and data path condition. Here's what that dialog box looks like:

**Figure 2.3.35 RAM display mode dialog box(SAM4)**

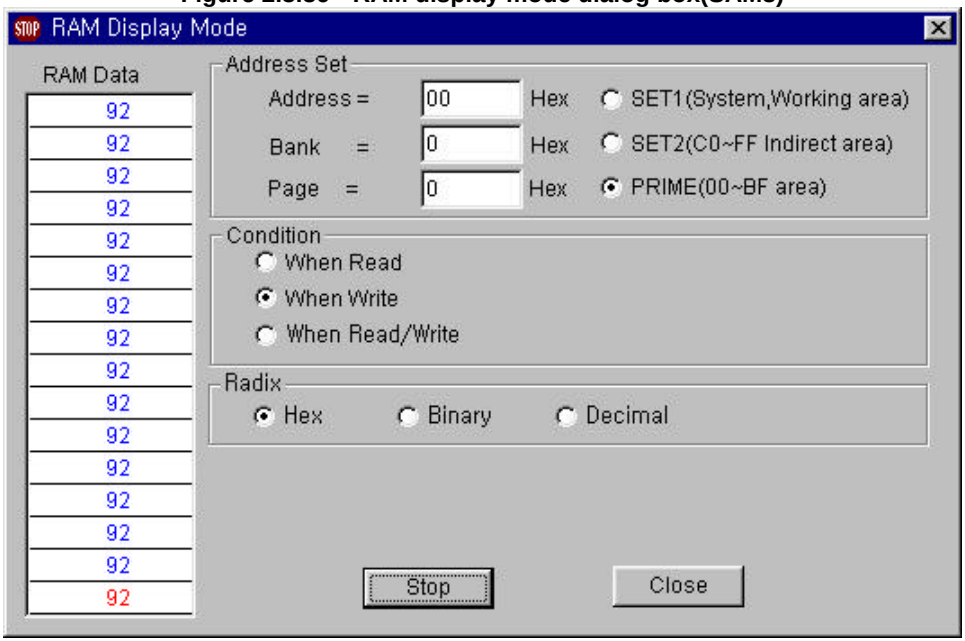



**[ Address Input Box ]**
Types a value between 0h and FFh directly to specify an address that you want to inspect. Hexadecimal is default radix.

**[ Bank Input Box ]**
Types a value between 0h and Fh directly to specify a bank that you want to inspect. Hexadecimal is default radix.

**< SAM8 >**

**Figure 2.3.36 RAM display mode dialog box(SAM8)**




**[ Address Input Box ]**
Types a value between 0h and FFh directly to specify an address that you want to inspect. Hexadecimal is default radix.

**[ Bank Input Box ]**
Types a value between 0h and 1h directly to specify a bank of the SET1 area that you want to inspect. Hexadecimal is default radix. There is not any bank register in the KS86C series (S3C9xxx series).

**[ Page Input Box ]**
Types a value between 0h and Fh directly to specify a page that you want to inspect. Hexadecimal is default radix.

**[ SET Radio buttons ]**
The SET radio buttons can be used to choose SET1, SET2, PRIME area of RAM display address. There is not any SET registers in the KS86C series (S3C9xxx series).

**[ Conditions radio buttons ]**
Decides whether to access on read, write or read/write of data to the RAM pointed by address.

**< Read mode >**
This is a comparison mode when input condition is read operation.

```
ld      02H,00H          ; 00H data is read operation
```

**< Write mode >**
This is a comparison mode when input condition is write operation.

```
ld      02H,00H          ; 02H data is write operation
```

**< Read/Write mode >**
This is a comparison mode when input condition is the read or the write operation.

**[ Radix radio buttons ]**
The Radix radio buttons can be used to choose the data format to be displayed in the RAM data list box.

**< Hex >**
This allows that RAM data is displayed hex-decimal format.

**< Binary >**
This allows that RAM data is displayed binary format.

**< Decimal >**
This allows that RAM data is displayed decimal format.

**[ RAM Data list box ]**
The RAM data list box displays the current RAM data and trace of data list. The current data is updated at the bottom line of list box every 1/10 seconds and previous data is shifted upward.

**[ OK button ]**
The RAM display mode will start to display the RAM data pointed by the address and conditions that are already set.

**[ Close button ]**
This button will close the RAM display mode dialog box.

## Debug|Trace Run

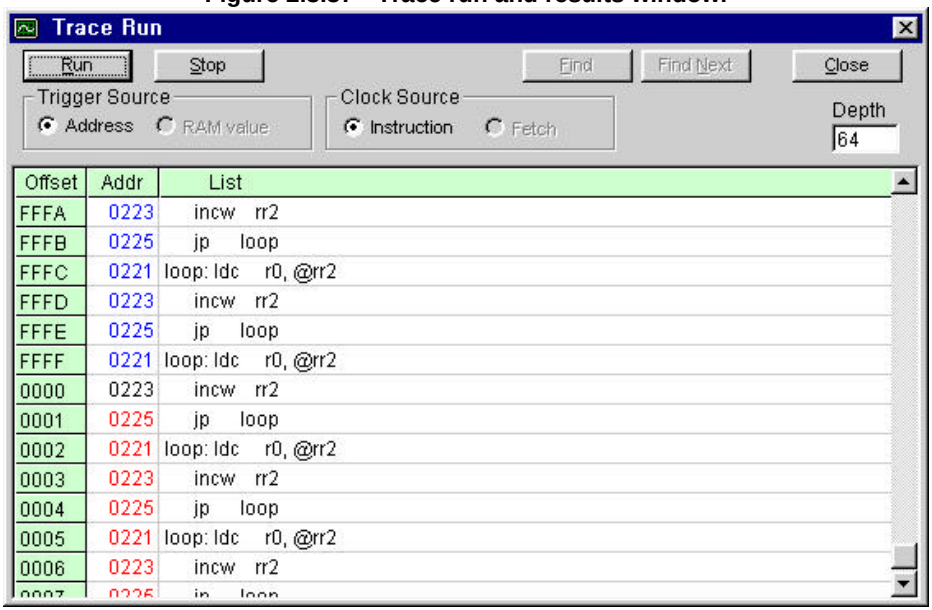
The Debug|Trace Run menu allows the capture of a program running in real-time operation. This menu brings up the trace run window. For shortcut operation, Shift+F7 key and the icon for 'Trace run' in the toolbar are supported.

***Getting the Trace Results***

- Set a trigger point( Flag T ) somewhere in the FLG column of your code that will get executed at least once. (Currently if you forget to set a trigger point that never gets hit, then trace will wait forever or until you press Stop button or Close button in the trace Run window.)
- Select the Debug|Trace Run menu to set the trace conditions, causing the trace buffer to start filling when one condition occurs, and continue until 8,192 of steps (default trace depth) go regardless of the stop flags. You can edit the value of trace depth.
- After setting the trace conditions, you can capture a trace data by clicking Run button. The trace will run until the trigger condition is encountered and brings up the trace results window.

Here is what Trace Run and Results window looks like:

**Figure 2.3.37 Trace run and results window.**




The window displays several instruction execution steps starting at the offset ("0000") in the capture buffer where the trigger condition was encountered. The buffer offset is shown at the left of the

window, followed by an address and instruction source list for the captured step.

**[ Trigger Source radio buttons ]**
The address radio button is chosen by default.

**< Address >**
Program address is selected as trigger condition that is the start point for counting of trigger depth.
**< RAM value >**
The value of RAM specified by the address and data can be a trigger condition. In this case, trigger point will be setup like RAM data breakpoint run operation.

**[ Clock Source radio buttons ]**
The Instruction radio button is chosen by default.

**< Instruction >**
The instruction execution cycle is selected as the sampling clock source for trace capture.
**< Fetch >**
The fetch cycle is selected as the sampling clock source for trace capture.

**[ Run button ]**
The trace run will start until the trigger condition is encountered and finishing trace depth count.

**[ Stop button ]**
Trace running will be stopped.

**[ Depth text input box ]**
Depth value is specified as the offset of the captured trace result. This means how many steps of instructions will be captured in the trace memory after trigger point was matched.

**[ Find button ]**
This button displays the Find a Text dialog box, which lets you type in the text you want to find for and set options that customize the find.

**[ Find Next button ]**
This button repeats the last Find menu. All settings you made in the last Find a Text dialog box used remain in effect when you choose Find Next.

**[ Close button ]**
This button will stop trace run and close the Trace Run window.

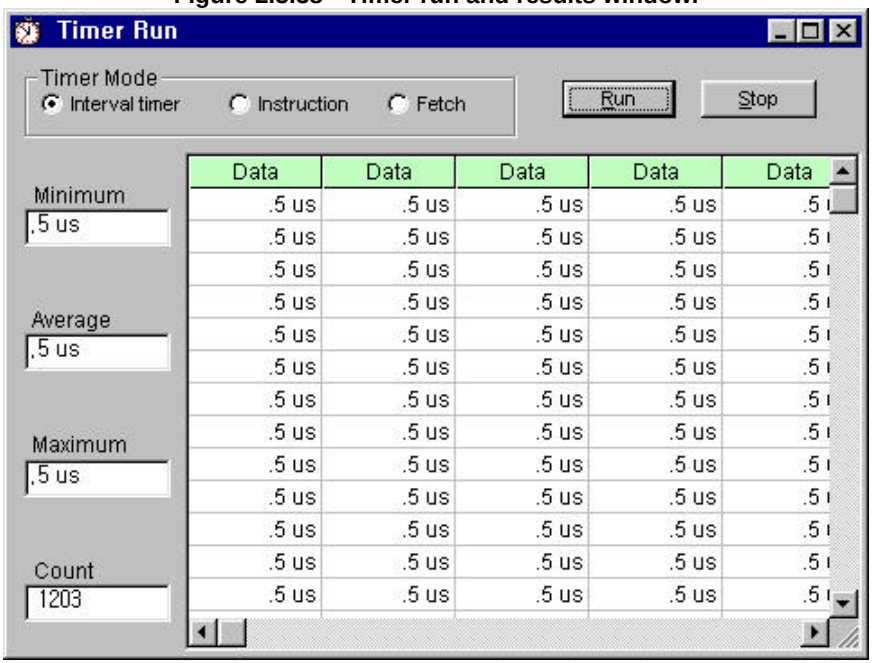
## Debug|Timer Run

The Debug|Timer Run menu allows the measurement of a program section running in real-time Operation. This menu brings up the timer results window. For shortcut operation, Ctrl+F8 key and the icon for 'Timer run' in the toolbar are supported.

***Getting the Timer Results***

- Set a trigger point and a stop point somewhere in your code that will both get executed at least once. (Currently if you forget to set a trigger and stop point, or set a trigger point that never gets hit, then timer will wait forever or until you press Stop or Close button in the Timer run window.
- Select the Debug|Timer Run menu to set the timer mode for timer operation.
- After setting the timer mode, you can capture a timer measurement by clicking Run button. The timer will run until the trigger condition is encountered, and update a new data every capturing trigger and stop flag. Here is what Timer Results window looks like:

**Figure 2.3.38 Timer run and results window.**




The Timer Results window shows 6 columns of numbers (at maximum window size). The Count part

shows the number of the sampling (results of timer) in the next column. The running average, minimum and maximum values are also shown in the left side boxes. The sample data are displayed in units of the current clock for the Instruction or the Fetch mode, and in microseconds if the Interval timer mode is selected.

**[ Timer Mode radio buttons ]**
The Interval timer mode radio button is chosen by default.

**< Interval timer >**
To measure interval time value of real time execution between a trigger and stop point.
**< Instruction >**
To measure the number of instruction execution between a trigger and stop point.
**< Fetch >**
To measure the number of the fetch cycles between a trigger and stop point.

**[ Minimum text box ]**
This value indicates minimum data among the measured data list

**[ Average text box ]**
This value indicates the average value of all of data in the list.

**[ Maximum text box ]**
This value indicates maximum data among the measured data list.

**[ Count text box ]**
This indicates the number of sampling data.

**[ Run button ]**
The timer run will start until the trigger condition is encountered and stop button is pressed.

**[ Stop button ]**
Timer running will be stop.

# Setup

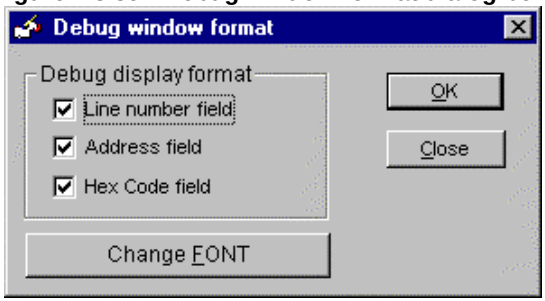
Alt + S

The Setup menu offers choices for debug screen format, single step run count, ROM break run count and breakpoint count.

## Setup|Debug Window Format

Setup Debug window.
The Setup|Debug Window Format menu displays the Debug window format dialog box that lets you select items to be displayed in the debug window such as Line No., Address, and Hex code field. Also, you can choose your desired font type using change FONT button.

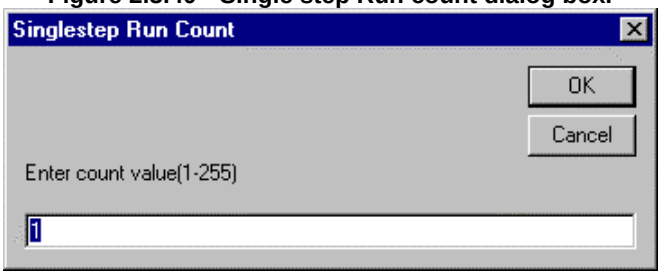
**Figure 2.3.39 Debug window format dialog box.**




## Setup|Single step Run Count

Set number of single step run. The Setup|Single step Run Count menu displays the Single step Run Count dialog box that lets you type value as needed.

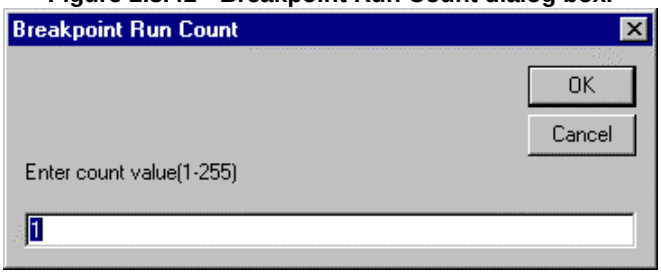
**Figure 2.3.40 Single step Run count dialog box.**




## Setup|Breakpoint Run Count

Set number of breakpoint run. The Setup|Breakpoint Run Count menu displays the Breakpoint Run Count dialog box that lets you type value as needed. This count means how many times breakpoint run will be executed. For example, count value is set to three, breakpoint run will operate three times.

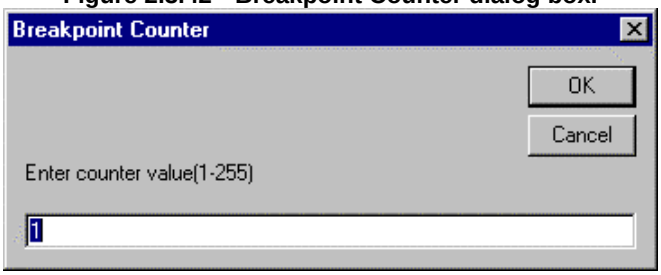
**Figure 2.3.41 Breakpoint Run Count dialog box.**




## Setup|Breakpoint Counter

Set counter (only for ROM Break run). The Setup|Breakpoint Counter menu displays the Breakpoint Counter dialog box that lets you enter counter as needed. This counter means how many same and different breakpoints will be executed. For example, count value is set to three, program execution will stop at the third breakpoint.

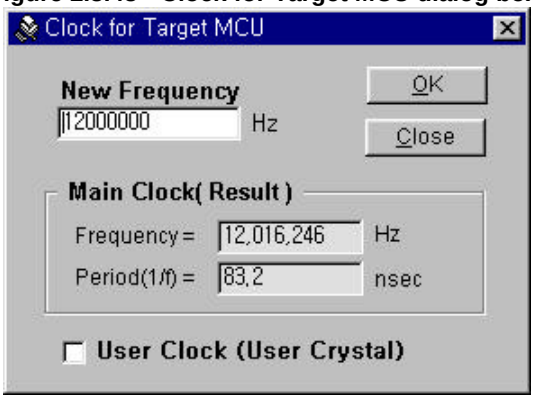
**Figure 2.3.42 Breakpoint Counter dialog box.**




## Setup|Clock for Target MCU

Set the main clock frequency of the target MCU. The Setup|Clock for target MCU menu displays the 'Clock for Target MCU' dialog box that lets you change the clock frequency as needed. The selected clock signal is provided into the main clock input pin of the bond-out chip (Evaluation Chip) processor on the target board via 100-pin POD cable of the Smart Kit.
The default value is 4.194304MHz for SAM4 family and 12MHz for SAM8 family respectively. The setting value of each project will be saved in the project setup file.
(This feature is not supported in SK-400 model or less.)

**Figure 2.3.43 Clock for Target MCU dialog box.**




**[ New frequency text input box ]**
You can type a desired value between 400Khz and 100MHz range in the New Frequency text input box.

**[ OK button ]**
If you press OK button or Enter key after typing a new value in the New Frequency text input box, a different value (a little higher or lower value than the desired value) will be displayed as the result value in the 'Main Clock( Result )' boxes because the Smart Kit generates a frequency as close as the desired value using the frequency synthesis circuitry.

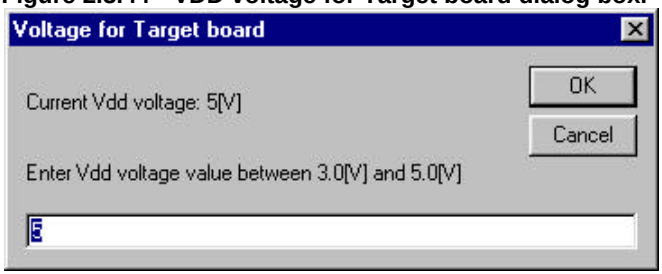
**[ Close button ]**
This button will close the Clock for Target MCU dialog box window.

**[ User Clock check box ]**
If you want to use your own crystal oscillation for your application system, please check the User Clock check box, and insert your crystal into the pin head and change jumper to the user clock side on the PCB after opening the iron case of the Smart Kit. In this case, the frequency synthesis circuit will pass user crystal oscillation to the target board directly.

## Setup|VDD voltage for Target MCU

Set the operating voltage of the target CPU. The Setup|VDD voltage for target MCU menu displays the 'Voltage for Target MCU' dialog box that lets you change the operating voltage of your target MCU as needed between 3.0[V] and 5.0[V]. Please be careful to limit total current capacity within 300mA of your system including the power consumption of the target board. If you need over than 300mA in your applications, you should supply another power source for your system whose voltage is same value as Target board power.
Also please remind that operating speed of the target MCU may be decreased when you select lower voltage than 5V. Please refer to the electrical specifications of your target MCU.
This feature is useful to develop a LCD display application with 3V operation.

**Figure 2.3.44 VDD voltage for Target board dialog box.**



# Option

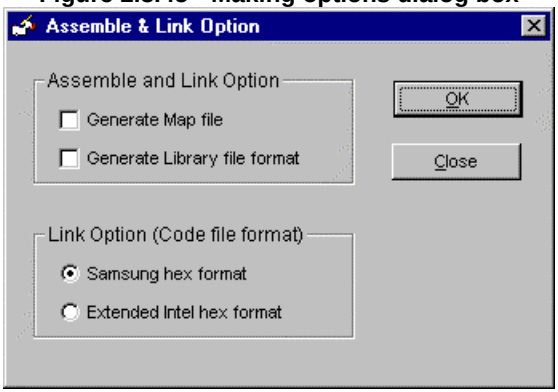
Alt + O

The Options menu contains commands that let you change the default settings of assembling, environment, and editor.

## Option|Assembling

The Assembling menu is Options menu for SASM assembling only. The Options|Assembling menu lets you make Assembling-options settings.

**Figure 2.3.45 Making options dialog box**




**[ Assemble and Link check box ]**

**< Generate Map file >**
This option sets the generating map file with extension *.map on. Default value is not generate Map file. Please refer to SASM assembler in the Part III.

**< Generate Library file format >**
This option sets generate library file format for SASM assembler. For details, please refer to SASM Assembler in the Part III.

**[ Link Option (Code file format) radio buttons ]**

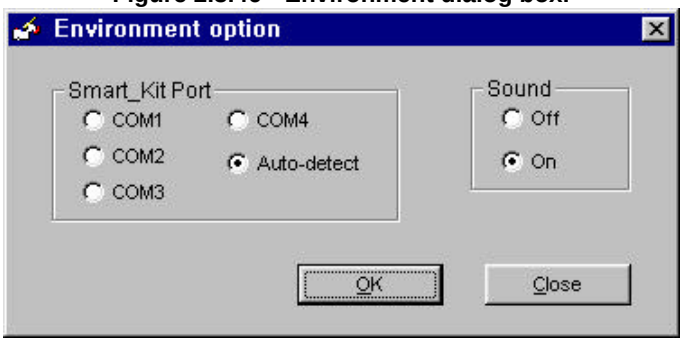
**< Samsung hex format >**
Hex file format is selected as Samsung hex format for SASM assembler. This is selected by default.

**< Intel hex format >**
Hex file format is selected as Extended Intel hex format for SASM assembler.

## Option|Environment

The Environment menu brings up the Environment dialog box, where you can customize the environment of the Smart Studio. The Options|Environment menu lets you make environment-wide settings. The Environment dialog box in the Smart Studio looks like:

**Figure 2.3.46 Environment dialog box.**




**[ Smart Kit Port radio buttons ]**
Serial Port setup. The Smart Kit Port radio buttons are used to select the serial port used for communications between the Smart Kit and the Host computer. The Smart Kit can use either serial port COM1, COM2, COM3, or COM4. It is recommended, however, that COM1 be used, since it has a higher interrupt priority than COM2, and is more reliable at high serial communication rates.

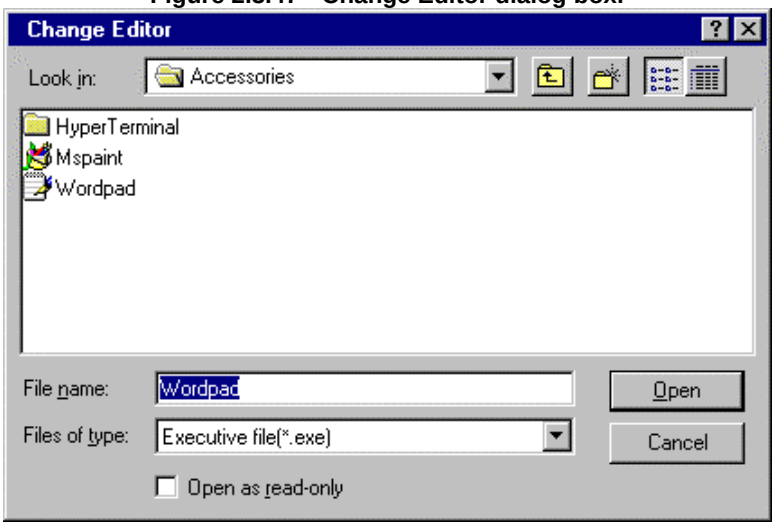
Auto-detect is useful to find COM port that is connected with Smart kit automatically. Also this option is selected by default.

**[ Sound radio button ]**
The Sound radio buttons let you specify whether sound on or off when processing is done or error message is displayed.

## Option|Editor

The Option|Editor menu brings up the Open a File dialog box, where you can customize the editor for the edit source file menu. Smart editor(skEditor) is default as a simple editor for Smart Studio. The Change Editor dialog box in the Smart Studio looks like:

**Figure 2.3.47 Change Editor dialog box.**




You can select your desired editor using the File name text input box, change folder icon, and clicking a file in the File list box of above File-opening dialog box.

# Windows

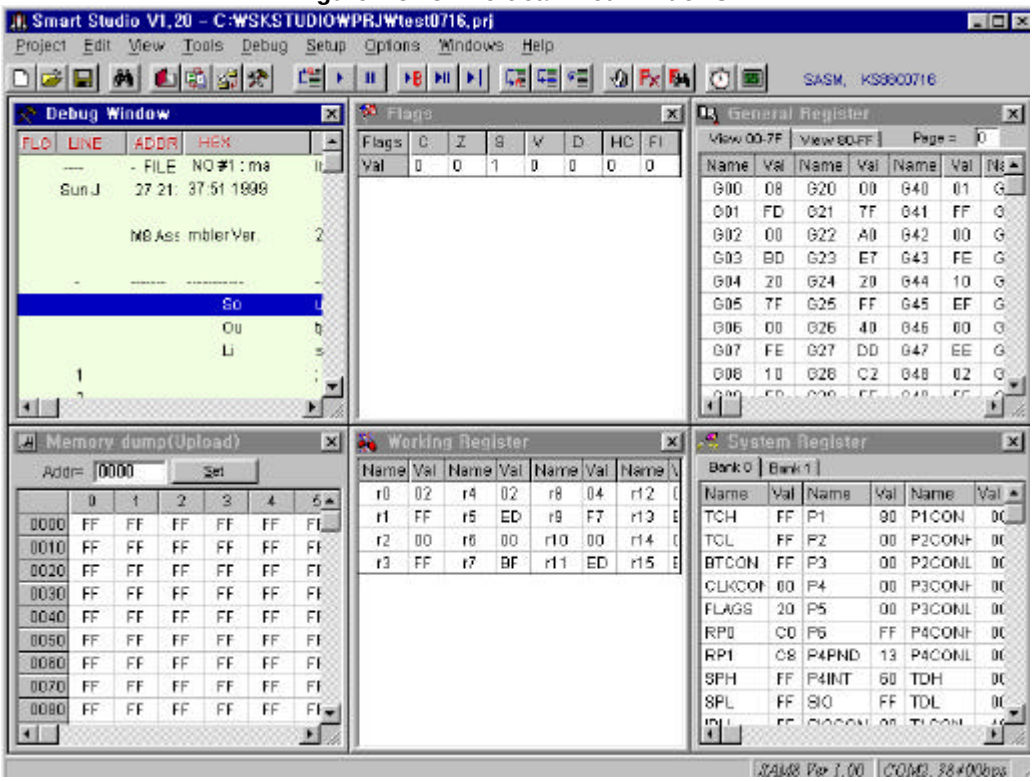
**Alt** + **W**

The Windows menu contains window-management command. Most of the windows in the Smart Studio has all the standard window elements, including scroll bars, a close box, and zoom boxes.

## Windows|Tile Vertical

Choose Windows|Tile vertical to arrange all opened windows on the screen as vertical direction.

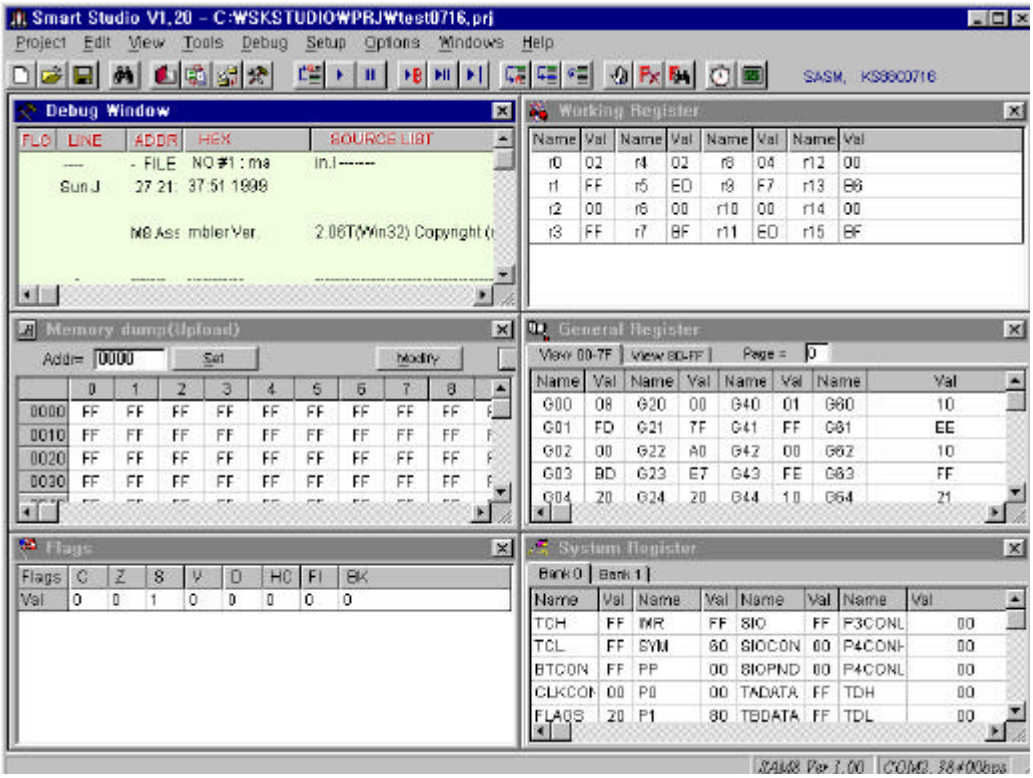
**Figure 2.3.48 Vertical Tiled windows.**



## Windows|Tile Horizontal

Choose Windows|Tile Horizontal to arrange all opened windows on the screen as horizontal direction.

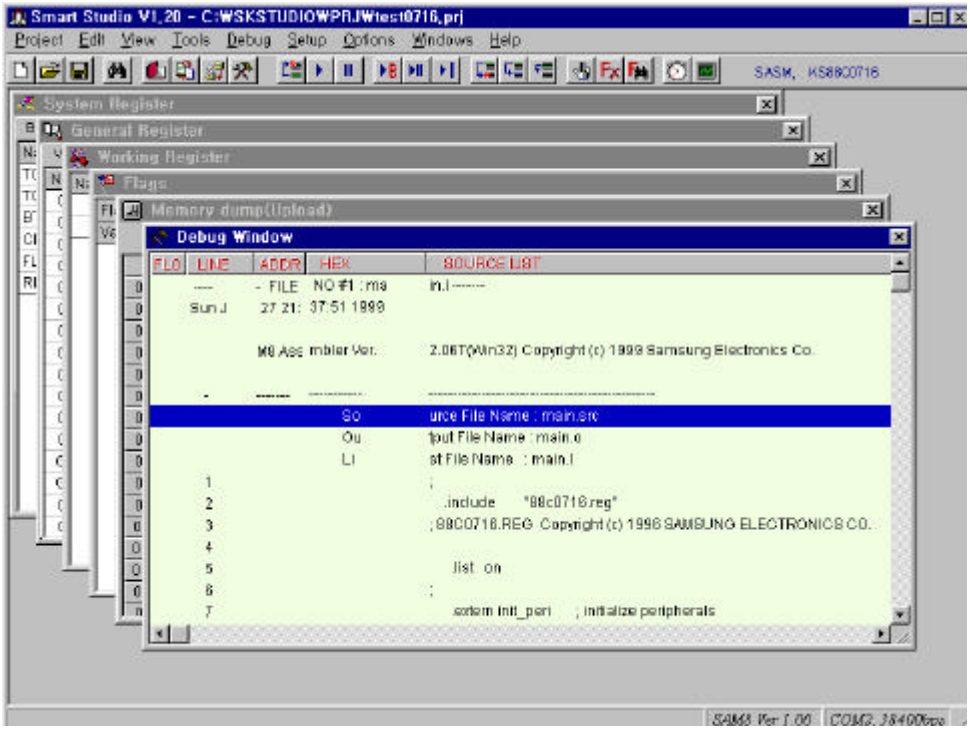
**Figure 2.3.49 Horizontal Tiled windows.**



## Windows|Cascade

Choose Windows|Cascade to stack all file viewers on the screen.

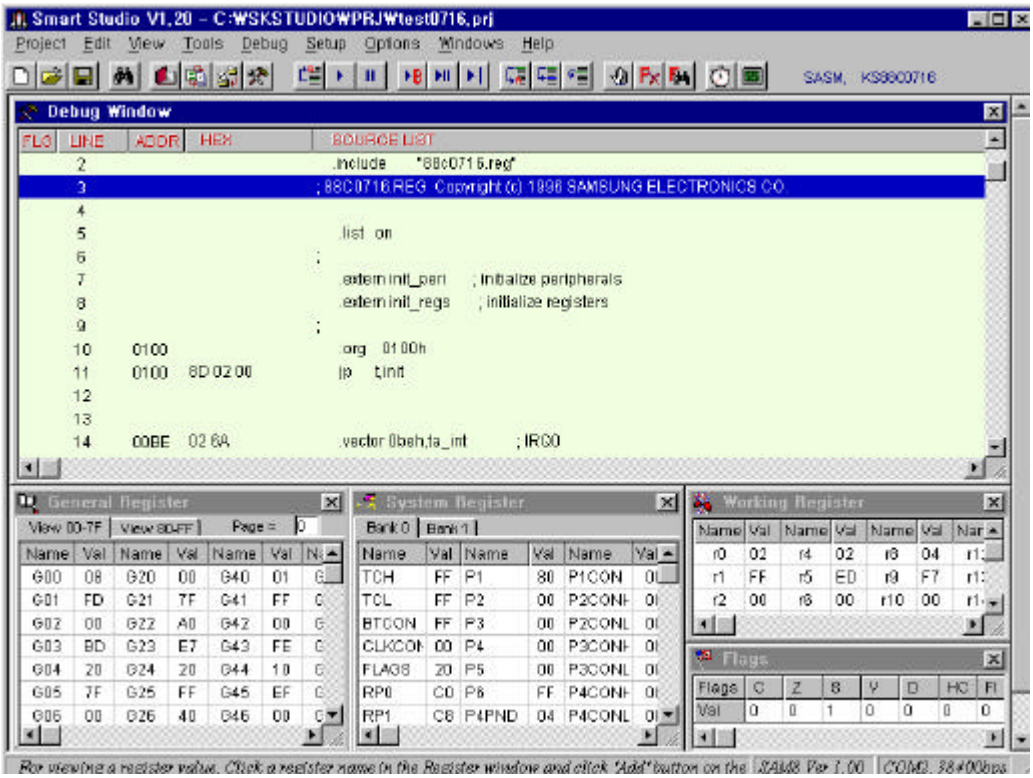
**Figure 2.3.50 Cascaded windows.**



## Windows|Default Screen Setup

Arrange the basic windows format including Debug, Working register, General register, and System register with Flag bit on the screen as the default screen format provided by Smart Studio.

**Figure 2.3.51 Default Screen setup windows**



# Help

Alt + H

The Help menu gives you an access to online help in a special window, connect to C & A Technology home page and information about Smart Studio. There is a help information on virtually all aspects of the Smart Studio.
To open the Help contents window, do one of these actions:

- Press F1 at any time, or click the Help|Contents command menu.
- To close the Help window, click the close box.

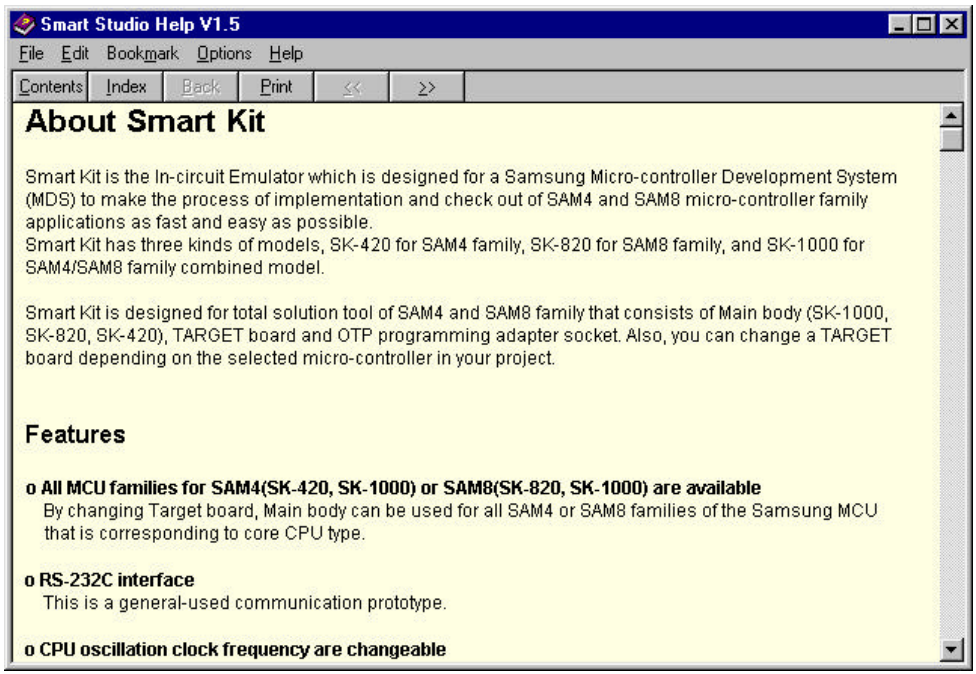
Help screens often contain keywords that you can choose to get more information. With a mouse, you can double-click any keyword to open the Help text for that item.

## Help|Contents

F1

The Help|Contents command displays a full list of Help. Help screens that let you choose Index or contents for moving to a related screen. You can scroll the list. When you find a keyword that interests you, choose it by cursoring to it and double-click it.
The Help contents window in the Smart Studio looks like:

**Figure 2.3.52 Help contents window.**



## Help|C & A Tech. Home Page

The Help|C & A Tech. Home page menu will connect you to the Home page of C & A Technology for new information or file download. You can download a new device.def, device.reg and device.mon files for Samsung's new products which are needed to operate assembler (SAMA, SASM) and Smart Studio debugger. Also, all of the updated software supported by C&A can be downloaded to the your PC. The Home page window in the Smart Studio looks like:

**Figure 2.3.53 Home page window.**

## Help|About

When you choose the About command from the (Help) menu, a dialog box appears, showing copyright, URL of C & A technology home page, E-mail address, and version information of Smart Studio. To close the box, click close box, or click the OK button.

**Figure 2.3.54 About Smart Studio**

# Part III

## Samsung Assembler (SASM)

# 3. Sasm Assembler

## Overview

Samsung re-locatable assembler takes a source file containing assembly language statement and translates it into a corresponding source code, object code, and comments. The assembler supports macros and conditional assembly. This assembler is written in C and runs on the DOS box in the WIN95 operating system. The assembler produces the re-locatable object code only successfully assembled, object files then can be linked and loaded into memory. For a description of the architecture of the SAM4 or SAM8 family of micro controllers, refer to the SAM4 or SAM8 User's Manual.

## Relocation and Linking

Relocation refers to the ability to combine a program module and its data to a particular memory area after the assembly process. The output of the assembler is an object module that contains enough information to allow the linker to assign that module to a memory area. Since many modules can be loaded together to form a complete program, an inter-module communication is needed. For example, one module can contain a call to a routine that was assembled as part of another module and is located in some arbitrary part of memory. Therefore, the assembler must provide information in the object module that allows the linker to link inter-module references. To achieve relocation, addresses are altered at linkage time for instructions that reference memory locations, and data values that serve as pointers to memory locations. This process is not difficult to the programmer.

#### Inter-module reference

The Samsung assembler supports two pseudo-opcodes (or assembler directives), **.global** and **.external**, so that instruction which is located in one assembler module can refer to **names** (either data values or entry points) located in other modules. The **.global** means that the listed names are defined in this module and are available for use by other modules. The **.extern** means that the names are used in this module, but are defined in another module in which they are declared as global.

The global name can be either absolute or re-locatable. A portion of the object module contains a list of both the global that is defined in the module and the externals that the module references. One function of the linker is to match all the externals with the appropriate global, so that every instruction will reference the correct address during program execution.

### Sections

Programs can be divided into sections that are mapped into various areas of memory when linked or loaded for execution. A single module can contain several sections that are allocated to a different area in the program or data memory. Likewise, portions of a section can be spread through several different modules and automatically combined into a single area by the linker. Sections provide the programmer with complete control over the memory mapping of a program without requiring absolute addressing.

## The Win32 Assembler Tools

The new Samsung re-locatable assembler language tools are developed as Win32 applications, that is, they can only be run under 32 bit Windows family of operating systems, such as Window95.

### Assembler command line options

Following assembler options will be supported:

| | |
|---|---|
| -e xxx | Set maximum number of errors (default value 15) |
| -E xxx | Output error information into file xxx |
| -m xxx | Set macro expanding level in list file (default value 64) |
| -S | Output SLO format debugging information into object file |
| -V | Display Version |

### File extensions

The assembler tools have several inputs or output file types. The input of assembler is the assembly source file and the included source files. The output of assembler is a list file showing the assembly results and an object file. The linker can accept input of several object files and library files. After linking, various kind of output files can be generated. The hex file and binary file contain code information. SIT file and DBG file are suitable for debugging. LST file is a list of source with final code, which is also used by debugger, MAP file contains section and global symbol table information.

The Samsung assembler tools will use the following default file extensions to identify different types of files:

| Extension | Type of file | Output from | Input to |
|---|---|---|---|
| .SRC | Assemble source program | Text Editor | Assembler |
| .O | Object file | Assembler | Linker |
| .L | List file | Assembler | |
| .OL | Library object file | Assembler | Librarian |
| .LIB | Library file | Librarian | Linker |
| .HEX | Hex code file | Linker | Debugger |
| .B | Binary code file | Linker | |
| .LST | Merged list file | Linker | Debugger |
| .MAP | Map file | Linker | |
| .SIT | SLO dbg file | Linker | |
| | COFF dbg file | Linker | |

# *Chapter 1* Syntax

A source program is composed of assembly language statements. Each statement must be completed in one line. a line may contain a maximum of 255 characters, and longer lines are truncated and lost.

The Samsung assembler statement may have as many as four fields. These fields are identified by their order within the statement and/or by separating characters between fields. The general format of the assembler statement is:

| (Label) | Opcode | Operands | (Comment) |
|---|---|---|---|

- The label and comment fields are optional.
- The opcode and operands fields are interdependent.
- An empty (null) or blank statement is allowed.
- Labels are case sensitive. However, machine opcodes, pseudo opcodes, condition codes and working registers are not case sensitive.
- Machine opcode, pseudo opcode, macro name and section name can be located in opcode field.

## Label field

Any statement that is referenced by another statement must be labeled, and any statement can contain only one label. When a label is being defined, it can begin in any column when immediately followed by a colon(:). If a colon is not used, the label must begin in column 1. A global label can be declared by placing two colons ( :: ) after the label on the line where it is defined (Example, label1:: ). The maximum identifiable character length of a label is 31 of characters. If you define label that has more than 31 characters, the assembler truncates the remainder of characters.
The label character set and regular expressions are as follows:

```
FIRST_CHAR          :='_' | 'A'-'Z' | 'A' -'Z'
SECOND_CHAR         :=FIRST_CHAR | '0'-'9'

LABEL_CHAR          :=FIRST_CHAR+(SECOND_CHAR)*

COLUMN1             :=^LABEL_CHAR
WITH_COLON          :=LABEL_CHAR:
WITH_COLON2         :=LABEL_CHAR::
```

**LABEL:=COLUMN1 | WITH_COLON | WITH_COLON2**

| | |
|---|---|
| \| | means alternation. |
| - | means range, e.g. 'a'-'z' means 'a' \| 'b' \|…….\| 'z'. |
| + | means concatenation. |
| * | means repeat from 0. |
| [ ] | means optional. |
| ^ | means beginning of a line or statement. |

**The following examples show some uses of label definition.**

```
1 3 5 7 9               ;column
jp_addr:                ;ok
jp_addr                 ;ok
      jp_addr:          ;ok
      jp_addr::         ;ok
      jp_addr           ;illegal, label should start at column 1
        _jp_addr:       ;ok
_jp_addr1:              ;ok
_123456789:             ;ok
1123456789:             ;illegal, label cannot begin with digit
_abc?^&*():             ;illegal, special characters not allowed in label
l123456789:             ;ok
```

## Opcodes and Operands

An opcode is a mnemonic that represents an instruction. Machine opcodes, pseudo opcode, macro name and section names are located in opcode field. Pseudo-opcode field can start with dot(.) for program readability. An opcode in a program statement can be followed by one or more operands, separated by commas if two or more operands are used. The operand can be direct number that is to be used by arithmetic operations, or the address of what is to be used by operation. A carriage return always serves as a statement delimiter. No more than one statement can be on a single line, and a single statement cannot span more than one line.

## Condition Codes

Condition codes are recognized only as operands of instructions that take them.
For example, the statement:

```
JR              Z,LABEL
```

caused Z to be treated as the condition code for zero.

## Comments

A comment is any string of characters following a semicolon (;) in a statement line. Comments have no functional effect on assembling process of a program - they are used only for documentation. Comments can be located at any column of a line, and a line can be filled with only a comment. Comments terminate at the end of the line.

# *Chapter 2* Addressing Modes

## SAM 8 Addressing Modes

The addressing modes in assembler are listed below, for more detailed information see the SAM8 user's manual.

**1. Immediate**

```
LD      R0,#12h
LD      R0,#equ_data
ADD     r1,#34h
```

**2. Register**

```
LD      R0,12h
LD      R0,equ_data
ADD     r1,12h
```

**3. Register Indirect**

```
LD      R0,@12h
LD      R0,@equ_data
ADD     r1,@12h
```

**4. Relative**

```
JR      t,78h
JR      t, _main
```

**5. Direct Address**

```
JP      t,100h
JP      t, _main
CALL    _main
```

**6. Indirect Address**

This addressing mode is different from the SAM8 user's manual. To use indirect addressing mode, you must begin with #.

```
CALL    #12h
CALL    #table          ; indirect address
CALL    table           ; direct address
```

**7. Indexed**

```
LD      R0,#12h[r4]     ; 8 bit effective address
LD      #12h[r4],r1
LDE     R0,#1234h[rr2]  ;16 bit effective address
LDE     #1234h[rr2],r1
```

## SAM 4 Addressing Modes

The addressing modes in assembler are listed below, for more detailed information see the SAM4(KS57-series, S3C7xxx series) user's manual.

**1. Immediate**

```
LD      A,#0Fh
LD      EA,#equ_data
ADD     EA,#34h
```

**2. Register**

```
LD      A, 0Fh
LD      EA,equ_data
ADD     EA,12h
```

**3. Register Indirect**

```
LD      EA,@HL
LD      @HL,A
ADD     A,@HL
```

**4. relative**

```
JR      @EA
JR      _main
```

**5. direct address**

```
JP      100h
JP      _main
```

# *Chapter 3* Pseudo-Operations

## Notations

| Mnemonics | Descriptions |
|---|---|
| ll | Label required |
| l | Label optional |
| n | Number expression |
| fn | Floating-point number expression |
| s | Character string |
| [ ] | Optional |
| , … | Repetition |
| f | Formal parameter to macro |
| r | Radix set |
| false | Value 0 |
| true | All values except 0 |

All alternative directive names in a title are synonymous and have identical syntax. For example, .org and .origin have same meaning and syntax.

## Relocation Operation

**org/origin**

Set the location counter to the given value. Note that the address set by .org is not an absolute address, but an offset from current the section start. The final address after linking will be the sum of section start and this value. So you can make this address absolute by setting section start address to 0.

Format:

```
        |       .origin       n
        |       .org          n
```

Examples:

```
        Prg:    .section      direct,0
                Prg
                .origin       2000h    ; final address 2000h
```

```
Prg1:   .section    direct,1000h
        Prg1
        .org        0a000h  ; final address 0a000h+1000h
```

**align**

This directive is necessary for avoiding run- time errors when handling Data properly. This directive aligns the next item on a multiple of n bytes. In the SAM 8 family case, you may use .align 2.
In the SAM 4 family case, you may not use .align directive. The effect of this directive is seen after assembling and linking. In order to use this directive properly, the sections in each file should be relocated into the code memory address aligned as 2.

Format:

```
        .align  n
```

Examples:

```
        .align  1       ; this usage is possible, but has no meaning.
                        ; it is for SAM4 family
        .align  4       ; align the program counter to the multiple of 4
```

## Section Operations

The Samsung assembler has the predefined sections CODE (default) and DATA. If you use these sections, they will be linked to each file in the link. You can generate your own section names, including register sections, with the section directive. The name of each section can be up to 31 characters long, and you can have up to 250 of them except predefined sections. You cannot nest sections. After you define a section, you can switch to and from other section by using the section name. It should be noticed that the defining of section does not mean changing code to that section, that is, the section is not changed automatically.
In order to get align effect, the sections in each file have to be relocated with address aligned as 2. Section starts with 0 by default. And the section size is determined not by the real code size but by the last location counter.

## section/sec

Section is used to define specific areas of source code that can be relocated separately.

Format:

```
||          .section  option
||          .sec
```

Examples:

```
;location counter
. . . .          . . . .
; this instruction goes into the code section by default
0100        add             r0,#0
; define a new section named program
            program:        .section
; define a new section named data
            data:           .sec
; still code section
0102        add             r0,#0
; switch to program section
            .program
; these instructions go into the program section
0000        add             r0,#0

; switch to data section
            .data
; these codes switch into the data section
0000        .dw             1234h
            .program
0002        add             r0,#0       ; no operation
```

**section options**

1. DIRECT, ADDR / STACKED
   Link the section at a starting address ADDR. Where, ADDR is the address in CODE memory with the range from 0 to 0FFFFh (even number only). Instead of using a specified address, you can also use STACKED to link the section immediately on the top of the previous section

2. INDIRECT, ADDR / STACKED
   Link the section indirectly at address ADDR. The ADDR is the address of internal register file or external DATA memory. The range of ADDR is also from 0 to 0FFFFh (even number only). And STACKED can also be used instead of an address. If you want this section to be located indirectly at internal register file in run-time, ADDR must be less than 100h. Code in such section is stacked on previous section at code memory. This option is generally used in the compiler for program startup.

3. No options
   If no options provided, the linker will ask the loading type and address for the section during linking time unless linking option –q is used, which means use DIRECT, STACKED options for all sections with no options.

   Examples:

```
 program:        .section        DIRECT, 0
 ; define a new section named program as DIRECT at address 0
ddata:           .sec            DIRECT, 120h
; define a new section named ddata as DIRECT at address 120h
sdata:           .sec            DIRECT, STACKED
; define a new section named sdata as STACKED.
rdata:           .sec            INDIRECT, 80h
; define a new section named rdata as INDIRECT at address 80h

                 .program                ; begin section program
                  .org    50h            ; linking address 50h+0
                 ld      ea, r           ; ea will be 56h.

                 .org    100h            ; linking address 100h+0
                 ld      wx, #(d & 0ffh) ; d will be 120h after link.
                 ldc     ea, @wx         ; ea will be 12h.

                 incs    wx
                 ldc     ea, @wx         ; ea will be 34, and s will be 121h

                 .ddata
           d:    .db     12h     ; the address of d will be 120h after link.
```

```
        .sdata
s:      .db     34h     ; the address of s will be 121 after link.

        .rdata
r:      .db     56h     ; the address of r will be 80h after link.
                        ; but the code 56h is stacked on
                        ; previous section code 56h should be
                        ; transferred to internal register file in run-time.
```

## Label Definition Operations

**equate/equal/equ**

This directive equates a value to a defined name. The intermediate value of n which is equated is in signed 32 bit value. External variable could not be equated, and it is not allowed to refer label that is defined in later part.

Format:

```
||          .equate  n
||          .equal   n
||          .equ     n
```

Examples:

```
hours:      .equate  24
mins:       .equal   60
number:     .equ     65536*2/128
divs:       .equ     ext             ;error
            .extern  ext

equ1:       .equ     equ2            ;error
equ2:       .equ     0
equ3:       .equ     equ2            ;ok
```

**global/public**

This is a global label definition. Global label indicator "::" is used for global variable.

Format:

```
            .global  l,...
            .public  l,...
```

Examples:

```
            .global  function, clock
            .public  main
```

Each of their operands is a symbol defined in the current module, and some of symbols that are declared with .extern directive on other modules can be used in those modules. There can be one or more names separated by commas. The .global directive can occur anywhere within the source text.

**external/extern**

It is used for external symbol reference. An external symbol must be defined as global symbol in another file in order to be used in other modules.

Format:

```
.extern     |,...
.external   |,...
```

Examples:

```
.extern     var, main
.external   hours

ld          r0,var
ld          r1,#main
call        hours
```

This pseudo-opcode specifies that each of the operand is a symbol defined in some other module, but it can be referred in the current module. The .extern directive can occur anywhere within the source text. The .extern directive assigns each name an external mode, which allows the name to be used in arbitrary expressions elsewhere in the module, subject to the rules for external expressions. The directive .equ assigns a value( zero as a default ) to a external symbol. Arithmetic operations for external symbol are only addition(+), subtract(-), high byte operation(<) and low byte operation(>).

Example of Label Definition Symbol usage

```
min::   .equ        60
hour:   .equ        min*60

        .extern     second
        .global     hour

        ld          r0,#min*60+hour
        ld          r1,#second
```

**defvar and setvar**

.defvar defines and initializes variable symbols. If no initial value specified, all variables will be initialized to 0. Symbols defined by .defvar can change their values later using .setvar. (Symbols defined by .equ directive, on the contrary, should always keep same value.)

Format:

```
.defvar [ = num, ]    |,…
```

.setvar is used to assign new value to a variable, which is previously defined by .defvar.

Format:

```
        |       .setvar     num
```

Example1:

```
                    .defvar =10h,a, b, c, d   ;variable a, b, c, d defined
                                              ; with initial value 10h
                    ld        r0, #a          ; r0 = 10h
            a:      .setvar   20h             ; a's value changed to 20h after this line.
                    ld        r0, #a          ; r0 = 20h
```

Example2: set variable symbol value using constant

```
  this_year         .equ      1998
                    .defvar =this_year,date     ; date = 1998
  date              .setvar     date –1         ; date = 1997
  date              .setvar     date +2         ; date = 1999
```

**ram_org**

The RAM pointer takes the value specified in the operand of .ram_org directive. The operand value should be within the range specified by the RAM_RANGE function in the .reg file. This used with .ram_ds usually. See ram_ds for example.

## Data Definition Operations

**vector(SAM8 family only)**

The directive .vector sets the address to a given vector location. And it performs the operation that is a combination of .org and .dw. So, the new location counter value should be determined after .vector directive is used. The vector address is relative address (not absolute address) and only recommended for using interrupt vector declaration.

Format:

```
.vector     vector_addr, service_addr
```

Examples:

```
.vector     0be0h,reset     ; same as
                            ; .org      0be0h
                            ; .dw       reset
.vector     0fe0h,undefined
.vector     0f40h,IRQ_mode
            ; the location counter is 0f6h at this point
```

**vent (SAM4 family only)**

The directive .vent sets various interrupt vector addresses. It has an operand field and the state information of the EMB and ERB should be in the operand field with the address of interrupt service routines. In the case, the mnemonic EMB and ERB are not necessary to be written, only the values are enough to describe the state.

Format:

```
            .ventn      EMB,ERB,service
```

Example:

```
            . org       0h
            .vent0      1,1, Reset
            .vent1      1,0,BTMR
```

If there are unused interrupt in an application, the Vent command must be set carefully when user writes the source program. Especially, in case of S3C7234(KS57C2304), the device has no VENT4.

Followings are the ways to avoid problems caused by unused VENTs:

1. Substitute a NOP for the unused VENT in the instruction sequenced.
2. Declare a new ORG pseudo-command in the place the unused VENT command.
3. Leave unused VENTs, but modify the source problem so that the corresponding interrupt service routines are not allowed to execute.

**byte/db**

This directive allocates byte( 8 bit ) data to the memory.

Format:

```
|       .db      n,...
|       .byte    n,...
```

Examples:

```
data1   .db     1,2,3
data2   .byte   11,12,13

in the SAM8 family

                ld      r0,data1
        ld      r1,data2+1

in the SAM4 family

        ld      E,data1
        ld      A,data2+2
```

**word/dw**

This directive allocates word (16 bit) data to the memory.

Format:

```
|       .dw      n,...
|       .word    n,...
```

Examples:

```
data1   .dw     1,2,3
data2   .word   1234h
```

**long/dl**

This directive allocates long word (32 bit) data to the memory.

Format:

```
        |       .dl       n,...
        |       .long     n,...
```

Examples:

```
        data3   .dl       4,5,6
        data4   .long     9abch
```

**ascii/asciz**

This directive allocates string data to the memory. The directive .ascii does not generate null character at the end of the string. The asciz generates NULL character at the end of the string.

Format:

```
        |       .ascii    s,...
        |       .asciz    s,...
```

Examples:

```
        title:  .ascii    “SAM8 test program.”
        Errmsg: .ascii    “Error1\0”,”Error2\0”

        title:  .asciz    “SAM4 test program”
                          ; NULL is added
        Errmsg: .asciz    “I AM”,”aby”
                          ;NULL is added to each string
```

**float**

The directive .float converts a value to a single-precision floating-point format ( 32 bit ), and 32-bit IEEE real format is used for its type.

Format:

```
        |       .float    fn
```

Examples:

```
        Pi:     .float    3.14159
        mpi:              .float    -3.14159
```

#### double

The directive .double converts a value to a double-precision floating-point format ( 64 bit ), and 64-bit format is used for its type.

Format:

```
        |           .double   fn
```

Examples:

```
        pi:         .double   3.14159
        mpi:                  .double   -3.14159
```

## Reserve Space Operations

#### blkb/block

It reserves n bytes of block with the given value. If the value is omitted in the declaration, the assembler reserves with default value 0.

Format:

```
        |           .blkb     n[,value]
        |           .block    n[,value]
```

Examples:

```
                    .blkb     10        ; 10 bytes of 00
                    .block    4,33h     ; 4 bytes of 33h
```

#### blkw

It reserves n words of block with the given 32bit value. If the value is omitted in the declaration, the assembler reserves with default value 0.

Format:

```
        |           .blkw     n[,value]
```

Examples:

```
                    .blkw     10        ; 20 bytes of 0000
                    .blkw     5,1234h   ; 10 bytes of 1234
```

**defs/ds**

It reserves the number of bytes specified by n, but stores nothing in the reserved area. If several of .defs or .ds are declared sequentially, the reserved area by each directive will be located adjacent to each other, that is, sequentially allocated.

Format:

```
|       .defs    n
|       .ds      n
```

Examples:

```
_i:     .defs    1
_i:     .ds      2
```

In this example, the value of _j will be value of _i+1.

**ram_ds**

This directive allocates RAM or the register space at the current RAM address, and increases the RAM address pointer by the value specified in the operand field. Any equate or label used in the operand must have been defined prior to the .ram_ds directive. A label is required in the label field.

Format:

```
||      .ram_ds         n
```

Examples:

```
        0040    .ram_org        40h


        0000    A       .ram_ds 1       ;=40h
        0000    B       .ram_ds 1       ;=41h
        0000    E       .ram_ds 1       ;=42h
        0000    C       .ram_ds 1       ;=44h
        0000    D       .ram_ds 1       ;=45h
        0000    ;
        0020            .org    20h
        0020 E44140     ld      A,B     ;=ld 40h,41h
        0023 E44544     ld      C,D     ;=ld 44h,45h
```

## Table Reference Pseudo Command

** Though tcall and tjp are pseudo command, they generate codes. **

### tcall(SAM4 family only)

The TCALL - it means table call - command allocates the data for the REF instructions corresponding to the CALL instructions (typically two or three bytes) as one byte in order to compress the code memory size. It should be placed in Reference Table (address 20h to 7Fh) in the program memory.

Format:

```
||      .tcall      |
```

Examples:

```
||      .org        20h
rtci0 .tcall        ABC
rtci1 .tcall        XYZ
        ....
      .org          80h
        ....
ABC ld              A,#04h
        ....
XYZ ld              A,#2ah
        ....
      ref           rtci0           ; same as call ABC
        ....
      ref           rtci1           ; same as call XYZ
```

### tjp(SAM4 family only)

The TJP - it means table jump - command allocates the data for the REF instruction corresponding to the JP instruction in the same manner as TCALL.

Format:

```
||      .tjp        |
```

Examples:

```
        .org        20h
rtci2   .tjp        ABC
rtci3   .tjp        XYZ
        ....
        .org        80h
```

```
ABC     ld      A,#04h
XYZ     ld      A,#2ah
        ....
        ref     rtci2     ; same as JP ABC
        ....
        ref     rtci3     ; same as JP XYZ
```

## Macro Definition Operations

**macro/mac/endm/macend**

Macros provide a method for users to define their own opcodes, and a macro is a portion of a program invoked by its name. It begins and ends with pseudo-ops, and can contain any assembler constructs, including pseudo-ops and macros. Macros are the familiar string substitution method used in the other assemblers. Parameter strings provided in the macro invocation are substituted in the body of the macro. Parameters must be separated by comma. The parameter passing method is called by name, not by value, which means the parameter values are not calculated when macro is called. The parameters are substituted into the arguments of the macro (See the examples below). The statements of the macro body are not assembled at the definition time. As a result, they do not define labels, generate code, nor cause errors until the macro is invoked. Macros must be defined before they are invoked. A macro is invoked by using its name as an opcode at any point once it is defined.

Format:

```
        ||      .macro  f, ...      ;macro head
                ……                  ;macro body
                .endm               ;end of macro define
```

Examples:

```
;Macro definition
add32:  .macro  S1,S2,S3,S4
        add     S1,S3
        adc     S2,S4
        .endm

;Macro use
        add32   r5,r4,r3,r2
;will be expanded as follows
        add     r5,r3
        adc     r4,r2
```

The required label serves as the name of the macro, to be used on invocation. A formal parameter can either be a value or a label.

**Local label**

In general, labels in the macro body are local labels. A local label has a local label format. For example, *locallabel$$*, | *locallabel* |, *locallabel*\., etc. In samsung assembler, there is no difference between the local label format and the global label format. This assembler only allows the programmer to use the local label in macro definition, not in others.

Examples:

```
;Macro Definition
ramclr:     .macro   s1,s2
            ld       r0,s1
            ld       r1,s2

clr:                 ld       r0,#0
            inc      r0
            cmp      r1,r0
            jr       neq,clr
            .endm

;Macro use
            ramclr   #00h,#80h

;will be expanded as follows
            ld       r0,#00h
            ld       r1,#80h
clr:        ld       r0,#00h
            inc      r0
            cmp      r1,r0
            jr       neq,clr
```

## Conditional Assembly Operations

**if/else/endif**

Conditional directives allow the programmer to disable or enable a portion of the source code depending on a pre-determined condition.

Format:

```
        |       .if      n
                  ...                       ; if n is true
                .endif
  or
        |       .if      n
                  ...                       ; if n is true
                .else
                  ...                       ; if n is false
                .endif
```

The .if defines the beginning of the conditional code block and tests n for the true (non-zero) or false( zero ) state. The .else performs the code defined below when if statement is false. The .endif defines the end of the conditional code block. Notice that the .else is optional. Conditional block can be nested. Forward or external expressions cause an error.

Examples:

```
example 1:      .if     value>40
                        add     r0,r1
                .else
                        add     r3,r4
                .endif

example 2:      .if     day==0
                        .ascii  "sun"
                .else
                        .if     day==1
                                .ascii  "mon"
                        .endif
                .endif
```

**ifdef/else/endif**
**ifndef/else/endif**

Conditional directives allow the programmer to disable or enable a portion of the source code depending on whether a symbol is defined before.

Format:

```
    |       .ifdef/.ifndef     |
                                ; assemble if l is defined/not defined
            .endif
or
    |       .ifdef/.ifndef     |
                                        ; assemble if l is defined/not defined
            .else
                                ; assemble if l is not defined/defined
            .endif
```

Note:

1. A symbol is “defined” when
   - The symbol is appeared at the label field, such as program label, section name, macro name, etc.
   - The symbol is defined using .extern, .defvar directives.

2. The directive .global does not make the symbol defined.

Examples:

```
            .global  aa
            .ifdef   aa
            .db      aa      ; -- this line will not be assembled
            .endif

    aa      .equ     100     ; aa defined
            .ifdef   aa
            .db      aa      ; -- this line will be assembled
            .endif
```

3. ifdef and if blocks can be nested

Examples(SAM8):

```
            .global  yy
```

```
                .defvar = 10, a, b
                .ifdef a                        ; true, 'a' defined (a=10)
                        ld   r1, #a+9           ; this line will be assembled
                        .if a>5                 ; this is true, a=10
                                ld      r0, #a  ; this line will be assembled
                        .else
                                ld      r0, #5  ; this line will not be assembled
                        .endif

                        .ifdef  yy              ; 'yy' not defined yet
                                ld      r0, #yy ; this line will not be assembled
                        .endif
                .endif
yy:             .db   10
```

## Assembler Control Operations

**include**

Format:

```
        .include        string
```

Examples:

```
        .include        "reg_def.src"
        .include        "c:\SkStudio\Example\test.src"
```

The directive **.include** includes the source file identified by the parameter string into the source stream immediately. The use of **.include** is to contain items such as files of macro definitions, a list of .extern declarations, a list of **.equ** statements, a list of **.reg** declarations of special registers, or commonly used subroutines into the source stream. The directive **.include** can be used anywhere in a application program. The given file name must follow the normal convention for specifying source file names. The **.include** directive can be used in a file which is included by other file containing .include directive. These directives can be nested up to 8 depth.

This **.include** directive can be used within macro definitions and in conditionals. The file name part should begin and end with "(double quotes). Samsung assembler searches the default directory at first if the included string has no path. The default directory is the DOS system variable named REG. If the file is not found there, the search continues in the current directory.

**radix**

The directive .radix sets default number base. The hexadecimal radix should have h, hex, or hexadecimal, the decimal radix should have d, dec, or decimal, and the binary radix should have b, bin, or binary. The assembler default radix is decimal, and the radix words are not case sensitive.

Format:

```
.radix  r
```

Examples:

```
.radix   h       ; set to hexadecimal
add      r0,#10  ; value of #10 is 16 in decimal
.radix   dec     ; set to decimal
add      r0,#10  ; value of #10 is 10 in decimal
.radix   bin     ; set to binary
.db      011     ; decimal 3
.radix   h       ; set to hexadecimal
.db      011b    ; decimal 3 (not hex 011bh), b here is binary suffix
```

**end**

The .end specifies the end of the source program, and thus any succeeding text will be ignored and will not appear in a listing file. Any label will be assigned the current value of the location counter. Operands are ignored. If a source program does not contain an .end directive, assembly error will occur. .end directive should not be used in included file, instead, an empty line is needed after the last source line to serve for file end.

Format:

```
|  .end
```

# *Chapter 4* Arithmetic Operations

## Arithmetic Operation Definitions

| Operator | Unary / Binary | Priority | Description |
|---|---|---|---|
| ( | - | 1(high) | grouping begin |
| ) | - | 1 | grouping end |
| unary + | unary | 1 | plus |
| unary- | unary | 1 | minus |
| unary > | unary | 1 | low byte (num&0ffh) |
| unary < | unary | 1 | High byte (num&0ff00h)>>8) |
| ~ | unary | 1 | 1s complement |
| * | binary | 2 | multiplication |
| / | binary | 2 | division |
| % | binary | 2 | remainder |
| + | binary | 3 | addition |
| – | binary | 3 | subtraction |
| << | binary | 4 | shift left |
| >> | binary | 4 | shift right |
| & | binary | 5 | Bit-wise AND |
| \| | binary | 5 | Bit-wise OR |
| ^ | binary | 5 | Bit-wise XOR |
| && | binary | 5 | logical AND |
| \| \| | binary | 6 | logical OR |
| == | binary | 6 | equal |
| ! = | binary | 6 | not equal |
| > | binary | 6 | greater than |
| < | binary | 6 | less than |
| >= | binary | 6 | greater than or equal |
| <= | binary | 6 | less than or equal |

## Arithmetic Operation Example

```
ex 1)    ld      r0,#1+2*3/4              ;2
         ld      r1,#000000f0h&input
         ld      r1,#00000003h|input
         ld      r1,#~0ffffff00h
         ld      r1, #(>1234h)            ;34h
         ld      r1, #(<1234h)            ;12h

ex 2)  mins:    .equ    60*24
```

```
                ld      r1,#1234h>>4     ;0123h


ex 3)   .if     (val==val2)&&(val2==val3)
        ld      r3,#45h
        .endif


ex 4)   .if     (val==val2)
                .if     (val2==val3)
                ld      r3,#45h
                .endif
        .endif
```

# *Chapter 5* Punctuation

| Symbol | Name | Description |
|---|---|---|
| : | Colon | Label terminator<br>ex. Loop: |
| :: | Double Colon | Label terminator<br>defines the label as global<br>ex. Loop:: |
| | Tab | Item or field terminator |
| | Space | Item or field terminator |
| , | Comma | Operand field separator<br>ex. LD r1,r2 |
| ; | Semicolon | Line comment indicator<br>ex. LD r1,r2 ;this is a comment |
| . | Period | Bit field indicator<br>ex. LDB r1,3.4 |
| . | Period | Pseudo opcode indicator<br>ex. .org 0020h |
| ‘ | Single quote | Character constant indicator<br>ex. LD r1,’a |
| “ | Double quotes | String constant indicator<br>ex. .ascii ”double quotes” |
| # | Number sign | direct addressing mode indicator<br>ex. LD r1,#2*3 |
| @ | At sign | Indirect addressing mode indicator<br>ex. LD r1,@r3 |
| [ ] | Brackets | Indexed addressing mode indicator<br>ex. LD r1,#2[r3] |
| $ | Dollar | Location counter value<br>ex. JP t,$-4 |

# *Chapter 6* Constant Representation

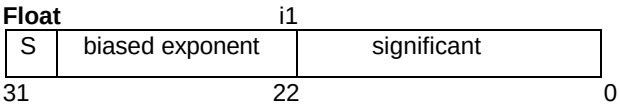
## Number Constant

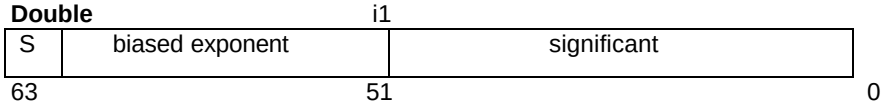
- Default number base is decimal
- Default number base can be changed by .radix. Decimal, hexadecimal and octal number base is permitted. The suffix can be omitted.
- Any number must begin with a digit. (0-9)

| Types | Base Suffix | Digits | Examples |
|---|---|---|---|
| Hexadecimal | h H | 0 - 9, a - f, A - F | 1234h, 0012H, 0aBcDh |
| Decimal | | 0 - 9 | 1234,0012, 0 |
| Octal | o O | 0 - 7 | 1234o,0012O |
| Binary | b B | 0 - 1 | 01000010b |

## Floating-Point Number

The IEEE floating-point formats are used for floating-point types. The float type uses 32-bit IEEE real format. The double type uses 64-bit IEEE real format. The range of 32-bit format is from $3.4x10^{-38}$ to $3.4x10^{-38}$ and 7-digit precision. The range of 64-bit format is from $1.7x10^{-308}$ to $1.7x10^{-308}$ and 15-digit precision.




- S = Sign bit ( 0 = positive, 1 = negative )
- i = Position of implicit binary point
- 1 = Integer bit of significant
- Exponent bias ( normalized values )
  float : 127 (7Fh), double : 1023 (3FFh)

## Character and String Constant

- Character constant: Must begin and end with ‘( single quote )
- String constant: Must begin and end with “( double quotes )
- Special character representation ( C style )

| Sequence | Value | Character | What it means |
|---|---|---|---|
| \ a | 0x07 | BELL | Audible bell |
| \ b | 0x08 | BS | Backspace |
| \ f | 0x0C | FF | Form feed |
| \ n | 0x0A | LF | New line(linefeed) |
| \ r | 0x0D | CR | Carriage return |
| \ t | 0x09 | HT | Horizontal tab |
| \ v | 0x0B | VT | Vertical tab |
| \ \ | 0x5C | \ | Back slash |
| \ | 0x27 | ‘ | Single quote |
| \ | 0x22 | “ | Double quote |
| \ ? | 0x3F | ? | Question mark |
| \ ooo | | any | ooo means a string of up to three octal digits |

But the **.include** directive is not affected by this rule.

# *Chapter 7* External SymbolReference

One of the most important features of the Samsung assembler is re-locatable assembler which means that the address of a module or label value is determined not at assemble-time but at link-time. The symbols have to be declared as global in order to be referred by another file. To declare a global symbol, use .global / .public or global symbol indicator(::).

```
        .public  sym1
        .global  sym2
sym1:   nop
        nop
sym2:   nop
        nop
main::  nop
        nop
```

The symbol defined at another file can be referred by using .extern/.external directive. There are some limitations on using external symbol in expression, for example, only addition, subtraction, low byte operation, and high byte operation are allowed for external symbols, and using more than one external symbols in an expression should also be avoided.

```
.extern  sym1
.extern  sym2,main
call     main              ; ok
jp       c,main            ; ok
ldw      rr0,#sym1+2       ; ok
ldw      rr0,#sym1*2       ; error, multiplication cannot applied to
                           ; extern symbol
ldw      rr0,#sym1+sym2    ; error, more than one extern symbols are
                           ; not allowed in one expression.

.db      >sym1             ; ok
.db      <sym1             ; ok
```

# *Chapter 8* Linker

## Introduction

The linker is used to assign addresses to re-locatable sections in object input modules, and to combine two or more separate object modules into one module. Linking allows programs to be developed as groups of smaller, easier-to-manage modules that can be combined to form a single object module. All of program modules can be merged at linking time.

## General Description

The SAM8 Linker will be a command-line program, which runs under command line OS.

### Host Environment

The command line version of the Samsung linker will operate in MS-DOS environment and DOS box in the 32-bit Windows platform, such as Win95 environment.

### Linker command line options

Following linker options will be supported:

| | |
|---|---|
| -b | Generate binary file |
| -B xxx | Read input libraries from file xxx |
| -cx | Code file format: |
| -cs | Samsung Hex Format (default) |
| -ci | Extended Intel Hex Format |
| -C xx | Fill blanks in binary file with hexadecimal xx (default FF) |
| -f xxx | Read input object file names from file xxx |
| -g | Generate debug information file |
| -L xxx | Linking with library xxx |
| -m | Generate map file |
| -M | Generated merged list file |
| -o xxx | Specify output file name xxx |
| -O | Allow overlay |
| -q | Quiet mode, use DIRECT, STATCK option by default for all sections |
| -S | Output SLO format debugging information file |
| -V | Display version number. |

**File extensions**

The Samsung linker will use the following default file extensions to identify different types of files:

| Extension | Type of file | Output from | Input to |
|---|---|---|---|
| .O | Object file | Assembler | Linker |
| .HEX | Samsung hex code file | Linker | Debugger |
| .LST | merged list file | Linker | Debugger |
| .DBG | debug information file | Linker | |
| .MAP | symbol file | Linker | |
| .SIT | SLO format dbg file | Linker | Debugger |
| .B | Binary file | Linker | |

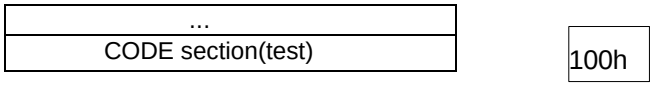
## Linker Examples

These examples illustrate how to use the linker. Examples use <cr> (carriage return) only when a prompt requires no other response. All other inputs must end with a carriage return, which in the examples here ignore.

#### Single file assembled at desired address run

You have one assembled file and only one section.
The linker prompts and responses are:

```
C:>slink88 test
File test.o
Enter offset for 'CODE' : 100
```

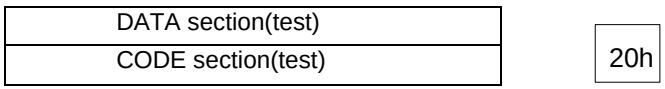



The linker will read the file test.o, add 100h (hexadecimal) to all re-locatable address, and output code file test.hex.

#### Single file with multiple sections

You want to link a single file with multiple sections, using the predefined CODE and DATA sections and stacking the DATA section on top of the CODE section.

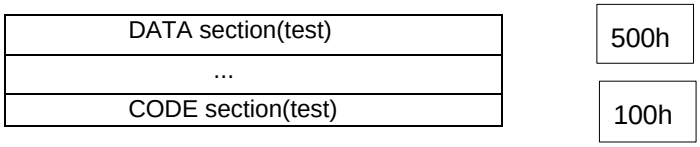
```
C:>slink88 test
File test.o
Enter offset for 'CODE' :20
Enter offset for 'DATA' :<cr>
```




The linker will make DATA section stacked on top of the CODE section. If you want to determine the addresses of the sections, for example, the address of CODE is 100h and the address of DATA is 500h.

```
C:>slink88 test
File test.o
```

```
Enter offset for 'CODE' :100
Enter offset for 'DATA' :500
```

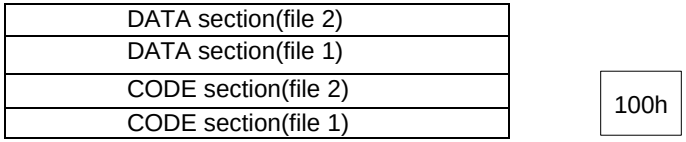



**Multiple files with multiple sections**

You want to link file1 and file2 using CODE and DATA sections, with file1 starting at 100h.

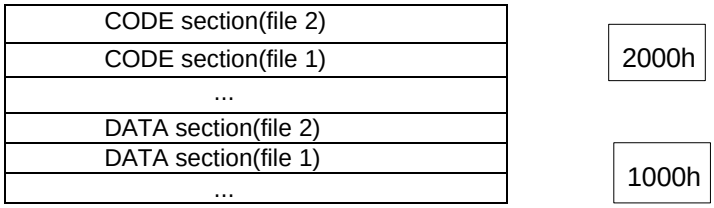
```
C:>slink88 file1 file2
File file1.o
Enter offset for 'CODE' :100
Enter offset for 'DATA' :<cr>
File file2.o
Enter offset for 'CODE' :<cr>
Enter offset for 'DATA' :<cr>
```

This diagram shows how the Linker stacks sections.




You cannot stack each DATA before each CODE. If you need to stack data before code, you have to put your code in user-defined section instead of the predefined CODE section. You want to stack CODE and DATA at specific addresses, with CODE starting at 2000h and data starting at 1000h.

```
C:>link8 file1 file2
File file1.o
Enter offset for 'CODE' :2000
Enter offset for 'DATA' :1000
File file2.o
Enter offset for 'CODE' :<cr>
Enter offset for 'DATA' :<cr>
```




**Indirect linking**

Indirect linking is the linking of a file section to run at an address other than its load address. This is a like phase and de-phase; but whereas phase and de-phase change the address at assembly time, indirect linking changes it at link time. This can be confusing, and it is likely to generate output with addresses so mixed-up that the program cannot run. However, indirect linking is necessary sometimes.

(a) Case 1
You have a single board controller application and your program resides in ROM. The data consists entirely of lookup tables or constants. Since it will never be written, there is no reason to move it out of ROM. You can link normally.

(b) Case 2
You have the same application and program as above, but the data has initialized values that will change as the program runs. You need a start-up routine to move it from ROM to RAM. Indirect linking achieves this by stacking the data normally in ROM but linking it to a run-time RAM address.

- At least the first section in a link must be linked directly so that the linker has a fixed address from which Linker can calculate the indirect addresses.
- Use the \verb+@+ sign before the load address in order to tell the linker to calculate an indirect address.
- Once you tag a section as indirect, the linker automatically gives an indirect tag to every section of the same name in all files.
- Indirect sections are stacked in the same order as if they were direct.

If you have three files, file1.o, file2.o and file3.o, each have three sections, program, const_data and init_data. You want program to reside at 100h, const_data stacked on top of program, and init_data stored in ROM at power-on but running at 2000h in RAM.

```
C:>link8 file1 file2 file3
File file1.o
Enter offset for 'program'      :100
Enter offset for 'const_data'   :<cr>
Enter offset for 'init_data'    :@2000
```

```
File file2.o
Enter offset for 'program'      :<cr>
Enter offset for 'const_data'   :<cr>
Enter offset for 'init_data'    :@<cr>
File file3.o
Enter offset for 'program'      :<cr>
Enter offset for 'const_data'   :<cr>
Enter offset for 'init_data'    :@<cr>
```

# *Chapter 9* Installing and Using SAMASM

## Installing SAMASM

**Distribution Files**
The distribution disk should contain these files:

| | |
|---|---|
| samasm.exe | assembler and linker integrated executable file |
| sasm88.exe | KS88(S3C8xxx) series assembler executable file |
| sasm86.exe | KS86(S3C9xxx) series assembler executable file |
| sasm57.exe | KS57(S3C7xxx) series assembler executable file |
| slink88.exe | KS88(S3C8xxx) series linker executable file |
| slink86.exe | KS86(S3C9xxx) series linker executable file |
| slink57.exe | KS57(S3C7xxx) series linker executable file |
| | |
| 88c????.reg | KS88(S3C8xxx) series MCU system register header file |
| 86c????.reg | KS86(S3C9xxx) series MCU system register header file |
| 57c????.reg | KS57(S3C7xxx) series MCU system register header file |
| S3C8???.reg | KS88(S3C8xxx) series MCU system register header file |
| S3C9???.reg | KS86(S3C9xxx) series MCU system register header file |
| S3C7???.reg | KS57(S3C7xxx) series MCU system register header file |

## What is SAMASM.EXE ?

SAMASM.EXE is the integrated executable assembler driver program for the SAM4 and SAM8 CPU. SAMASM.EXE (-57) calls SASM57.EXE as many times as the number of files to be assembled, and then calls SLINK57.EXE once.

**For example,**

```
c:>SAMASM -57 file1.src file2.src file3.src
```

**is equivalent to**

```
c:>SASM57 file1.src
c:>SASM57 file2.src
c:>SASM57 file3.src
c:>SLINK57 -q file1 file2 file3
```

SAMASM.EXE is designed for convenience of users. Assembling and linking source programs

through SAMASM.EXE is simple, however, for specified relocation of sections at linking time, you may use SASM57.EXE and SLINK57.EXE.

## Running Assembler

To invoke Samsung assembler from the command line, type SAMASM at the DOS prompt and then a set of command-line arguments. The generic command-line format is:

Command [option1[option2...]] filename[filename ...]

Where, command can be one of SAMASM , SASM57, SASM86, SASM88, SLINK57, SLINK86 or SLINK88. Each command-line option may be preceded by either a hyphen (-) or slash (£ ), whichever you prefer. Each option must be separated from the SAMASM command, other options, and any following file names are separated by at least one space.

#### SAMASM.EXE Command-line Options

Options for build or make program:

**-57** S3C7xxx(KS57) series assembler and linker execution
**-86** S3C9xxx(KS86) series assembler and linker execution
**-88** S3C8xxx(KS88) series assembler and linker execution

**-a** Assemble only
**-t** Make program by checking time stamp. Assemble only the files changed from last built, then link the new object files with the unchanged ones for program making.
**-V** Display version number.

Options for assembling:

**-e** Set the maximum number of errors(default 15).
**-E xxx** Output error information into file xxx
**-P xxx** Set macro expanding level in list file (default 64)

Options for linking:

**-b** Generate binary file.
**-B xxx** Input library names from file xxx
**-cx** Set hex file format:
**-cs** Samsung Hex Format (default)
**-ci** Extended Intel Hex Format
**-C xx** Fill blanks in binary file with hexadecimal xx (default FF)
**-g** Generate debugging info file
**-L xxx** Linking with lib file xxx
**-o file** Set the output file name. The outputs are .hex, .lst and .map files.
**-O** Allow linking overlay
**-S** Output SLO format debugging information file
**-m** Generate map file.
**-M** Generate merged list file. This option should be set for debugging.
**-f file** Read object file names from the specified file instead of from command line.

# *Chapter 10* To SAMA User

The Samsung assembler is based on SAMA, however, there are some differences between Samsung re-locatable assembler SASM and SAMA which are as follows:

1. Samsung assembler supports include instead of chip in SAMA, and the device file name extension is changed from .def to .reg. If the target micro-controller is S3C7054(KS57C0504), the program may contain these lines:

```
CHIP      57c0504.def          ;SAMA
.include  "57c0504.reg"        ;SASM57 Assembler
```

   In the new product code system:

```
CHIP      S3C7054.def          ;SAMA
.include  “S3C7054.reg”        ;SASM57 Assembler
```

   Then, Samsung assembler(SASM) searches the SAM57REG environment variable directory first, and then searches current directory.

2. To include a file using .include directive, the file name should begin and end with “ (double quotes ) as follows:

```
.include   "c:\data\int.src"
```

3. Working register names are defined in the SAM8 User's Manual, and you can not use rah or r0ah instead of r10. Especially, using r with label like rLABEL is not permitted. Instead, Samsung assembler (SASM) supports labeling to the working register (SAM8 family only) as follow:

```
; in SAM8 Assembler

TIM:        .reg    r5
            ld      r0,TIM

; in SAMA

TIM:        equ     5
            ld      r0,rTIM
```

4. The .equ directive cannot be used to equate bits of register. If you want to equate a name to a bit, use .bit directive.

```
TBINT     .reg    0EFh,w      ; KS88C0716(S3C8075) register
EBPWM:    .bit    TBINT.0,w   ; bit 0 of TBINT

or

EBPWM:    .bit    0EFh.0,w
```

5. The labels and symbols are case sensitive. For example, the label Main is different from main and MAIN. The register definitions in 88cxxxx.reg are also case sensitive, and the all registers in these files are defined using upper characters.

```
EMT:      .reg    0Feh        ; defined in 88C0716.reg

          ld      Emt,#01h    ; error
          ld      emt,#01h    ; error
          ld      EMT,#01h    ; o.k
```

6. To generate final output, you should link even though there is only one source file. In most cases, your program is consist of several source files more than one.

7. Do not use .org directive unless there is special purpose. Samsung assembler is re-locatable, so your program addresses are determined at link-time not at assemble time.

# *Chapter 11* Limitations

In this section, the limitations of this assembler are described, for example, the length of a label. Where, a file means a unit of assembly and a program means unit of linking - all files in a program.

## 1. Length

| | |
|---|---|
| The characters in a statement | 255 |
| The characters in a operand | 255 |
| The characters in a label | 31 |
| The number of tokens in a statement | 128 |
| The number of operands in a statement | 64 |
| The number of statement of a file | 65535 |

## 2. Macro

| | |
|---|---|
| The number of arguments in a macro | 32 |
| The depth of nested macro | 64 |

## 3. Sections

| | |
|---|---|
| The number of user defined sections in a file | 250 |
| The number of user defined sections in a program | 250 |

## 4. Symbols

| | |
|---|---|
| The number of defined symbols in a file | 4100 |
| The number of external symbols in a file | 4100 |
| The number of global symbols in a file | 4100 |
| The number of global symbols in a program | 12000 |

## 5. Others

| | |
|---|---|
| The depth of nested including files | 8 |
| The depth of nested if-else-endif | 32 |
| The depth of assemble-time operator parsing | unlimited |

# Part IV

## SM2SA Converter

# 4. SM2SA Converter

## Overview

This program makes a program that converts a SAMA's assembly source to SASM's assembly source. We have developed a new assembler for SAM4(KS57Cxxxx, S3C7xxx) series, SASM that is a enhanced version of SAMA.

The more powerful assembler SASM converts SAMA's pseudo-commands to SASM's pseudo-commands. But this conversion has some constraints, for this conversion can't cover all areas connected to assembler parts, such as parsing area.

As though SAM8 assembler is a re-locatable assembler, the files assembled by the SAMA assembler are not re-locatable files that have the advantages of re-locatable assembler. In SAM8, the section command provides the programmer with complete control over the memory mapping of a program without requiring absolute addressing. Therefore, the main purpose of this Converter is rather to introduce the SAM8's programming environment to you than to give a completely converted program.

# *Chapter 1* Using SM2SA Converter

## Execution environment

Running on MS-DOS

## Command line format

- SM2SA[-u|-U|-l|-L] [source_file_name|/?]
- The SAMA's assembly source program, source_file_name, will be converted to SASM's assembly source program.
- -u|-U is an option that converts all the lowercases in source to uppercases except comment.
- -l|-L is an option that converts all the uppercases in source to lowercases except comment.
- -/?
  This parameter displays useful information about using SM2SA.

## Input file

- Original source file that is a file without assembling errors in SAMA.
- All the files that are included in the argument file - main source file - will be also converted automatically.

## Output file

- Output files are typically named filename.cnv where the filename portion is the same as the name of the source file.
- xxx .wrn file which contains converting information.

- Example:

```
C:>SM2SA main.src      ->  main.cnv
                       ->  main.wrn
```

# *Chapter 2* Functions of SM2SA Converter

**1.**

- **SAMA: CHIP**
  CHIP directive specifies the name of a file (.def file) to be loaded by the SAMA assembler.
- **SASM: .include**
  Because SASM assembler uses .reg files instead of using .def files. SAMA's CHIP is replaced by .include. You should have .reg files in the directory in which .def file is located.
- **Example:**

```
        CHIP 88c0716.def
->      .include "88c0716.reg"
```

* note: .include directive uses the double quote as above.

**2.**

- **SAMA: END**
  END directive informs the SAMA assembler the end of source program.
- **SASM: .end**

**3.**

- **SAMA: EQU**
  EQU command assigns values to the symbol for later use.
- **SASM: .equ / .bit**
- **Example:**

| | | | | | | |
|---|---|---|---|---|---|---|
| hours: | equ | 24 | -> | hours: | .equ | 24 |
| mins: | equ | 60 | -> | mins: | .equ | 60 |
| h_m: | equ | hours*mins | -> | h_m: | .equ | hours*mins |
| t_flag: | equ | 55h.1 | -> | t_flag: | .bit | 55h.1 |

* note: equ conversion has some constraints which are described in limitation part.

**4.**

- **SAMA: ORG**
  The ORG command sets the starting address for the blocks of code that follows.
- **SASM: .org**
- **Example:**

| | | | | | |
|---|---|---|---|---|---|
| ORG | 0100H | -> | .org | 0100H | |

**5.**

- **SAMA: REG_ORG**
  The RAM pointer takes the value specified in the operand of the REG_ORG directive.
- **SASM: .ram_org**

**6.**

- **SAMA: DB**
  The DB command causes one or more bytes to be placed in successive location in the code memory.
- **SASM: .db/ .ascii**
- **Example:**

```
            DB      'TIME',TIME_REG+APM,05H

    ->      .ascii  "TIME"
            .db     TIME_REG+APM
            .db     05h
```

**7.**

- **SAMA: DW**
  DW command causes one or more words of data to be placed in successive locations in the code memory.
- **SASM: .dw**
- **Example:**

```
             DW     1234H,5678H,AAA
    ->       .dw    1234H,5678H,AAA
```

**8.**

- **SAMA: DH**
  DH command causes high byte of the address obtained by a label to be placed in the code memory at the current location.
- **SASM: .db<**
- **Example:**

```
            ORG     01234H          ->          .org    01234H
LABEL1:     NOP                     ->LABEL1:   NOP
            LD      EA,#10H         ->          LD      EA,#10H

            DH      LABEL1          ->          .db     <LABEL1
            DL      LABEL1          ->          .db     >LABEL1
```

**9.**

- **SAMA: DL**
  DL command causes low byte of the address obtained by a label to be placed in the code memory at the current location.
- **SASM: .db>**
- **Example:**
  see DH example

**10.**

- **SAMA: DCS**
  The DCS command allocates memory space at the current address for later use.
- **SASM: .blkb**
- **Example:**
  In a list file:

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0100 | 23 | DCS | 10H,23H | -> | 0100 | 23 | .blkb | 10H,23H |
| 0110 | FF | DCS | 10H,FFH | -> | 0110 | FF | .blkb | 10H,FFH |

**11.**

- **SAMA: DRS**
  This directive allocates RAM or Register space at the current RAM address, and increases the RAM address pointer by the value specified in the operand field.
- **SASM: .ram_ds**
- **Example:**

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0040 | | REG_ORG | 40H | -> | 0040 | | .ram_org | 40H |
| 0000 | A | DRS | 1 | -> | 0000 | A | .ram_ds | 1 |
| 0000 | B | DRS | 1 | -> | 0000 | B | .ram_ds | 1 |
| 0000 | E | Drs | 2 | -> | 0000 | E | .ram_ds | 2 |
| 0020 | | ORG | 20H | -> | 0020 | | .org | 20H |
| | | LD | A,B | -> | | | LD | A,B |

**12.**

- **SAMA: INCLUDE**
  The INCLUDE command is used to include other source files in a source program and require a file name in operand field.
- **SASM: .include**
- **Example:**

| | | | |
|---|---|---|---|
| INCLUDE | "head.src" | -> | .include "head.src" |
| Include | C:\source\equate.equ | -> | .include "C:\source\equate.equ" |

**13.**

- **SAMA: TITLE**
  TITLE command allows the user to put his or hers own program title in the header at the top of the output file listing.
- **SASM: Comment**
- **Example:**

```
        TITLE       KS57C0504    SAMPLE PROGRAM
->      ;TITLE      KS57C0504    SAMPLE PROGRAM
```

**14.**

- **SAMA: LISTING**
  The particular part of the source program can be either listed or by not using the LISTING command.
- **SASM: .list**
- **Example:**

```
LISTING     OFF     ->      .list   OFF
LISTING     ON      ->      .list   ON
```

**15.**

- **SAMA: FORM**
  FORM command can change the number of lines of a page.
- **SASM: Comment**
- **Example:**

```
FORM    ON,10H      ->      ;FORM   ON,10H
FORM    OFF         ->      ;FORM   OFF
```

**16.**

- **SAMA: EJECT**
  The EJECT command tells assembler that new page of the list file will begin at current position.
- **SASM: Comment**
- **Example:**

```
EJECT               ->      ;EJECT
```

**17.**

- **SAMA: OBJECT**
  The OBJECT command controls the number of bytes of an instruction to be shown in the list file.
- **SASM: Comment**
- **Example:**

```
OBJECT      ON      ->      ;OBJECT     ON
OBJECT      OFF     ->      ;OBJECT     OFF
```

**18.**

- **SAMA: IF/ELSE/THEN**
  These three commands are used to create conditional assemblies.
- **SASM: .if/.else/.endif**
- **Example:**

```
MODE    EQU     1           ->  MODE    .equ    1
SEC     EQU     60          ->  SEC     .equ    60
MIN     EQU     60          ->  MIN     .equ    60
        IF      MODE=1      ->          .if     MODE==1
HOUR    EQU     12          ->  HOUR    .equ    12
        ELSE                ->          .else
HOUR    EQU     24          ->  HOUR    .equ    24

        THEN                ->          .endif
        LD      R0,#HOUR    ->          LD      R0,#HOUR
```

**19.**

- **SAMA: MACRO/ENDM**
  MACRO command makes the definition of a user defined function. ENDM stands for end of MACRO.
- **SASM: .macro/.endm**
- **Example:**

```
add16   MACRO a1,a2,a3,a4   ->  add16   .macro  a1,a2,a3,a4
        add     a2,a4       ->          add     a2,a4
        adc     a1,a3       ->          adc     a1,a3
        ENDM                ->          .endm
```

**20.**

- **SAMA: .**
  .\ denotes local label in SAMA.
- **SASM: It has no meaning in SASM.**
- **Example:**

```
        .\:cp val,P0        ->  ||:cp   val,P0
```

**21.**

- **SAMA: BINARY/DECIMAL/HEX**
  RADIX command can be substituted by one of these directives and then the base can be specified behind the number.
- **SASM: .radix decimal/ .radix binary/ .radix hex**
- **Example:**

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | DECIMAL | | -> | | .radix | decimal | |
| Q_1 | EQU | 9 | -> | Q_1 | .equ | 9 | |
| | LD | R0, #12 | -> | | LD | R0,#12 | |
| | LD | R0, #12 | -> | | LD | R0,#12H | |
| | LD | R0, #1B | -> | | LD | R0,#1B | |
| | BINARY | | -> | | .radix | binary | |
| A_1 | EQU | 0011 | -> | A_1 | .equ | 0011 | |
| A_2 | EQU | 05H | -> | A_2 | .equ | 05H | |
| A_3 | EQU | 1010.1B | -> | A_3 | .bit | 1010b.1b | |
| A_4 | EQU | 10O ;octa | -> | A_4 | .equ | 10O | ;octal |
| | LD | R4, #00110011 | -> | | LD | R4,#00110011b | |
| | HEX | | -> | | .radix | hex | |
| C_1 | EQU | 05 | -> | C_1 | .equ | 05 | |
| | LD | rA_1,#05H | -> | | LD | r3,#05H | |
| | LD | R9,rA_4 | -> | | LD | R9,r8 | |

# *Chapter 3* Limitations & Warning Messages

1. Arithmetic statement in the declaration of EQU is not allowed in the working register addressing conversion.
   - ◆ Example:

```
AA              EQU     05H
BB              EQU     03H
CC              EQU     AA+BB
…
LD      R0,rCC  -> this line will not be converted as LD        R0,r8
…
```

2. If symbols like HOUR with two or more definition use working register addressing, they are not converted properly.
   - ◆ Example:

```
MODE            EQU     1
…
                IF      MODE = 1
HOUR            EQU     2
…
                ELSE
HOUR            EQU     4
…
                THEN
                LD      R0, rHOUR
```

   * The warning message is displayed at the second HOUR.
     -> HOUR already exist.

3. For the following two cases. You must convert source program manually.

   Case 1) You must insert comma between two operands.
   - ◆ Example:

```
LD      EA 05H          ->      LD      EA,05H
VENT0   1 0 RESET       ->      VENT0   1,0,RESET
```

   Case 2) You must change comma to dot in bit operand.
   - ◆ Example:

```
BITS    51,2H           ->      BITS    51.2H
```

# *Chapter 4* Converter Example

## Before converting

main.src

```
Chip c:skStudio\Include\Def\88c0716.def
        ...
include c:\temp\equ.tbl
        ...
include c:\temp\lcd.tbl
        ...
end
```

equu.tbl

lcd.tbl

## After converting

main.cnv

```
.include “c:\skStudio\Include\Reg\88c0716.reg “
                ...
.include    “c:\temp\equ.cnv”
                ...
.include    “c:\temp\lcd.cnv”
                ...
.end
```

equu.cnv

lcd.cnv

main.wrn

```
.include “c:\skStudio\Include\Def\88c0716.reg”
            ...
.include “c:\temp\equ.cnv”

        --   equu.cnv --
             ...
.include “c:\temp\lcd.cnv”

        --   lcd.cnv --
             ...

.end
```

# Appendix

# Samsung Assembler Error Messages

- **DUPLICATED LABEL**
  Labels are duplicated.
- **UNDEFINED SYMBOL IN OPERAND**
  Undefined symbol or label is used in operand field.
- **UNDEFINED MNEMONIC**
  Undefined symbol is used in opcode field.
- **NO. OF OPERANDS MISMATCH**
  The number of operands is mismatched.
- **OPERAND SYNTAX PARSING**
  Operand parsing error. E.g. 1*,(1+2
- **MISSING LABEL**
  Label is missed in label field.
- **LABEL VALUE CHANGED BETWEEN PASSES**
  Label value is changed between pass 1 and pass 2.
- **ILLEGAL READ/WRITE MODE**
  The attribute of .reg is not one of r, w, rw and wr.
- **UNKNOWN RADIX VALUE**
  Radix value is not one of hex, decimal, or binary.
- **UNKNOWN ADDRESSING MODE**
  An expression represents an addressing mode not available on the SAM8, such as add r0, #1[r4]

- **BAD VALUE**
  Illegal value is used to represent some directives, such as r: .reg $
- **DIVIDED BY ZERO**
  You have tried to divide by 0.
- **BAD CONDITION CODE**
  The condition codes described in SAM8 User's manual are not used.
- **DO NOT USE RR0 IN INDEXED ADDRESSING MODE**
  LDE and LDC instructions can not be used rr0 as base register in indexed addressing mode.
- **UNBALANCED ENDIF**
  .If is used but not followed by a matching .endif.
- **IF-ELSE DEPTH OVERFLOW**
  The depth of nested .if/.else/.endif exceeds limit.
- **MACRO DEPTH OVERFLOW**
  The depth of nested .macro/.endm exceeds limit.
- **INCLUDE DEPTH OVERFLOW**
  The depth of nested .include exceeds limit.
- **INCLUDE FILE OPEN FAILED**

Included file can not be opened.

◆ **exceeds NULL POINTER**
Something is wrong in assembler.

◆ **EXTERN LABEL IS DEFINED ALREADY**
A label is redefined in this module.

◆ **1ST OPERAND READ VIOLATION**
You have tried to read write-only register in first operand.

◆ **2ND OPERAND READ VIOLATION**
You have tried to read write-only register in second operand.

◆ **1ST OPERAND WRITE VIOLATION**
You have tried to write read-only register in first operand.

◆ **2ND OPERAND WRITE VIOLLATION**
You have tried to write read-only register in second operand.

◆ **MUST BE ONLY ONE SYMBOL**
The operand field must have only one symbol token.

◆ **ILLEGAL OPERAND SYNTAX**
The operand syntax is illegal.

◆ **RELATIVE JUMP BETWEEN SECTIONS**
You have tried to jump relatively to different section.

◆ **RELATIVE JUMP TO EXTERN SYMBOL**
You have tried to jump relatively to external label.

◆ **TOO MANY SECTIONS ARE DEFINED**
The number of section definitions exceeds limit.

◆ **ROM SIZE EXCEED**
Your program exceeds the ROM code size.

◆ **ORG VALUE IS SMALLER THAN CURRENT LOCATION COUNT**
The value in .org is smaller than current location counter.

◆ **MISSING .ENDM**
The .endm (macro end) directive is omitted.

◆ **.ENDM IS USED WITHOUT .MACRO**
The .endm is used without .macro(beginning of macro).

◆ **ILLEGAL SECTION OPTION**
The options for .section is illegal.

◆ **OUT OF RELATIVE JUMP RANGE**
The jump range is out of bound.

◆ **.END DIRECTIVE IS USED IN INCLUDED FILE**
The .end directive is used in the included file that is not in main file.

◆ **.ENDIF IS NOT DEFINED UNTIL END OF FILE**
The .endif is not defined until end of file.

◆ **ILLEGAL CHARACTER IN NUMBER CONSTANT**
An illegal character is in a number constant.

◆ **MISSING \') \'**
The expression contains unmatched ()'s.

◆ **NO MATCH MACRO ARGUMENTS**
Number of macro arguments is not matched.

◆ **OUT OF RAM RANGE**
Out of RAM definition range.

◆ **NOT DECLARED RAM_ORG**
The ram_ds directive is used without declaration of ram_org directive.

◆ **NOT ALLOWED FORWARD REFERENCE IN .REG STATEMENT**
If .reg statement is defined later and that symbol is reference beforehand, it is an error because the reference symbol can not be the working register or general register.

◆ **ONLY ALLOWED ADD AND SUBTRACT OPERATION FOR EXTERNAL SYMBOL**
In arithmetic operation for external symbol, it is permitted for the following operation.
Addition(+), Subtract(-), High byte(<), and Low byte(>).